

【递归转换为非递归的方法---1) 递归方法改迭代方法】

$$Fib(n) = \begin{cases} n & \text{若 } n = 0 \text{ 或 } 1 \\ Fib(n-1) + Fib(n-2) & \text{其它} \end{cases}$$

采用递归算法 --- 从顶到底

从n问题开始，为解决n的问题，先解决n-1问题

```
long fib(int n)
{
    if( n == 1 || n == 0)
        return n;
    else
        return fib(n-1)
            +fib(n-2);
}
```

递归算法复杂度:

估算为: $T(n)=2^nT(0)+(2^n-1)=2^n+2^n-1=O(2^n)$

精确计算为: $T(n)=\frac{1}{\sqrt{5}}\left[\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n\right]$

采用迭代算法 --- 从底到顶

从1问题开始，逐步解决n的问题

```
long fib2(int n)
{
    if( n == 1 || n == 0)
        return n;

    int rN0=0, rN1=1, rN2;
    rN2 = rN0 + rN1;
    for(int i=3; i<= n; i++)
    {
        rN0 = rN1;
        rN1 = rN2;
        rN2 = rN0+rN1;
    }
    return rN2;
}
```

递推算法复杂度: $O(n)$

2)对递归算法改写—通用方法（利用栈模拟递归）

以求 $n!$ 为例，用栈模拟递归函数的调用、返回

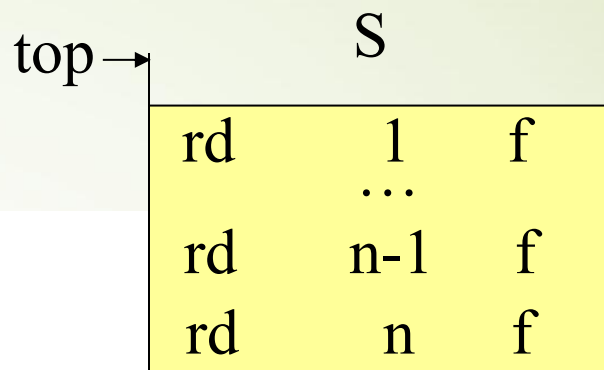
```
int fact(int n)
{  int f;
   InitStack(S);
0:  if (n==0) f=1;
   else {
1:  f = fact(n-1);
2:  f=n*f;
   }
   DestroyStack(S);
   return f;
} //fact
```

```
//函数调用是将本层的返回地址、
//返回值保存变量、参数值入栈
//将程序执行转向函数入口点
//入栈：(2是返回地址，n是参数，
//      f是保存返回值的)
Push( S, (2, n, f));
//为下一层递归准备参数n-1
n=n-1;
//转到下层函数入口执行
goto 0;
```

```
if ( !StackEmpty(S) )
{
   //模拟函数返回，把返回值装入栈顶
   (S.top-1)->f = f;
   //弹出本层参数
   Pop(S, (rd, n, f));
   //转到递归返回处执行
   goto rd ;
}
```

• 自设栈模拟系统工作栈

```
typedef struct {  
    int  rd; //递归返回后执行的程序标号  
    int  n;  //本层参数  
    int  f;  //保存返回值用  
}SElemType;
```



● 递归改写方法

- ➡ 在算法开头增加栈的初始化函数调用语句
- ➡ 改写递归调用语句 {
 - 本次参数、返回值保存变量及返回地址入栈处理；
 - 将参数调整为下层参数值
 - 转至调用的开始语句
- ➡ 改写所有递归出口 {
 - 保存返回值
 - 退栈还原参数（包括返回地址、返回值）；
 - 转至返回地址处
- ➡ 在算法结束增加栈销毁函数调用语句

```

int fact(int n)
{ int f;
  InitStack(S);
0: if (n==0) f=1;
  else {
1:  Push( S, (2, n, f ));
    n=n-1;
    goto 0;
2:  f =n*f;
  }
  if ( !StackEmpty(S) )
  {
    (S.top-1)->f=f;
    Pop(S, (rd, n, f) );
    goto rd ;
  }
  DestroyStack(S);
  return f;
} //fact

```

把2个goto结构
改成while循环

```

int fact(int n)
{ int f;
  InitStack(S);
  while (n!=0)
  {
    Push (S, n);
    n=n-1;
  }
  f=1;
  while ( !StackEmpty(S) )
  {
    Pop (S, n);
    f=n*f;
  }
  DestroyStack(S);
  return f;
} //fact

```

注：此时可确定栈中只存放n值即可，变参不需栈处理。

```

typedef int
SElemType;

```

2)对递归算法改写一通用方法（利用栈模拟递归）

$$Fib(n) = \begin{cases} n & \text{若 } n = 0 \text{ 或 } 1 \\ Fib(n-1) + Fib(n-2) & \text{其它} \end{cases}$$

采用递归算法 --- 从顶到底

从n问题开始，为解决n的问题，先解决n-1问题

```
long fib(int n)
{
0: if( n == 1 || n == 0)
    { fn1 = n; fn2 = 0; }
    else
    {
1:   int fn1= fib(n-1)
2:   int fn2= fib(n-2);
    }
3:   ret = fn1+fn2;
   return ret;
}
```

```
typedef struct {
    int rd; //递归返回后
            //执行的程序标号
            // 2 或者 3
    int n;   //本层参数
    int nReturn; //从那个调用
            //分支返回，
            //LEFT 或者 RIGHT
    int fL;   //保存左返回值用
    int fR;   //保存右返回值用
}SElemType;
```

2)对递归算法改写一通用方法 (利用栈模拟递归)

$$Fib(n) = \begin{cases} n & \text{若 } n = 0 \text{ 或 } 1 \\ Fib(n-1) + Fib(n-2) & \text{其它} \end{cases}$$

采用递归算法 --- 从顶到底

从n问题开始, 为解决n的问题,
先解决n-1问题

```
long fib(int n)
{
0: if( n == 1 || n == 0)
    { fn1 = n; fn2 = 0; }
    else
    {
1:   int fn1 = fib(n-1);
2:   int fn2 = fib(n-2);
    }
3: ret = fn1 + fn2;
   return ret;
}
```

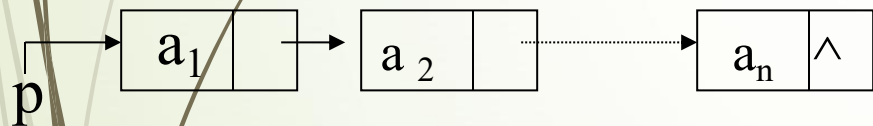
Push(S, (2, n, LEFT, fL, fR)); n=n-1; goto 0;

Push(S, (3, n, RIGHT, fL, fR)); n=n-1; goto 0;

if((S.top-1)->return == LEFT)
 (S.top-1)->fL = fn1 + fn2;
if((S.top-1)->return == RIGHT)
 (S.top-1)->fR = fn1 + fn2;
Pop(S, (rd, n, NONE, fn1, fn2));
goto rd;

3) 消除尾递归 例：顺序输出单链表中的结点数据

```
void linklist_output(LNode *p);  
{  
    if (p)  
    {  
        printf(p->data);  
        linklist_output(p->next)  
    }  
} //linklist_output
```



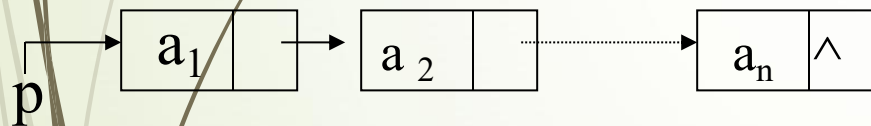
使用通用方法改写

```
void linklist_output1(LNode *p)  
{  
0:   if ( p )  
    {   printf(p->data);  
1:       //入栈, push  
        p=p->next;  
        goto 0;  
2:       // 空语句  
    }  
    //无返回值需要处理  
    //出栈, pop  
    goto 2;  
} //linklist_output1
```

尾递归：如果一个函数中所有递归形式的调用都出现在函数的末尾，我们称这个递归函数是尾递归的。当递归调用是整个函数体中最后执行的语句且它的返回值不属于表达式的一部分时，这个递归调用就是尾递归。尾递归函数的特点是在回归过程中不用做任何操作，这个特性很重要，因为大多数现代的编译器会利用这种特点自动生成优化的代码。

3) 消除尾递归 例：顺序输出单链表中的结点数据

```
void linklist_output(LNode *p);  
{  
    if (p)  
    {  
        printf(p->data);  
        linklist_output(p->next)  
    }  
} //linklist_output
```



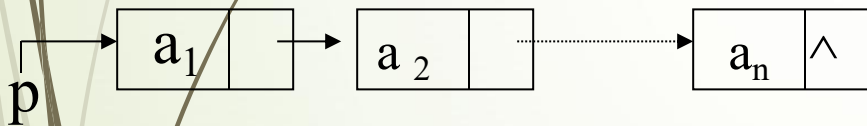
使用通用方法改写

```
void linklist_output1(LNode *p)  
{  
0:   if ( p )  
    {   printf(p->data);  
        p=p->next;  
        goto 0;  
    }  
} //linklist_output1
```

尾递归：如果一个函数中所有递归形式的调用都出现在函数的末尾，我们称这个递归函数是尾递归的。当递归调用是整个函数体中最后执行的语句且它的返回值不属于表达式的一部分时，这个递归调用就是尾递归。尾递归函数的特点是在回归过程中不用做任何操作，这个特性很重要，因为大多数现代的编译器会利用这种特点自动生成优化的代码。

3) 消除尾递归 例：顺序输出单链表中的结点数据

```
void linklist_output(LNode *p);  
{  
    if (p)  
    {  
        printf(p->data);  
        linklist_output(p->next)  
    }  
} //linklist_output
```

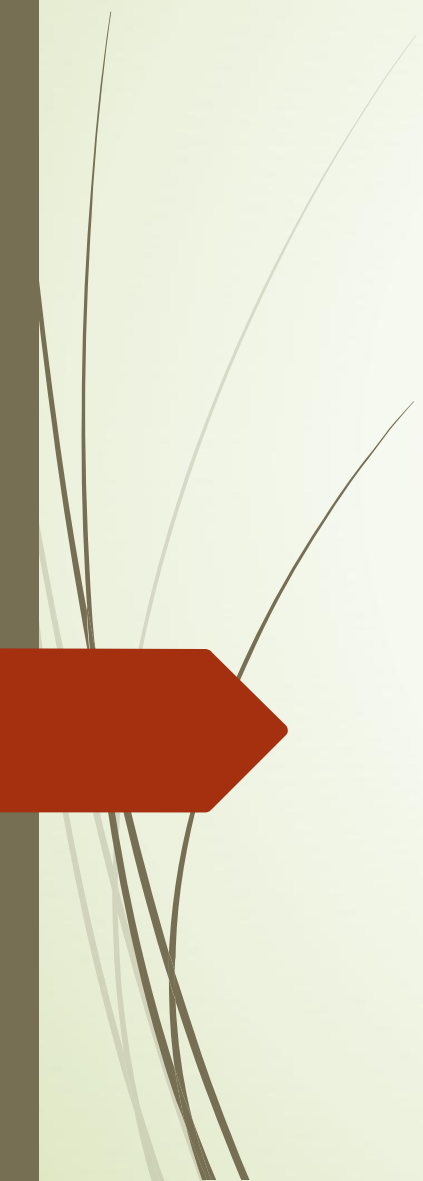


使用循环语句结构替换goto结构:

```
void linklist_output2(LNode *p)  
{  
    while ( p )  
    {  
        printf(p->data);  
        p=p->next;  
    }  
} //linklist_output2
```

尾递归消除，采用通用方法第一步；由于没有出栈，入栈也无需进行了。

尾递归：如果一个函数中所有递归形式的调用都出现在函数的末尾，我们称这个递归函数是尾递归的。当递归调用是整个函数体中最后执行的语句且它的返回值不属于表达式的一部分时，这个递归调用就是尾递归。尾递归函数的特点是在回归过程中不用做任何操作，这个特性很重要，因为大多数现代的编译器会利用这种特点自动生成优化的代码。



第6章 树和二叉树

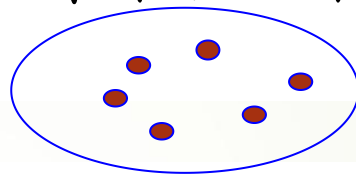
概念和二叉树的遍历

逻辑结构

方面 层次	数据表示	数据处理
抽象	逻辑结构	基本运算
实现	存储结构	算法
评价	不同数据结构的比较及算法分析	

划分方法一

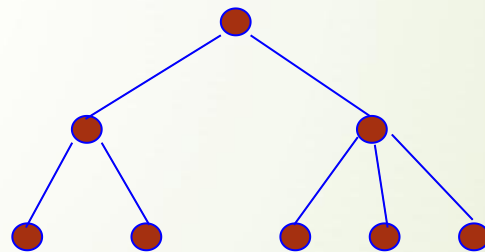
集合——数据元素间除“同属于一个集合”外，无其它关系
如，一堆沙子



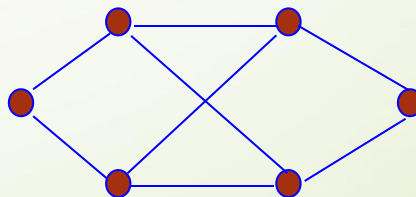
线性结构——一个对一个，如线性表、栈、队列
如，一根链条，一个单词中的所有字母



树形结构——一个对多个，如树
如，一棵树，家谱树等



图形结构——多个对多个，如图
如，交通图，高铁图



存储结构

方面 层次	数据表示	数据处理
抽象	逻辑结构	基本运算
实现	存储结构	算法
评价	不同数据结构的比较及算法分析	

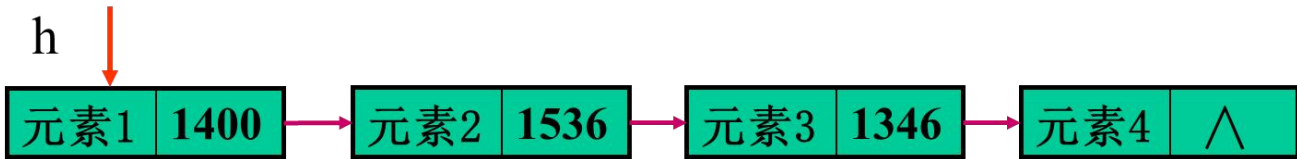
顺序存储结构——

借助元素在存储器中的**相对位置**
表示数据元素间的逻辑关系（如前后关系）
相对位置可通过计算得到

存储地址	存储内容
L_0	元素1
L_0+m	元素2
	
$L_0+(i-1)*m$	元素i
	
$L_0+(n-1)*m$	元素n

链式存储结构——

借助指示元素存储地址的**指针**
表示数据元素间的逻辑关系（如前后关系），
需通过遍历（依次访问）链表得到指定元素的存储位置



第6章 树和二叉树结构

- ➡ 6.1 树的定义和基本术语
- ➡ 6.2 二叉树
- ➡ 6.3 二叉树的遍历
- ➡ 二叉树遍历的应用
- ➡ 6.4 线索二叉树
- ➡ 6.5 哈夫曼树（最优二叉树）
- ➡ 6.6 树和森林

1. 教学内容

二叉树和树的概念、二叉树的遍历及其应用、线索二叉树、哈夫曼树及哈夫曼编码、树和森林与二叉树之间的转换、树或森林的遍历。

2. 教学目的

深刻理解二叉树的定义、性质及其存储方法；
熟练掌握二叉树的二叉链表存储方式、结点结构和类型定义；
掌握二叉树的三种遍历算法；了解二叉树的线索化方法；
灵活运用二叉树的遍历方法解决相关的应用问题；
掌握森林与二叉树间的相互转换；
掌握树和森林的遍历方法。

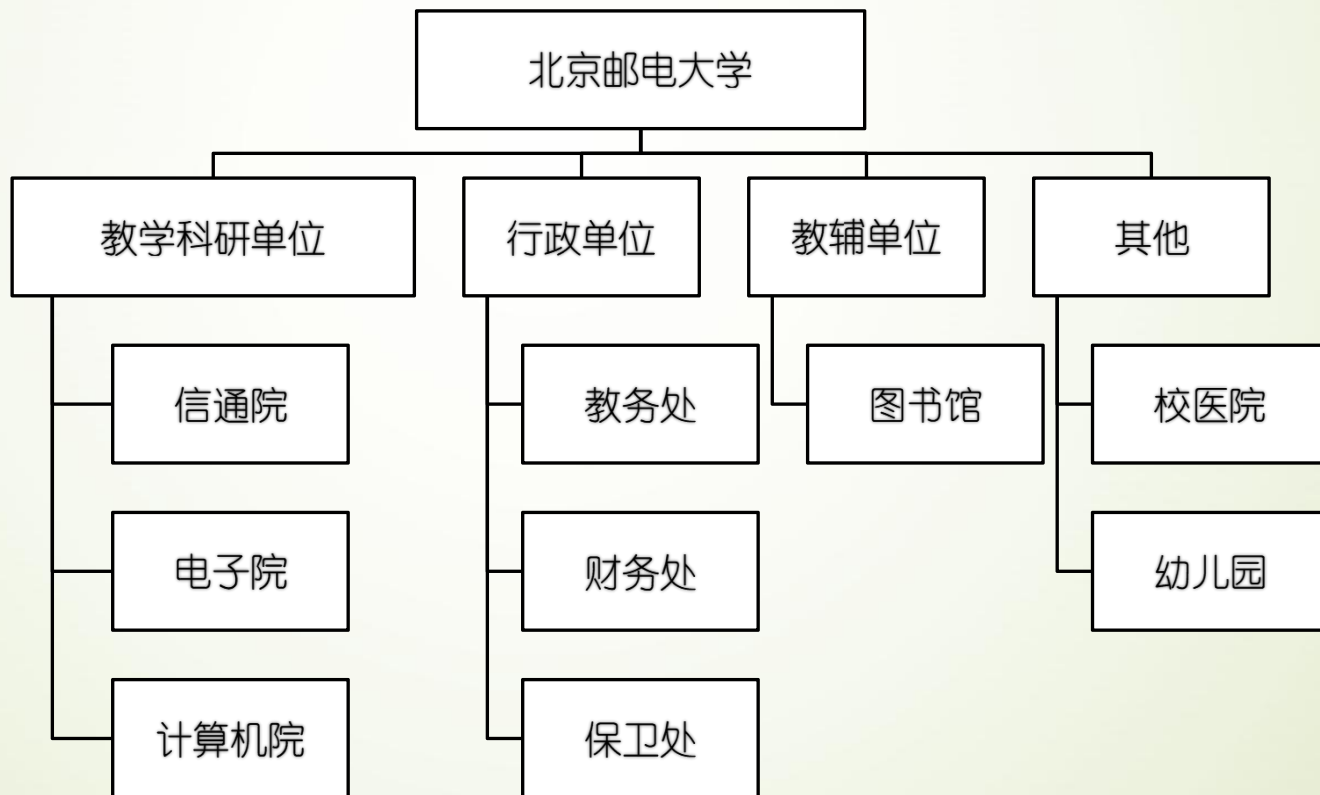
教学目标

1. 掌握二叉树的基本概念、性质和存储结构
2. 熟练掌握二叉树的前、中、后序遍历方法
3. 熟练掌握：哈夫曼树（最优二叉树）的实现方法、构造哈夫曼编码的方法
4. 掌握：线索二叉树、森林与二叉树的转换，树的遍历方法

问题提出

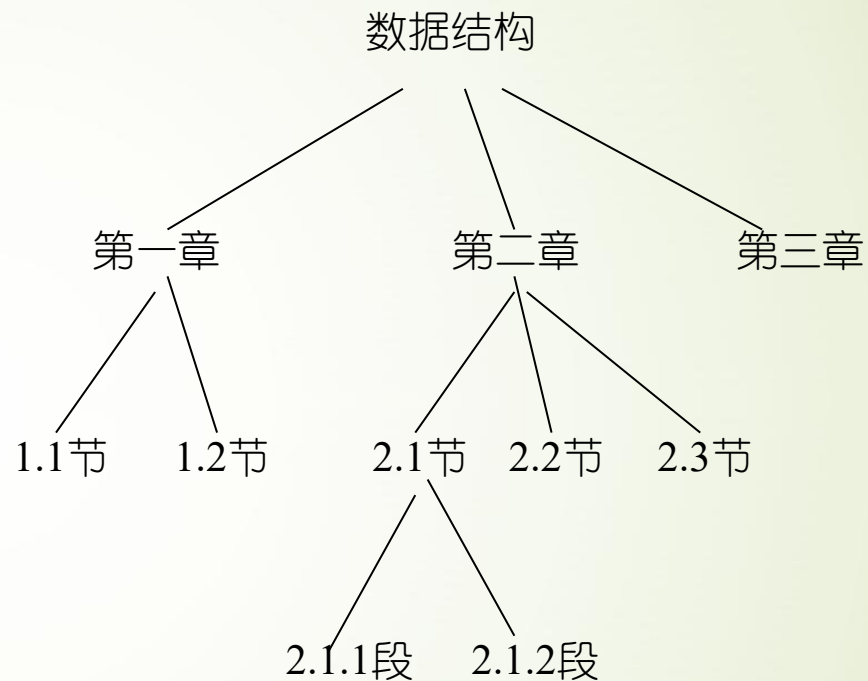
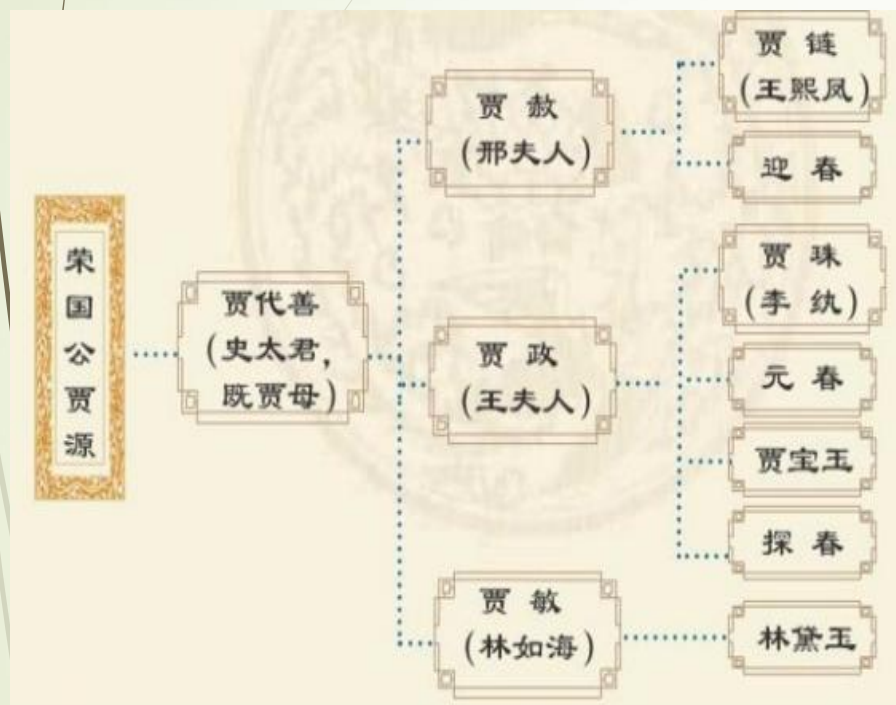
6.1 定义

线性结构是指元素之间至多有一个前驱元素或一个后继元素的情况，然而在现实生活中或数学抽象中还有一种情况是元素**至多有一个前驱元素**，而**可有多个后继元素**的情况，称之为**树结构**。

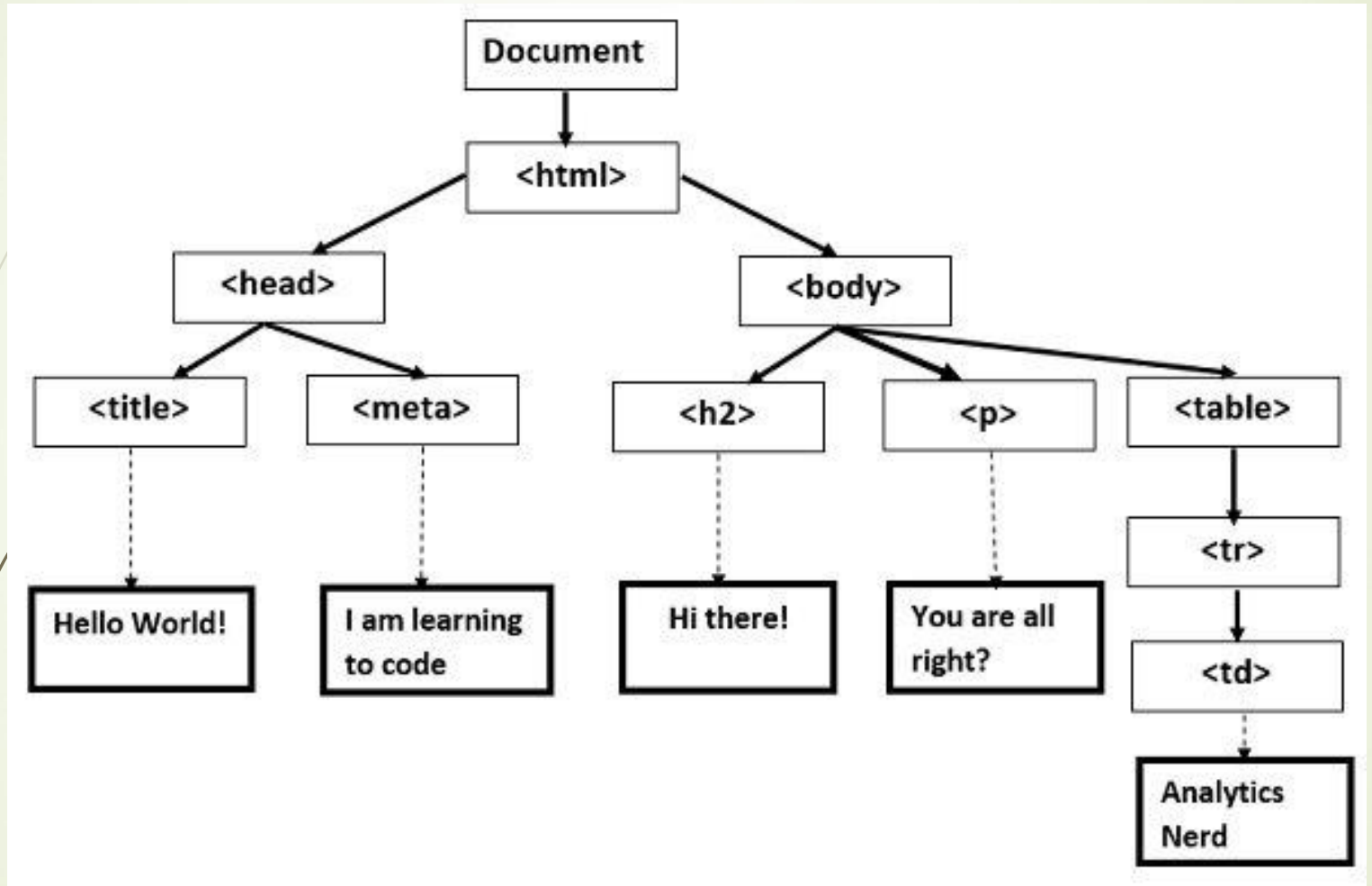


问题提出

6.1 定义



问题提出



问题提出

6.1 定义

0-1背包问题：给定n种物品和一背包。物品i的重量是 w_i ，其价值为 p_i ，背包的容量为C。问应如何选择装入背包的物品，使得装入背包中物品的总价值最大？

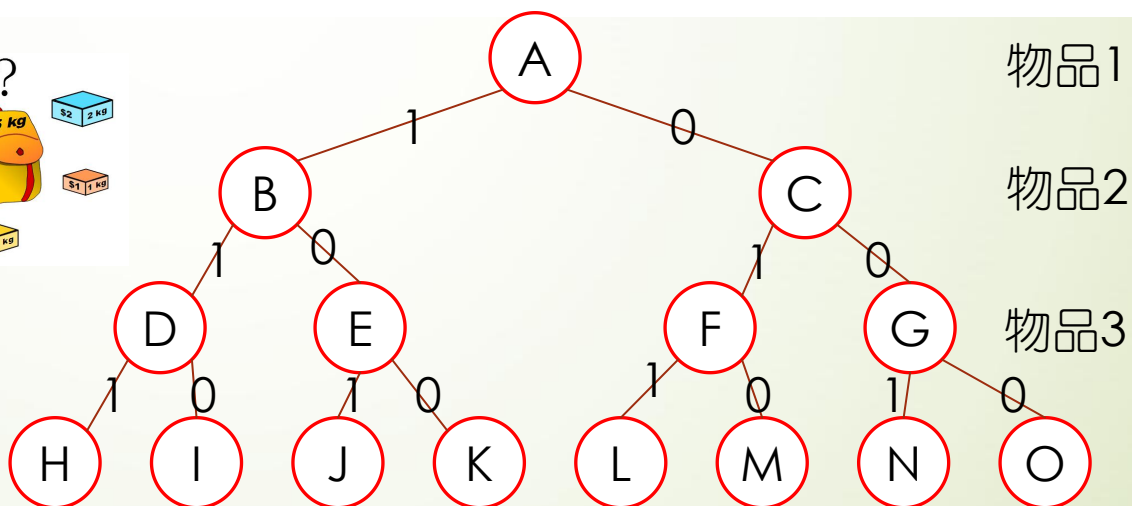
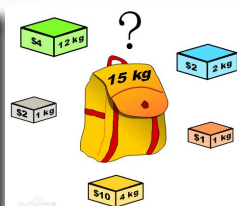
0-1背包问题是一个特殊的整数规划问题，出现在各种领域的现实世界的决策过程中，例如寻找最少浪费的方式来削减原材料，选择投资和投资组合等。

[用搜索树来求解]

- 1) 定义一个解空间（搜索树），它包含问题的解
- 2) 用适合于搜索的方式组织该空间
- 3) 在约束条件下先序遍历（深度优先）搜索该解空间，并在遍历过程中剪去那些不满足条件的分支。

$$\max \sum_{i=1}^n p_i x_i$$

$$\begin{cases} \sum_{i=1}^n w_i x_i \leq C \\ x_i \in \{0,1\}, 1 \leq i \leq n \end{cases}$$



该问题的3个物品的解空间如上图所示

6.1 树的定义---递归定义

6.1 定义

树是一类重要的非线性数据结构，是以分支关系定义的层次结构。

树 (Tree) 是 n ($n \geq 0$) 个结点的有限集，它或为空树 ($n = 0$)；或为非空树，对于非空树 T :

- (1) 有且仅有一个称之为根的结点；
- (2) 除根结点以外的其余结点可分为 m ($m > 0$) 个互不相交的有限集 $T_1, T_2, \dots, T_m$ ，其中每一个集合本身又是一棵树，并且称为根的子树 (SubTree)。

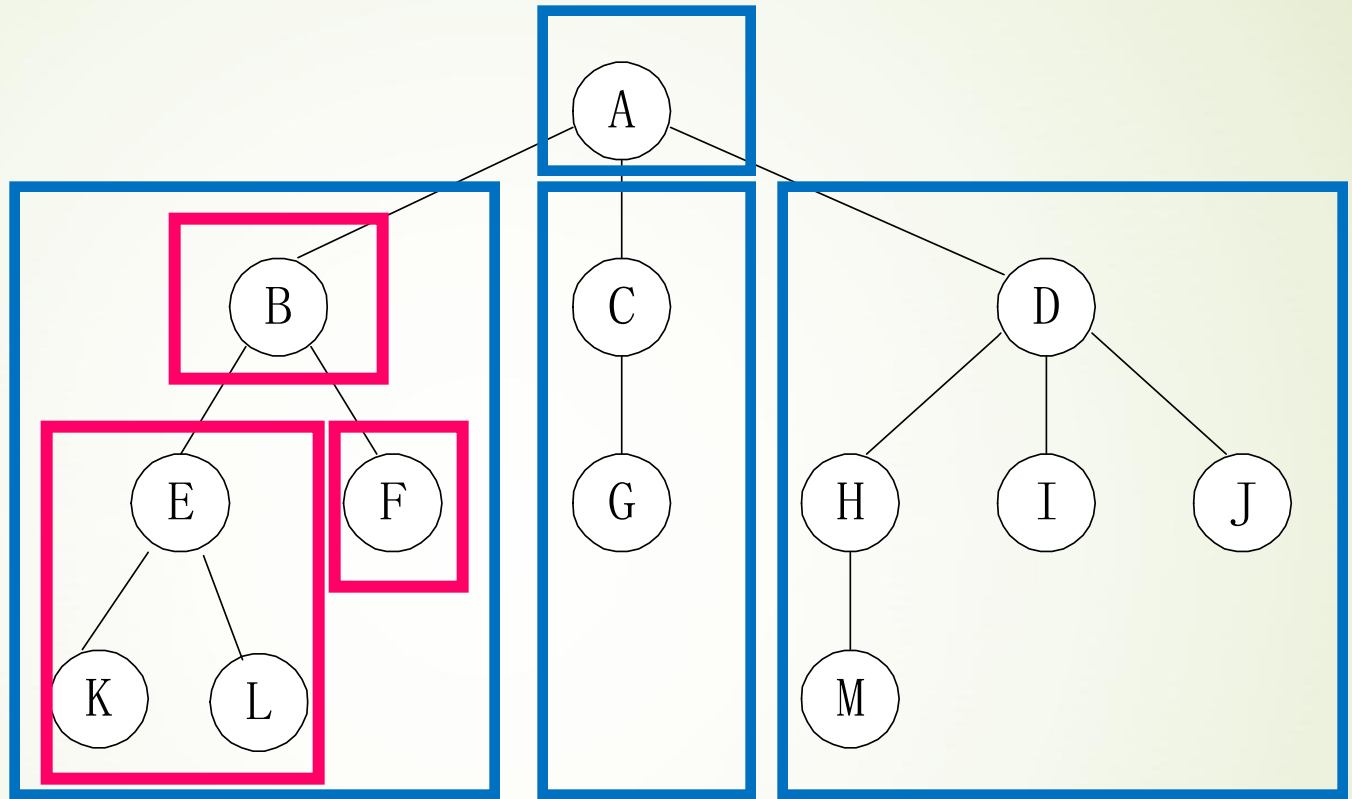
树的递归定义刻画了树的固有特性，即一棵非空树是由若干棵子树构成的，而子树又可由若干棵更小的子树构成。

树是n个结点的有限集

6.1 定义



T_0



T_1

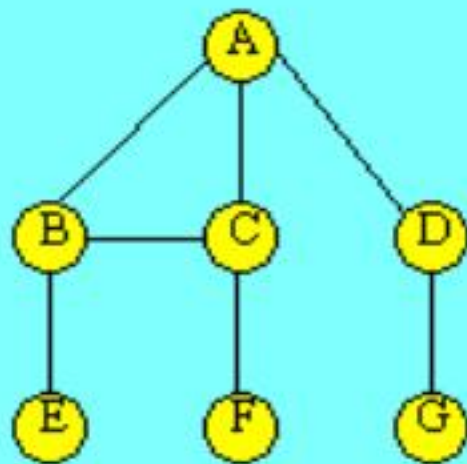
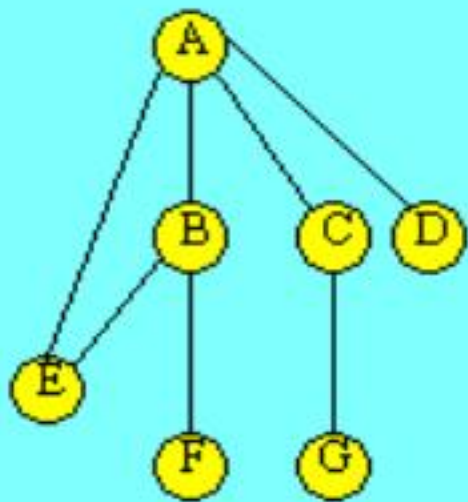
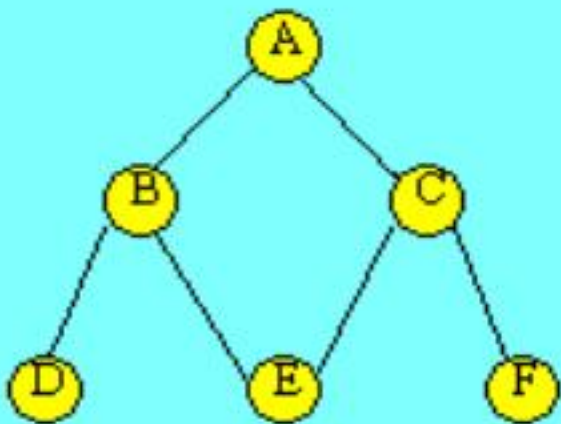
T_2

T_3

树的定义

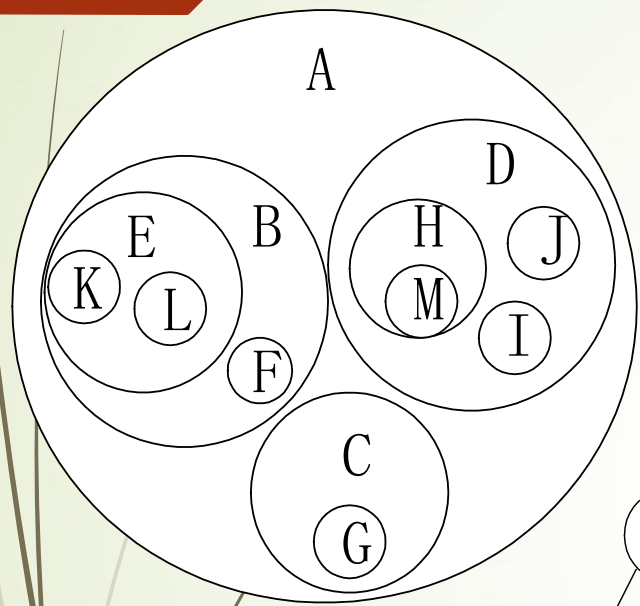
6.1 定义

- 树具有下面两个特点：
 - (1)树的根结点没有前驱结点，除根结点之外的所有结点有且只有一个前驱结点。
 - (2)树中所有结点可以有零个或多个后继结点。
- 由此特点可知，下图所示的都不是树结构。

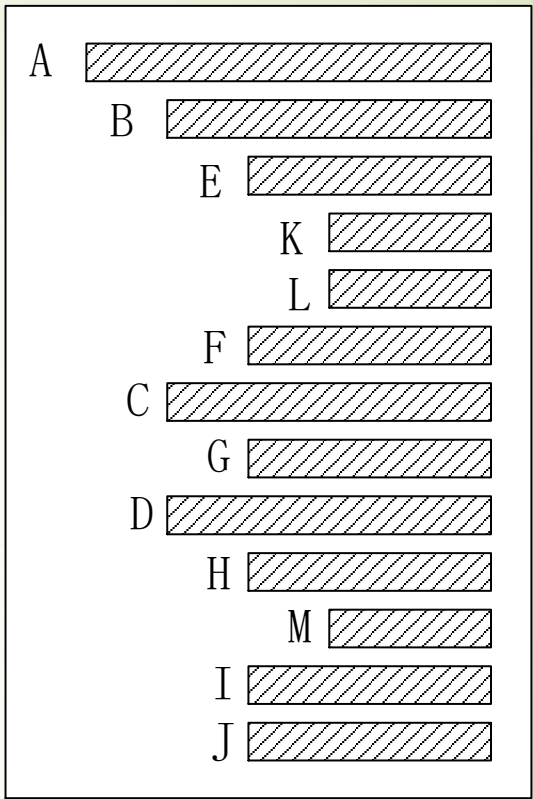
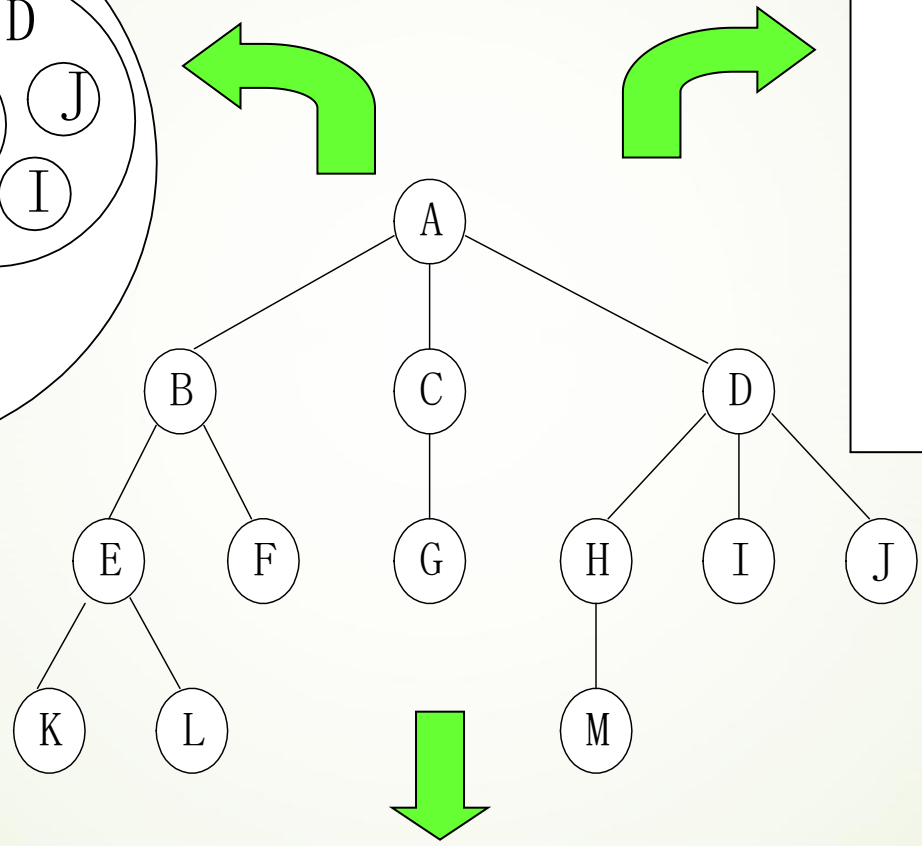


树的其它表示方式（逻辑结构）

6.1 定义



嵌套集合



凹入表示

$(A(B(E(K, L), F), C(G), D$

广义表

树的基本术语

6.1 定义

结点的度 结点拥有的子树数目。

叶子(终端)结点 度为0的结点。

分支(非终端)结点 度不为0的结点。

树的度 树的所有结点的度的最大值。

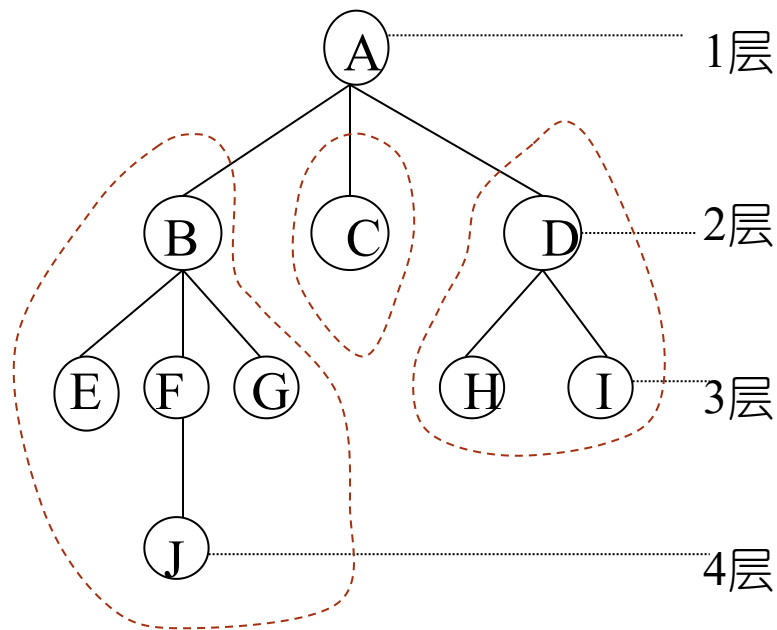
内部结点 除根结点之外的分支结点。

双亲与孩子(父与子)结点 结点的子树的根称为该结点的孩子；该结点称为孩子的双亲。

兄弟 属于同一双亲的孩子。

结点的祖先 从根到该结点所经分支上的所有结点。

结点的子孙 该结点为根的子树中的任一结点。



树的基本术语

6.1 定义

结点的层次 表示该结点在树中的相对位置。

根为第一层，其它的结点依次下推；

若某结点在第 L 层上，则其孩子在第 $L+1$ 层上。

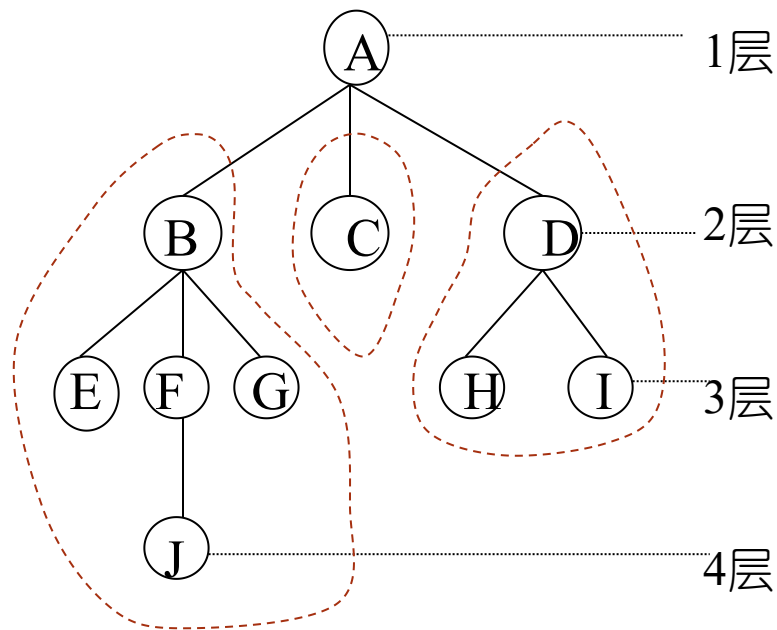
堂兄弟 双亲在同一层的结点互为堂兄弟。

树的深(高)度 树中结点的最大层次。

有序树 树中各结点的子树从左至右是有次序的，不能互换。否则，称为**无序树**。

路径长度 从树中某结点 N_i 出发，能够通过树中结点到达结点 N_j ，则称 N_i 到 N_j 存在一条路径，路径长度等于这两个结点之间的分支/边的个数。

森林 是 $m(m \geq 0)$ 棵互不相交的树的集合。



树的基本操作 (ADT)

6.1 定义

- | | |
|---------------------|--|
| 1) 构造空树 | <code>InitTree(&T)</code> |
| 2) 销毁已存在的树 | <code>DestroyTree(&T)</code> |
| 3) 将树置为空树 | <code>ClearTree(&T)</code> |
| 4) 求树的深度 | <code>TreeDepth(T)</code> |
| 5) 求树的根 | <code>Root(T)</code> |
| 6) 求指定结点的双亲 | <code>Parent(T, Cur_e)</code> |
| 7) 求指定结点的最左孩子 | <code>LeftChild(T, Cur_e)</code> |
| 8) 求指定结点的右兄弟 | <code>RightSibling(T, Cur_e)</code> |
| 9) 插入子树c作为结点p的第i棵子树 | <code>InsertChild(&T, &p, i, c)</code> |
| 10) 删除p结点的第i棵子树 | <code>DeleteChild(&T, &p, i)</code> |
| 11) 遍历操作 | <code>TraverseTree(T, Visit())</code> |

二叉树的定义---递归定义

6.1 定义

二叉树 (Binary Tree) 是 n ($n \geq 0$) 个结点所构成的集合, 它或为空树 ($n = 0$); 或为非空树, 对于非空树 T :

- (1) 有且仅有一个称之为根的结点;
- (2) 除根结点以外的其余结点分为两个互不相交的子集 T_1 和 T_2 , 分别称为 T 的左子树和右子树, 且 T_1 和 T_2 本身又都是二叉树。

引入二叉树是作为一般树 (森林) 的规范形式, 它与树 (森林) 可以建立一一对应, 为解决树的问题提供方便。

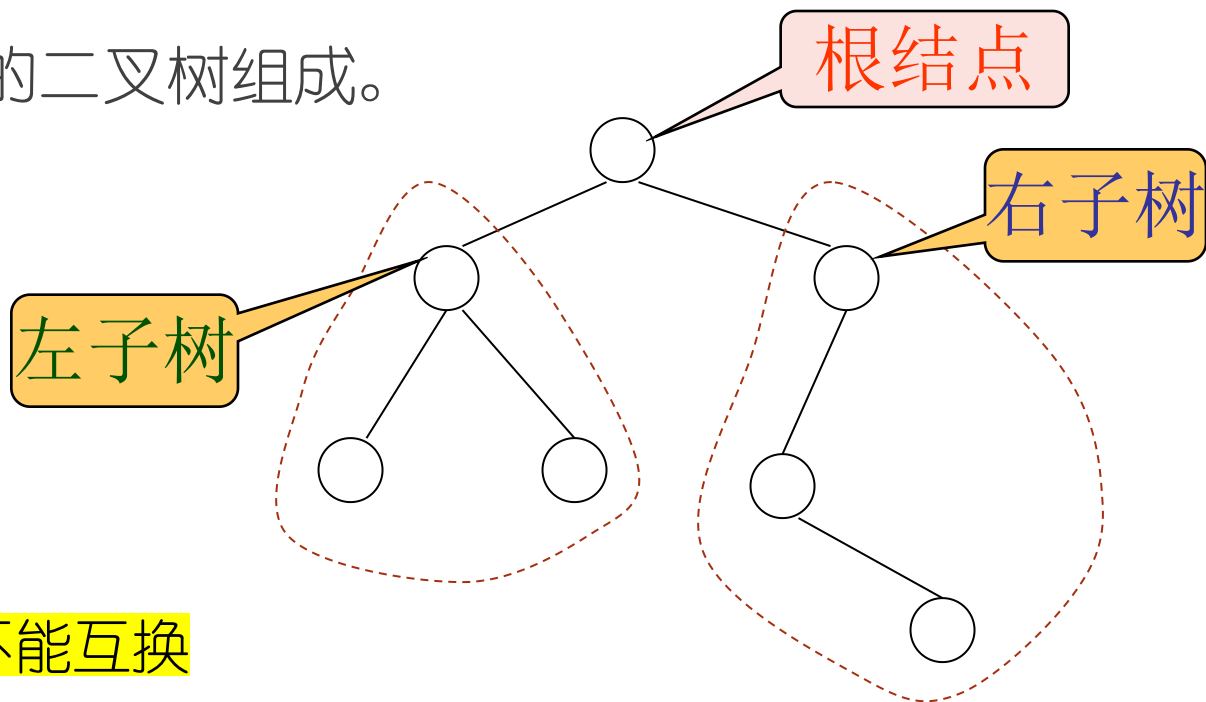
6.2.1 二叉树的概念

6.2 二叉树

二叉树的定义 二叉树是 $n(n \geq 0)$ 个结点的有限集合，它或为**空树** ($n=0$)，或由一个**根**结点和两棵互不相交的被分别称为根的**左子树**和根的**右子树**的二叉树组成。

二叉树的特点

- ➡ 定义是递归的
- ➡ $0 \leq \text{结点的度} \leq 2$
- ➡ 是有序树，左右不能互换



二叉树**每个结点至多有两棵子树**，子树有左右之分，其**次序**不能任意颠倒。

6.2.1 二叉树的概念

6.2 二叉树

- 二叉树结点的子树要区分左子树和右子树，即使只有一棵子树也要进行区分，说明它是左子树，还是右子树。这是二叉树与树的最主要的差别。
- 下图列出二叉树的5种基本形态，图(c) 和图 (d) 是不同的两棵二叉树。

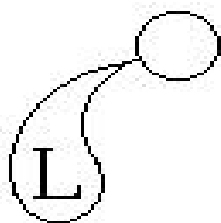
二叉树的五种不同形态

$\emptyset$

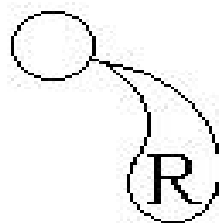
(a)



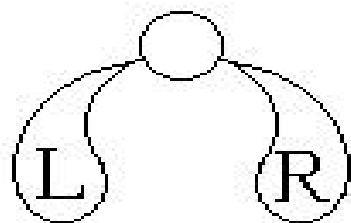
(b)



(c)



(d)



(e)

空树

仅有根
节点的
二叉树

右子树
为空的
二叉树

左子树
为空的
二叉树

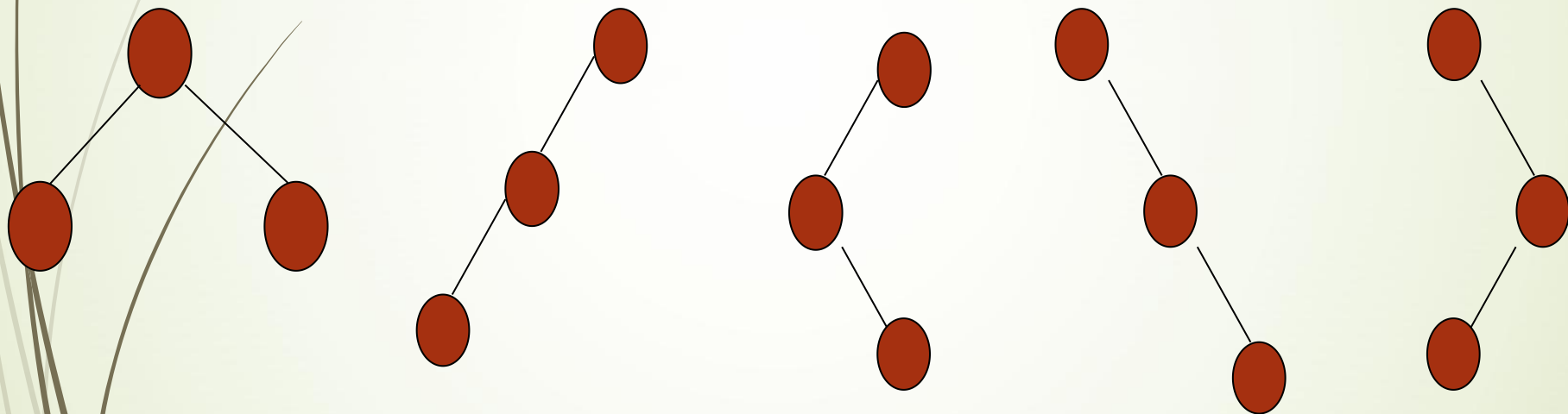
左、右子
树均非空
的二叉树

练习

6.2 二叉树

具有3个结点的**二叉树**可能有几种不同形态? **5种**

具有3个结点的**普通树**有几种可能的形态? **2种**



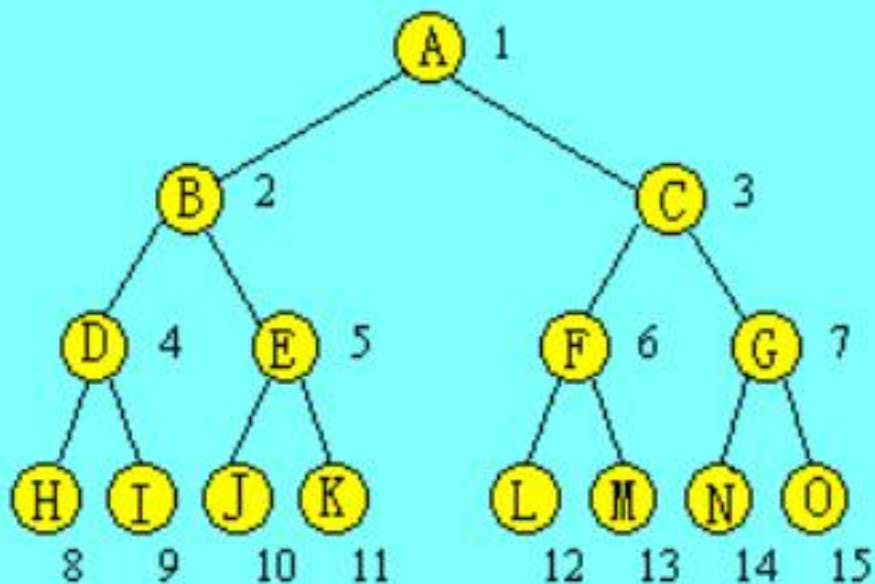
为什么普通树有2种? 二叉树有5种?

二叉树是有序树。

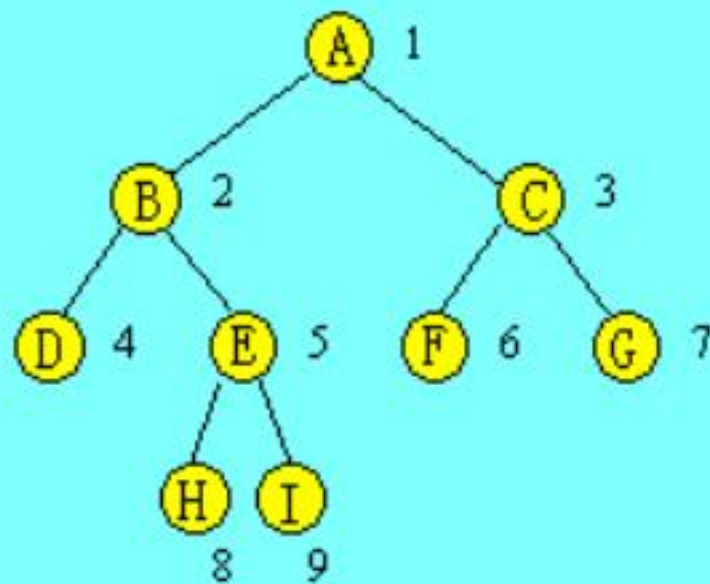
6.2 二叉树

满二叉树:

如果一棵二叉树每一层的结点个数都达到了最大，这棵二叉树称为满二叉树。对于满二叉树，所有的分支结点都存在左子树和右子树，所有叶子都在最下面一层。一棵深度为 k 的满二叉树有 2^k-1 个结点（由二叉树的性质2可得出）



(a) 一棵满二叉树



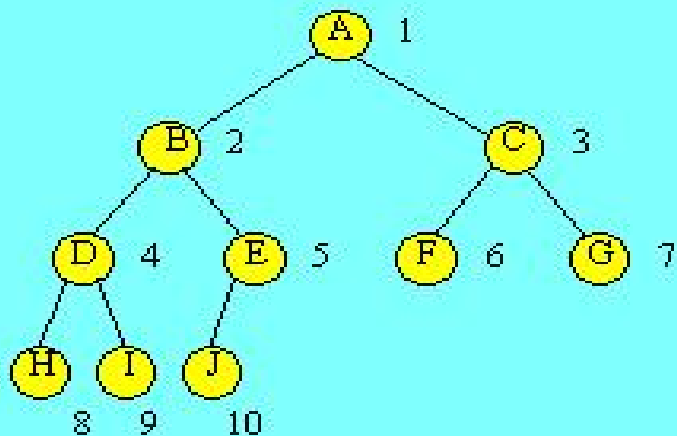
(b) 一棵非满二叉树

完全二叉树:

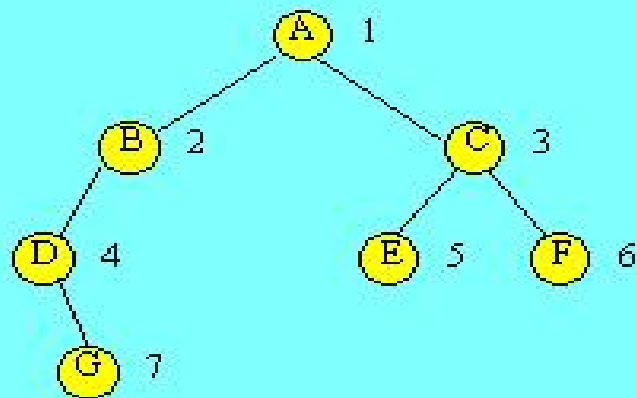
6.2 二叉树

一棵深度为 k 的有 n 个结点的二叉树，对其结点按从上至下，从左到右的顺序进行编号，如果编号为 i 的结点与满二叉树中编号为 i 的结点在二叉树中的位置相同，则这棵二叉树称为完全二叉树。

完全二叉树的特点是叶子结点只能出现在最下层和次最下层，并且最下面一层上的结点都集中在该层最左边的若干位置上，至多只有最下面的两层上结点的度数可以小于2，满二叉树肯定是完全二叉树，而完全二叉树未必是满二叉树。

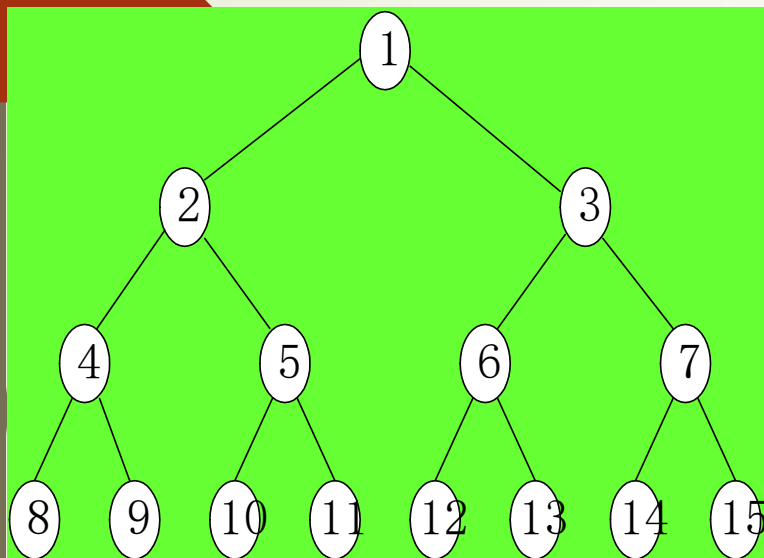


(a) 一棵完全二叉树



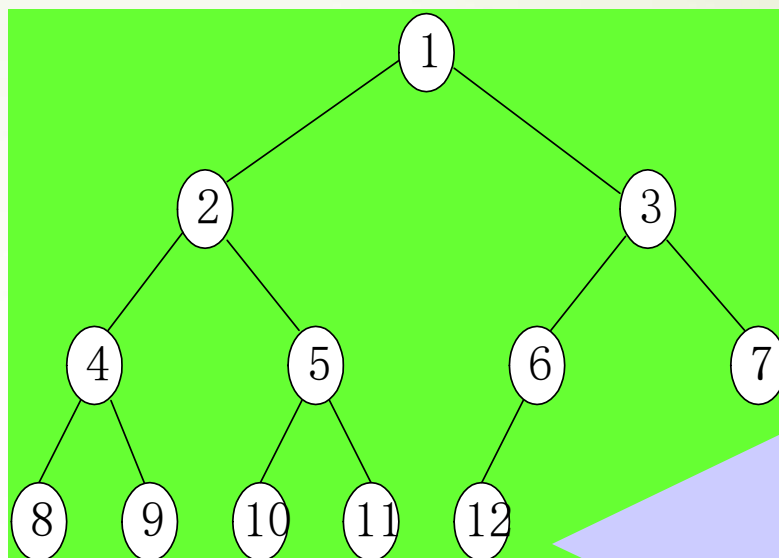
(b) 一棵非完全二叉树

特殊形态的二叉树



满二叉树：一棵深度为 k 且有 $2^k - 1$ 个结点的二叉树。
没有度为1的结点。

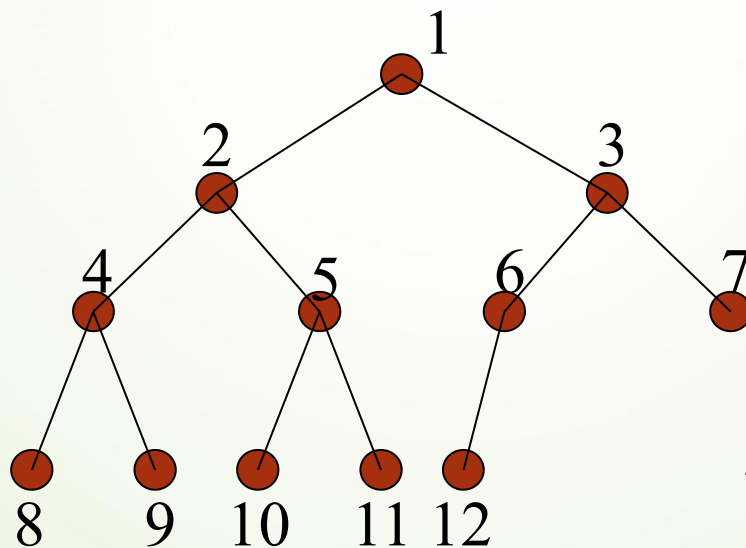
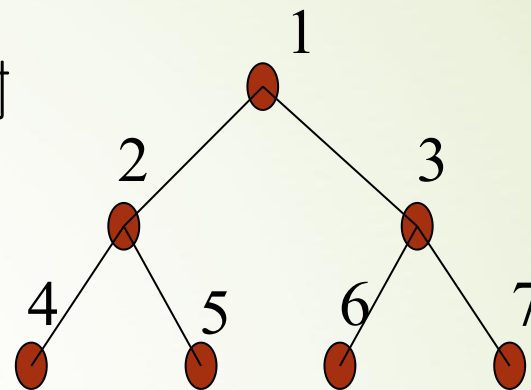
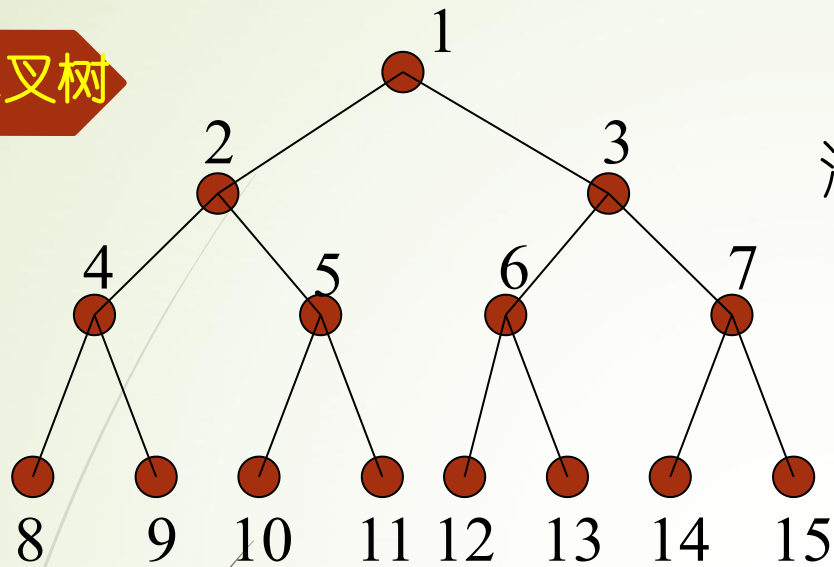
(特点：每层都“充满”了结点)



完全二叉树：深度为 k 的，有 n 个结点的二叉树，当且仅当其每一个结点都与深度为 k 的满二叉树中编号从1至 n 的结点——对应

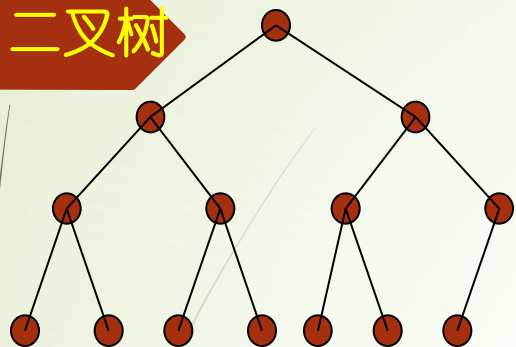
只有最后一层叶子不满，且全部集中在左边，度为1的结点0个或1个

6.2 二叉树

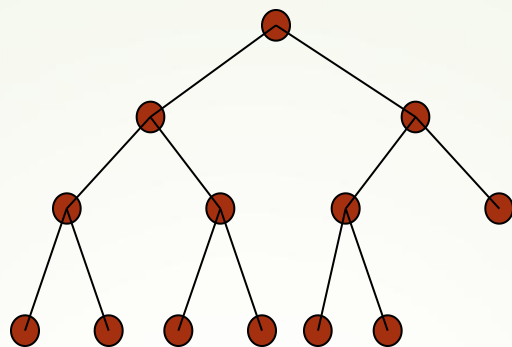


完全二叉树

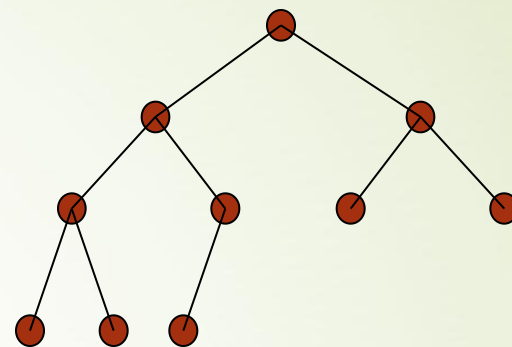
6.2 二叉树



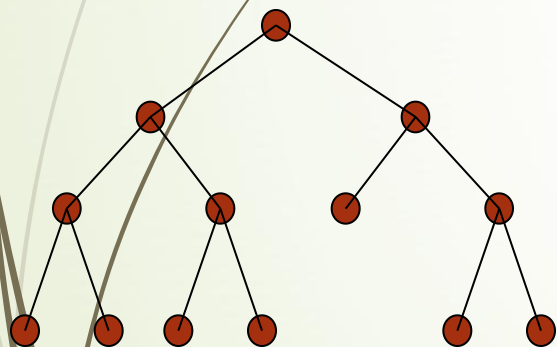
完全二叉树



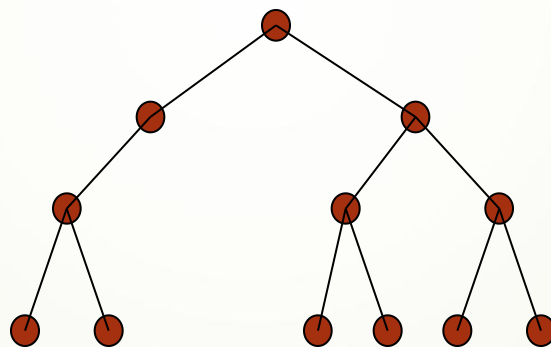
完全二叉树



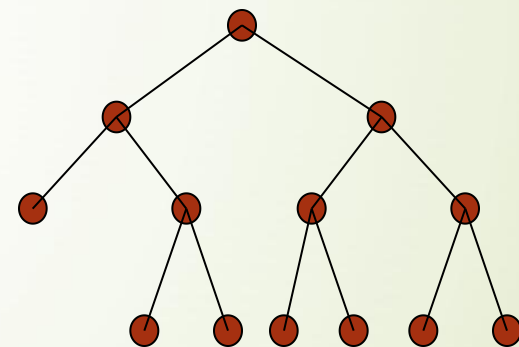
完全二叉树



非完全二叉树



非完全二叉树



非完全二叉树

6.2.2 二叉树的性质

6.2 二叉树

性质1: 在二叉树的第 i 层上至多有 2^{i-1} 个结点

证明：数学归纳法

$i=1$ 时，只有一个根结点， $2^0=1$ 正确；

假设 $i=k$ 时命题成立，即第 k 层最多有 2^{k-1} 个结点，由于二叉树每个结点最多有两个孩子结点，因此第 $k+1$ 层的结点数最多为第 k 层的两倍，即 $2 \times 2^{k-1} = 2^{(k+1)-1}$

提问：第 i 层上至少有 **1** 个结点？

6.2.2 二叉树的性质

6.2 二叉树

性质2: 深度为 k 的二叉树至多有 2^k-1 个结点

证明:

由性质1得: k 层的二叉树全部结点数为:

$$1+2^1+2^2+2^3+\dots+2^{k-1}=2^k-1$$

提问: 深度为 k 时至少有 **k** 个结点?



用1分2 的耳机线可以
同时给2个人听。

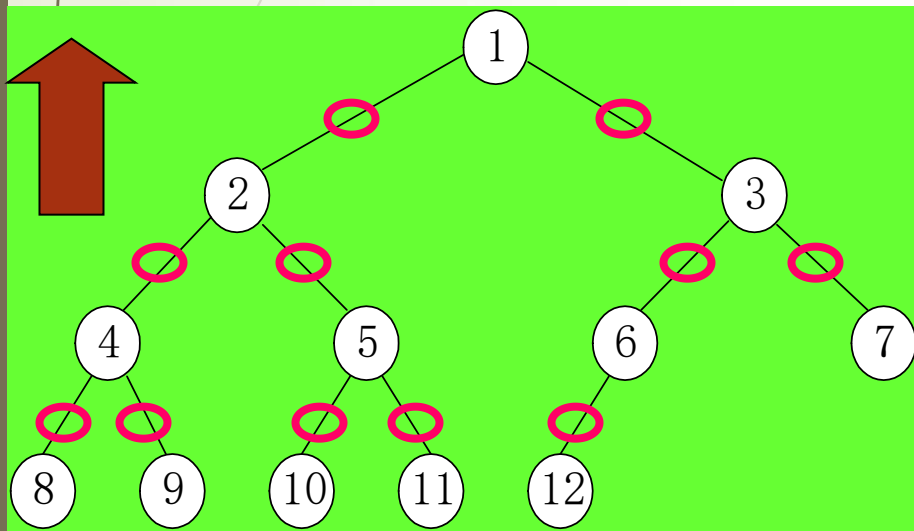
给 n 个人同时听，
需要几个这样的耳
机线？

问题转换为：
 n 个叶子结点的二
叉树有多少个非叶
子结点？

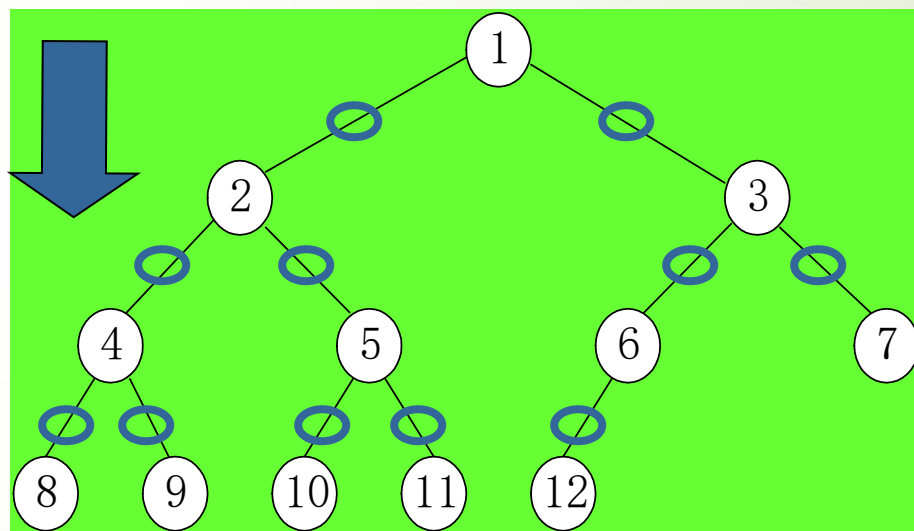
6.2.2 二叉树的性质

性质3: 对于任何一棵二叉树，若2度的结点数有 n_2 个，则叶子数 n_0 必定为 $n_2 + 1$ （即 $n_0 = n_2 + 1$ ）

设 B 为树中的连接（分支）的个数，设 n_1 为二叉树的度为1的结点个数



A) $B = n_0 + n_1 + n_2 - 1$



B) $B = n_2 \times 2 + n_1 \times 1$

由A)、B)得: $n_0 + n_1 + n_2 - 1 = n_2 \times 2 + n_1 \times 1$

得: $n_0 = n_2 + 1$

练习

6.2 二叉树

一棵完全二叉树有5000个结点，可以计算出其叶结点的个数是（ **2500** ）。

性质3: 对于任何一棵二叉树，若2度的结点数有 n_2 个，则叶子数 n_0 必定为 $n_2 + 1$ （即 $n_0 = n_2 + 1$ ）

由性质3结合完全二叉树的性质（ n_1 的个数为0或1）：

$$n_0 = n_2 + 1 \text{ 且 } n_0 + n_2 = 5000 \quad (1)$$

$$\text{或 } n_0 = n_2 + 1 \text{ 且 } 1 + n_0 + n_2 = 5000 \quad (2)$$

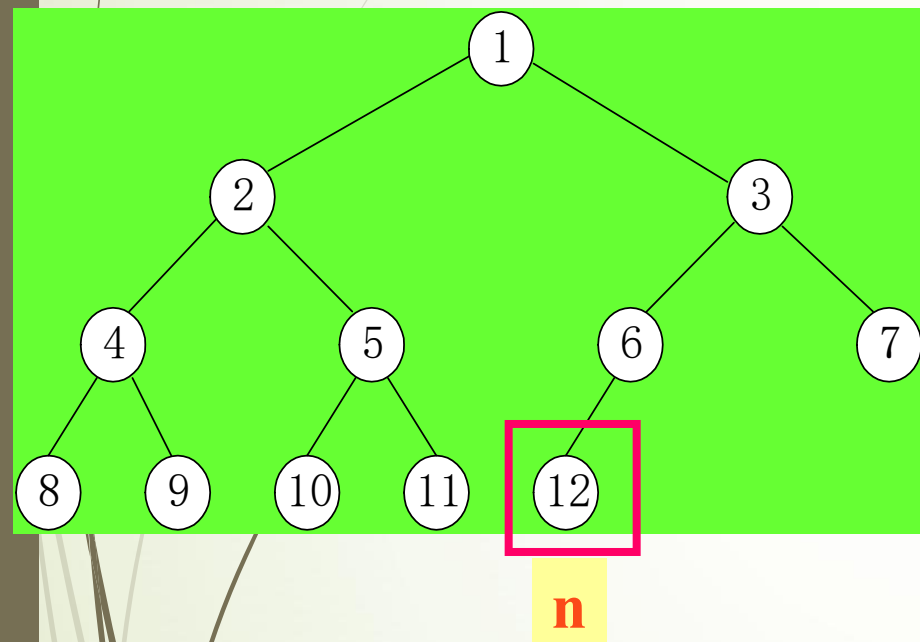
分别解2个方程组，(1)无正整数解，不成立；(2)成立，且 $n_0 = 2500$

当结点是5001时，5002时，叶子数分别是多少？
画一画分析一下

6.2.2 二叉树的性质

6.2 二叉树

性质4:具有n个结点的完全二叉树的深度为 $\lfloor \log_2 n \rfloor + 1$ 。



k-1层，由性质2，
结点数最多为：

$$2^{k-1} - 1$$

k层，由性质2，
结点数最多为：

$$2^k - 1$$

证明：假设深度为k，结点数为n，根据完全二叉树的定义和性质2：

n最小为 $2^{k-1} - 1 + 1$ ，n最大为 $2^k - 1$ ，

有 $2^{k-1} \leq n \leq 2^k - 1$ ，即 $2^{k-1} \leq n < 2^k$ 。对不等式取对数，则：

$k-1 \leq \log_2 n < k$ ， $k-1 = \lfloor \log_2 n \rfloor$ ， $k = \lfloor \log_2 n \rfloor + 1$ (或 $\lceil \log_2 (n+1) \rceil$ ，自行证明一下)

6.2.2 二叉树的性质

6.2 二叉树

性质5:一棵具有 n 个结点的完全二叉树(又称顺序二叉树), 对其结点按层从上至下(每层从左至右)进行1至 n 的编号, 则对任一结点 i ($1 \leq i \leq n$)有:

(1) 若 $i > 1$, 则 i 的双亲是 $\lfloor i/2 \rfloor$; 若 $i = 1$, 则 i 是根, 无双亲。

(2) 若 $2i \leq n$, 则 i 的左孩子是 $2i$; 否则, i 无左孩子。

(3) 若 $2i+1 \leq n$, 则 i 的右孩子是 $2i+1$; 否则, i 无右孩子。

构造顺序存储的二叉树时, 计算结点位置

6.2.2 二叉树的性质---构造顺序存储的二叉树时计算结点位置

性质1 二叉树的第 i 层上至多有 2^{i-1} ($i \geq 1$)个结点。

性质2 深度为 k 的二叉树至多有 $2^k - 1$ 个结点($k \geq 1$)。

性质3 对任何一棵二叉树 T ，如果其终端结点数为 n_0 ，度为2的结点数为 n_2 ，
则 $n_0 = n_2 + 1$ 。

性质4 具有 n 个结点的完全二叉树的深度为 $\lfloor \log_2 n \rfloor + 1$ 。

性质5 一棵具有 n 个结点的完全二叉树(又称顺序二叉树)，对其结点按层从上至下(每层从左至右)进行1至 n 的编号，则对任一结点 i ($1 \leq i \leq n$)有：

(1) 若 $i > 1$ ，则 i 的双亲是 $\lfloor i/2 \rfloor$ ；若 $i = 1$ ，则 i 是根，无双亲。

(2) 若 $2i \leq n$ ，则 i 的左孩子是 $2i$ ；否则， i 无左孩子。

(3) 若 $2i + 1 \leq n$ ，则 i 的右孩子是 $2i + 1$ ；否则， i 无右孩子。

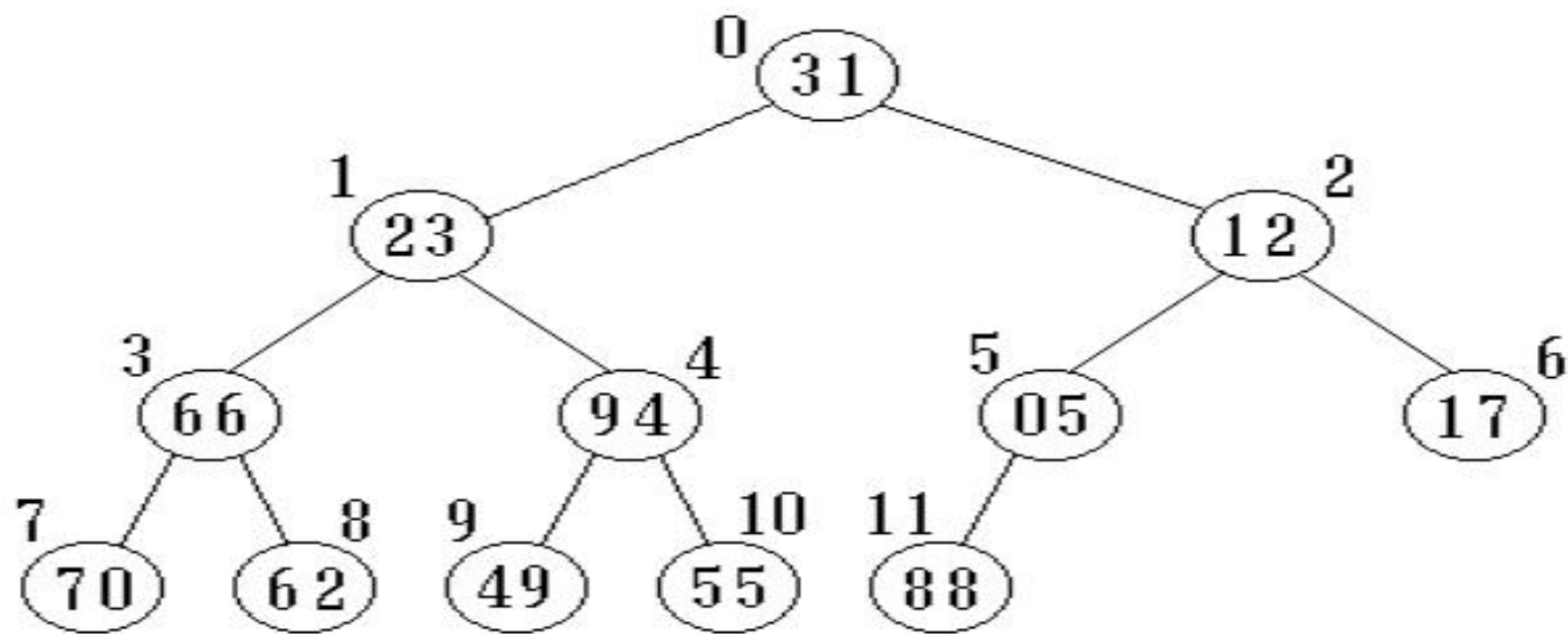
二叉树的基本操作 ADT

- | | |
|--------------------|--|
| 1)构造空二叉树 | InitBiTree(&T) |
| 2)销毁已存在的二叉树 | DestroyBiTree(&T) |
| 3)将二叉树置为空 二叉树 | ClearBiTree(&T) |
| 4)求二叉树的深度 | BiTreeDepth(T) |
| 5)求二叉树的根 | Root(T) |
| 6)求指定结点的双亲 | Parent(T, e) |
| 7)求指定结点的左/右孩子 | LeftChild(T, e)/ RightChild(T, e) |
| 8)求指定结点的左/右兄弟 | LeftSibling(T, e) / RightSibling(T, e) |
| 9)插入子树c作为结点p的左/右子树 | InsertChild(&T,&p,LR,c) |
| 10)删除p结点的左/右子树 | DeleteChild(&T,&p,LR) |
| 11)遍历操作 | TraverseBiTree(T, Visit()) |

二叉树的顺序存储

6.2 二叉树

实现：按**满二叉树**的结点层次编号，用一组连续的存储单元依次存放二叉树中的数据元素。



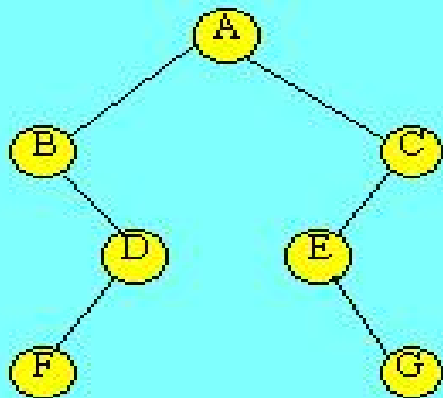
(a)

0	1	2	3	4	5	6	7	8	9	10	11
31	23	12	66	94	05	17	70	62	49	55	88

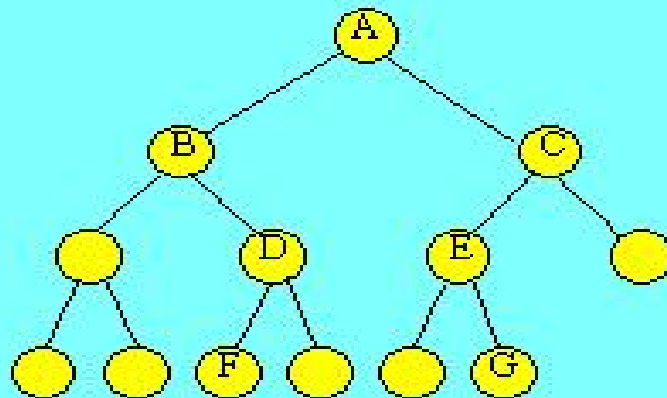
二叉树的顺序存储

6.2 二叉树

对于一般的二叉树，如果仍按从上至下和从左到右的顺序将树中的结点顺序存储在一维数组中，则**数组元素下标之间的关系不能够反映二叉树中结点之间的逻辑关系**，只有增添一些并不存在的空结点，使之成为一棵完全二叉树的形式，然后再用一维数组顺序存储。



(a) 一棵二叉树



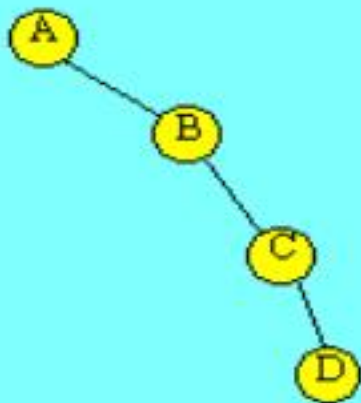
(b) 改造后的完全二叉树

A	B	C	^	D	E	^	^	^	F	^	^	G
---	---	---	---	---	---	---	---	---	---	---	---	---

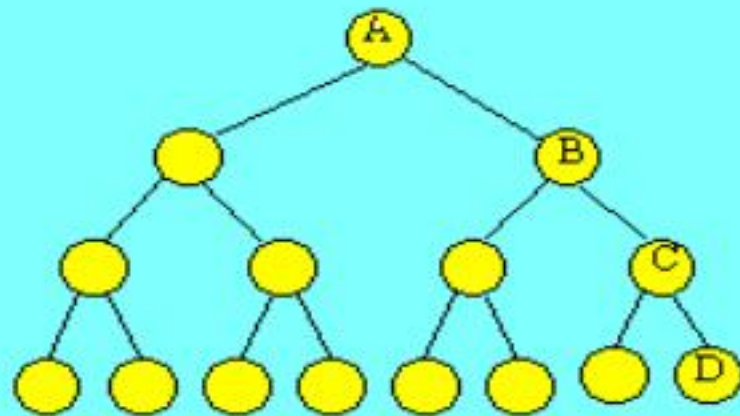
二叉树的顺序存储

6.2 二叉树

显然，完全二叉树和满二叉树采用顺序存储比较合适。对于需增加许多空结点才能将一棵二叉树改造成为一棵完全二叉树的存储时，会造成空间的大量浪费，不宜用顺序存储结构。极端情况是单支树，如一棵深度为 k 的右单支树，只有 k 个结点，却需分配 $2^k - 1$ 个存储单元。



(a) 一棵右单支二叉树



(b) 改造后的右单支树对应的完全二叉树



二叉树的顺序存储

6.2 二叉树

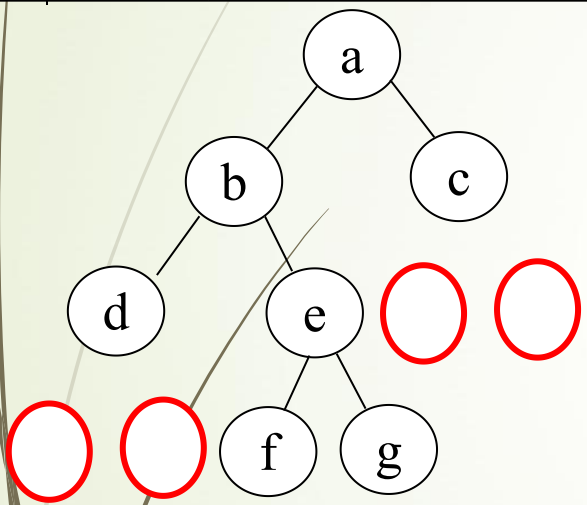
```
#define MAX_TREE_SIZE 100
```

//对应完全二叉树的最大结点编号

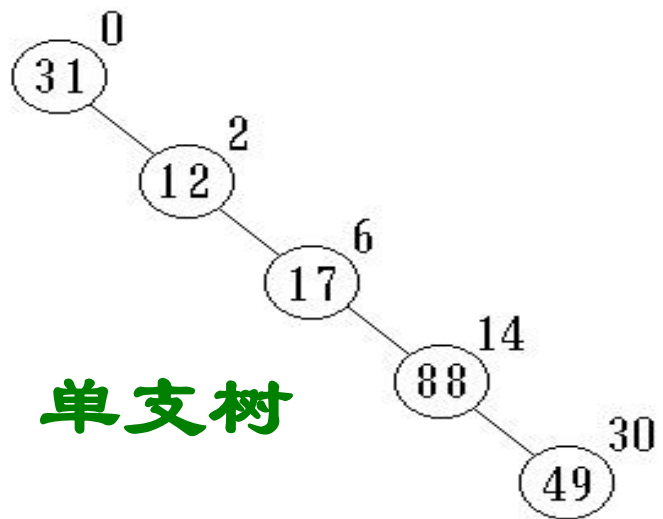
```
typedef TElemType SqBiTree[MAX_TREE_SIZE]
```

SqBiTree bt;

0	1	2	3	4	5	6	7	8	9	10
a	b	c	d	e	0	0	0	0	f	g



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
31	0	12	0	0	0	17	0	0	0	0	0	0	0	88	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	49

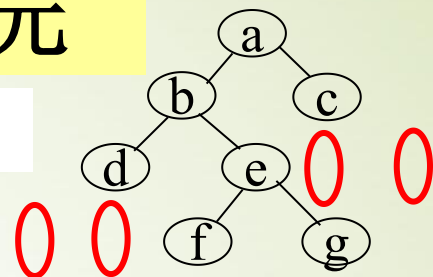


特点：
结点间关系蕴含在其存储位置中
浪费空间，适于存满二叉树和完全二叉树
当为右侧单枝树时： $2^k - 1 - k$ 个空间浪费了，空间浪费最大

二叉树的顺序存储---不使用数组0单元

6.2 二叉树

0	1	2	3	4	5	6	7	8	9	10	11
	a	b	c	d	e	0	0	0	0	f	g



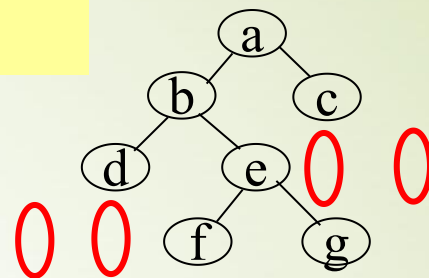
空出0单元不用，此时树结点的存储位置和性质5一致。
当 n 是二叉树对应的完全二叉树的结点数时

- (1) 若 $i > 1$ ，则 i 的双亲是 $\lfloor i/2 \rfloor$ ；若 $i = 1$ ，则 i 是根，无双亲。
- (2) 若 $2i \leq n$ ，则 i 的左孩子是 $2i$ ；否则， i 无左孩子。
- (3) 若 $2i+1 \leq n$ ，则 i 的右孩子是 $2i+1$ ；否则， i 无右孩子。

二叉树的顺序存储---使用数组0单元

6.2 二叉树

0	1	2	3	4	5	6	7	8	9	10
a	b	c	d	e	0	0	0	0	f	g



在c语言数组里，是从0开始存储，参考性质5，可计算双亲和孩子的位置。当n是二叉树对应的完全二叉树的结点数时

(1) 若 $i > 0$ ，则 i 的双亲是 $\lfloor (i+1)/2 \rfloor - 1$ ；

若 $i = 0$ ，则 i 是根，无双亲。

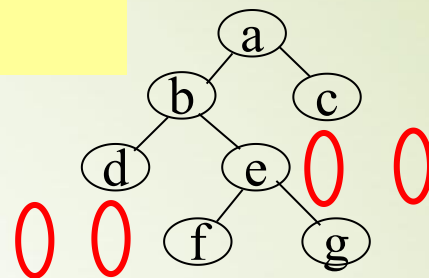
(2) 若 $2i+1 \leq n$ ，则 i 的左孩子是 $2i+1$ ；否则， i 无左孩子。

(3) 若 $2i+2 \leq n$ ，则 i 的右孩子是 $2i+2$ ；否则， i 无右孩子。

二叉树的顺序存储---使用数组0单元

6.2 二叉树

0	1	2	3	4	5	6	7	8	9	10
a	b	c	d	e	0	0	0	0	f	g



假设二叉树的各个结点值均不同，在顺序存储方式下实现：

Q1:求值为 X 的结点的双亲结点 $\text{Parent}(\text{BT}, x)$

Q2:求值为 X 的结点的左子树 $\text{Lchild}(\text{BT}, x)$

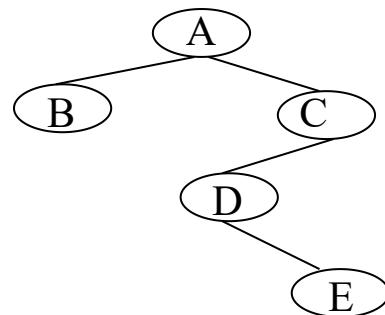
Q3:在值为 X 的结点下将 y 插入左子树
 $\text{InsertLChild}(\&\text{BT}, y, x)$

Q4:将值为 X 的结点的左子树删除
 $\text{DeleteLChild}(\&\text{BT}, x)$

二叉树的顺序存储的多种方法

1[按完全二叉树编号存放]

A	B	C		D		E
1	2	3		6		13



2[存储结点数据和左、右孩子在向量中的序号 0表示没有对应孩子]

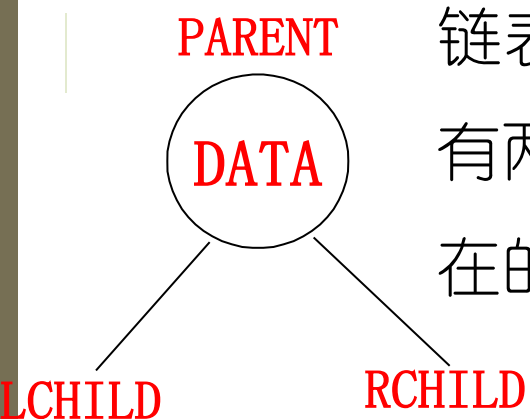
	1	2	3	4	5
data	A	B	C	D	E
lc	2	0	4	0	0
rc	3	0	0	5	0

3[存储结点数据和其双亲的序号，同时用正负号表示是双亲的那个孩子]

	1	2	3	4	5
data	A	B	C	D	E
parent	0	1	-1	3	-4

二叉树的链式存储

6.2 二叉树



(1) 二叉链表存储

链表中每个结点有3个域组成，除了数据域外，还有两个指针域，分别给出该结点左孩子和右孩子所在的链结点的存储地址。



(2) 三叉链表存储

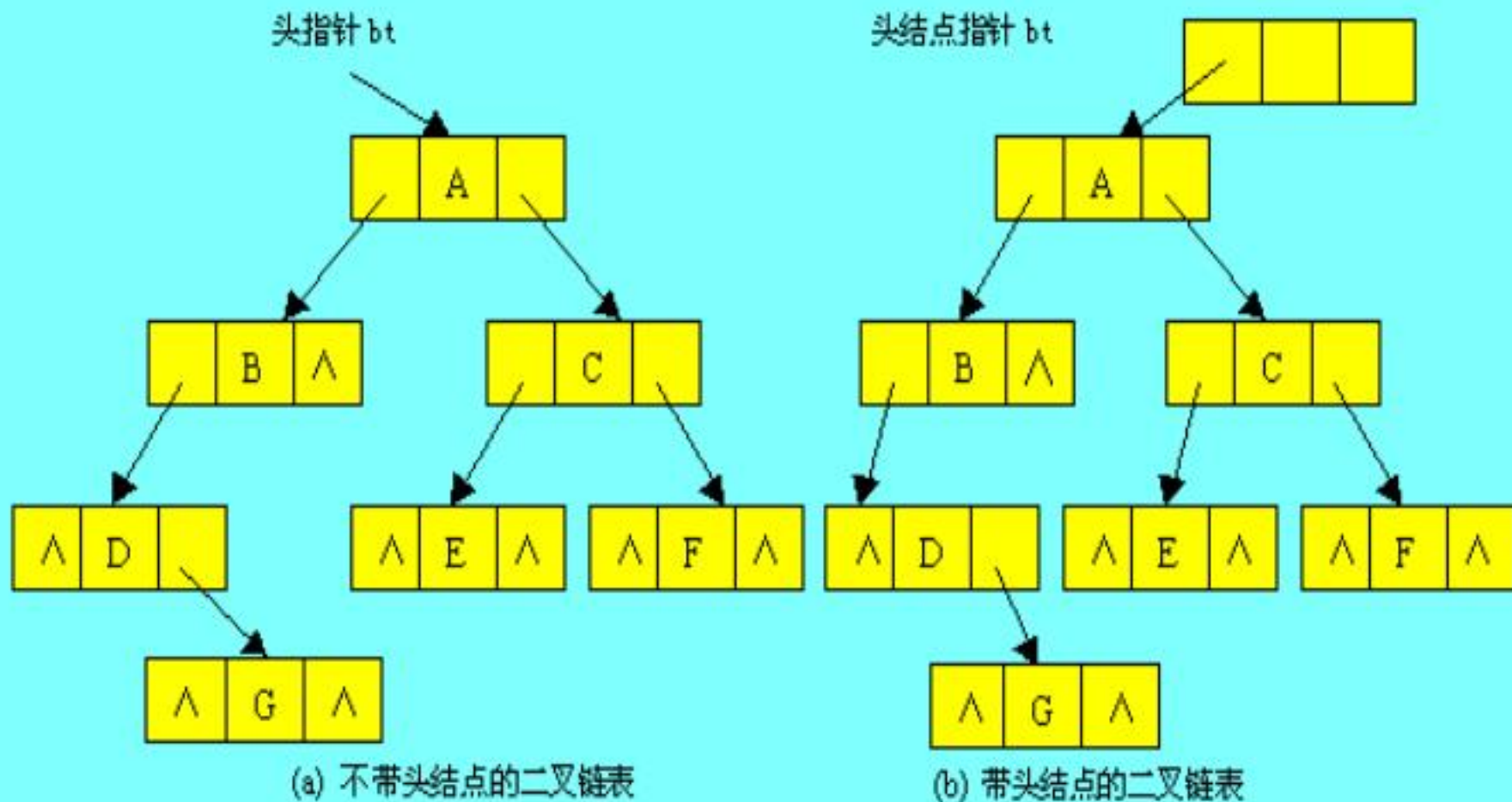
二叉链表存储可以直接找到结点的孩子结点，但不能直接找到其双亲结点，加入parent指针，用三叉链表存储可以解决这一问题，给运算带来方便



下图(a)给出一棵二叉树的二叉链表存储表示。

6.2 二叉树

二叉链表也可以带头结点的方式存放，如图(b)所示，根结点的地址存放在头结点的左孩子指针域。

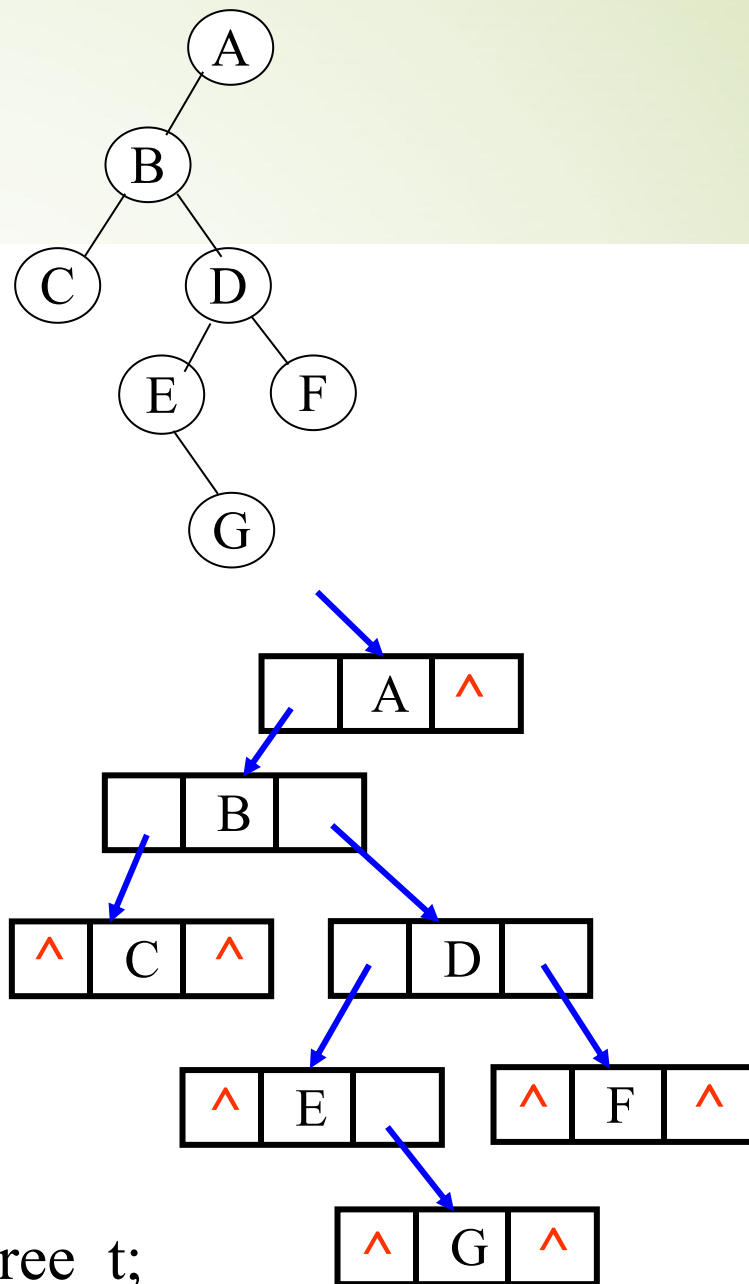


6.2 二叉树

二叉树的二叉链表存储结构可描述为：

```
typedef struct BiTNode
{
    TElemType data;
    struct BiTNode *lchild, *rchild;
    //左右孩子指针
} BiTNode, *BiTree;
```

定义一个指向二叉树的指针变量：BiTree t;



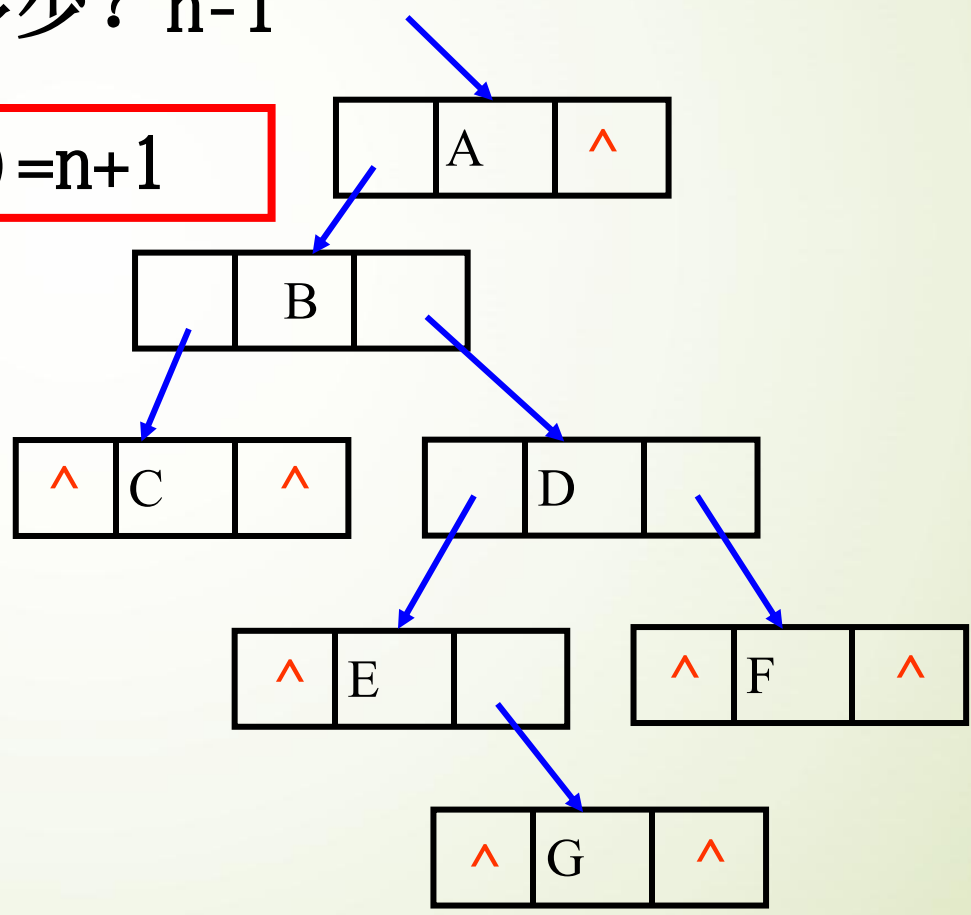
练习

6.2 二叉树

在n个结点的无头结点的二叉链表中，有 **n+1** 个空指针域

分析：必有2n个指针域。
n个结点指针的数目是多少？ n-1

空指针数目 = $2n - (n - 1) = n + 1$



6.2 二叉树

二叉树的三叉链表存储结构可描述为：

```
typedef struct bitnode
```

```
{
```

```
    datatype data;
```

```
    struct bitnode *lchild,*rchild;
```

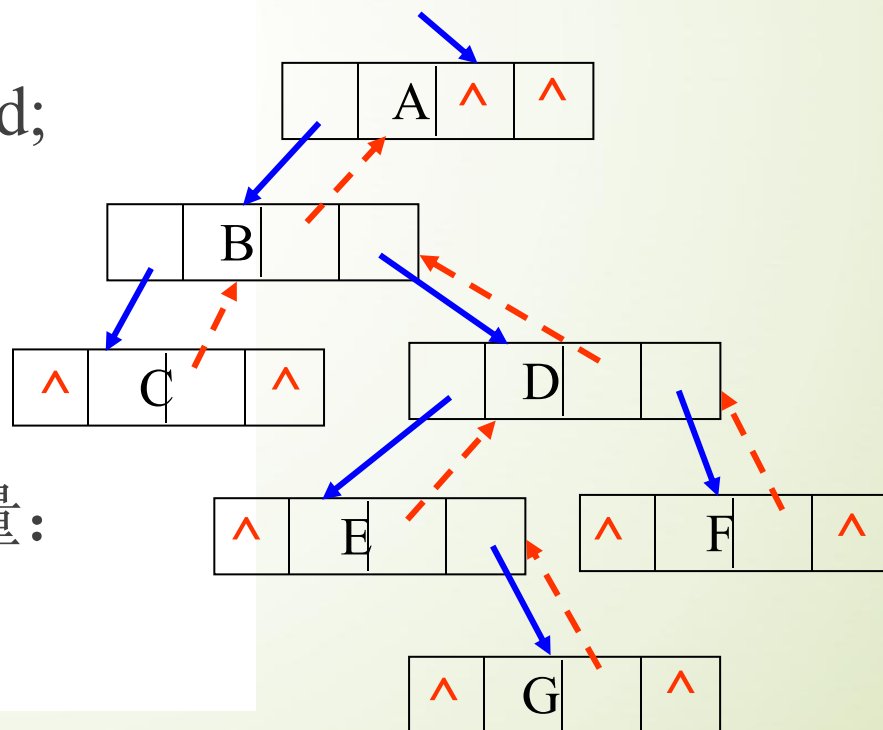
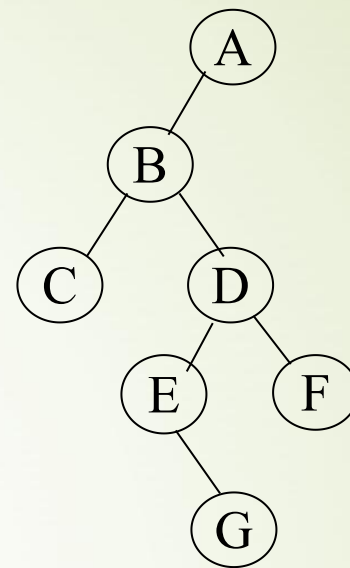
```
    //左右孩子指针
```

```
    struct bitnode *parent;
```

```
} BiTNode, *BiTree;
```

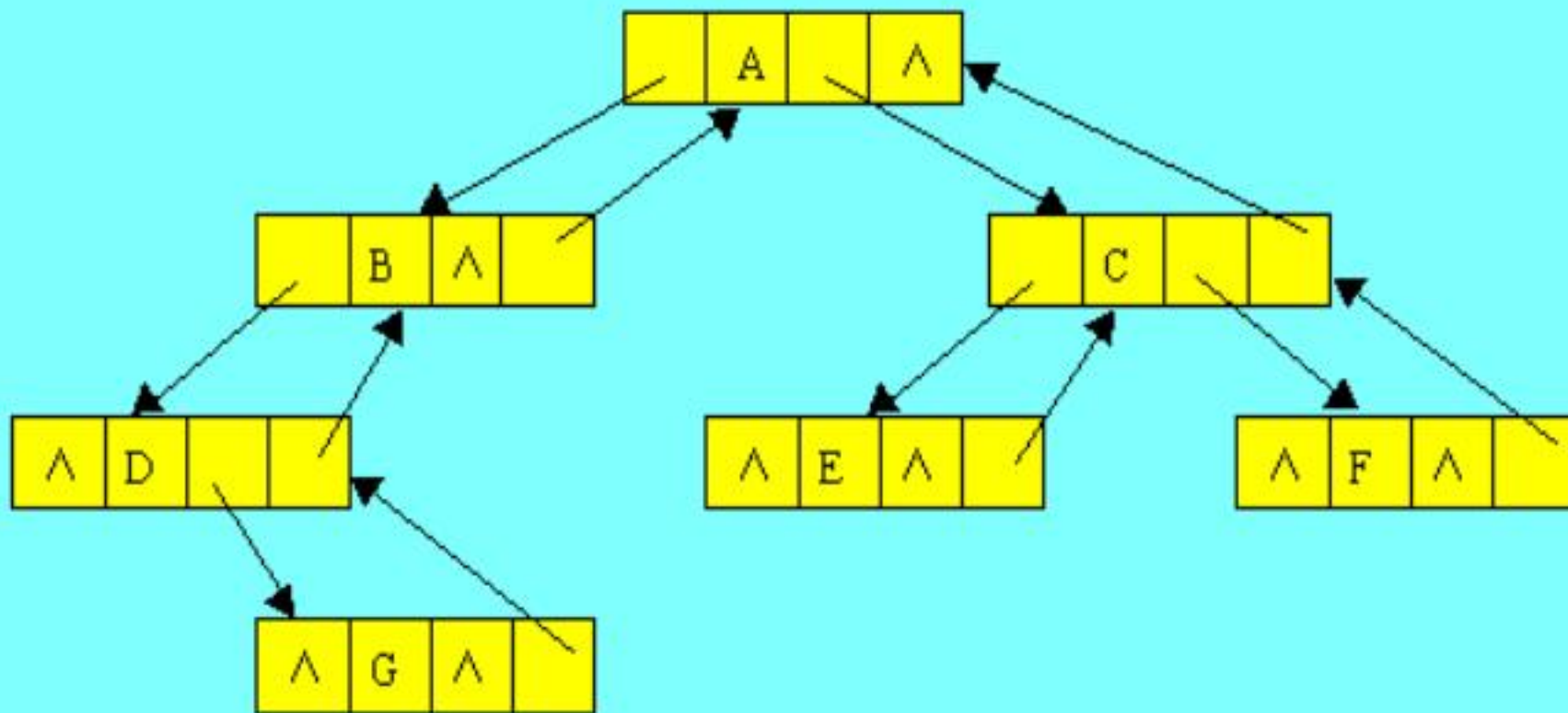
定义一个指向二叉树的指针变量：

```
BiTree t;
```



6.2 二叉树

下图给出一棵二叉树的三叉链表存储示意图。



一棵二叉树的三叉链表表示示意图

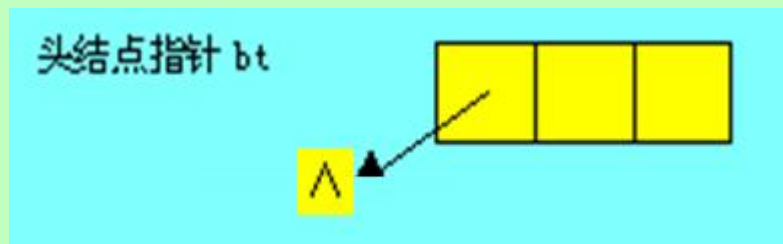
二叉树基本运算的实现（二叉链表的存储结构）

(1) Initiate(): 建立一棵空的二叉树bt, 并返回bt。

【算法 6-1(a)】建立一棵空的带头结点的二叉树

BiTree Initiate () //初始建立一棵带头结点的二叉树

```
{ BiTNode *bt;  
  bt=(BiTNode *)malloc(sizeof(BiTNode));  
  if(!bt) return NULL;  
  bt->lchild=NULL;  
  bt->rchild=NULL;  
  return bt;  
}
```



若要建立不带头结点的二叉树, 可描述如下:

【算法 6-1(b)】建立一棵空的不带头结点的二叉树

BiTree Initiate () //初始建立一棵不带头结点的二叉树

```
{ BiTNode *bt;  
  bt=NULL;  
  return bt;  
}
```



6.2 二叉树

(2) Create(x, lbt, rbt): 生成一棵以x为根结点的数据域值, 以lbt和rbt为左右子树的二叉树。

【算法 6-2】生成一棵以x为根结点的数据域值以lbt和rbt为左右子树的二叉树

```
BiTree Create(elemtype x, BiTree lbt, BiTree rbt)
{
    BiTree p;
    if (( p=(BiTNode*)malloc(sizeof(BiTNode)) )==NULL)
        return NULL;

    p->data=x;
    p->lchild=lbt;
    p->rchild=rbt;

    return p;
}
```

6.2 二叉树

(3) InsertL(bt, x, parent): 在二叉树bt中的parent所指结点及其左子树之间插入数据元素为x的结点, 且原左子树作为X结点的左子树)

【算法6-3】 在二叉树bt中的parent所指结点及其左子树之间插入数据元素为x的结点

```
BiTree InsertL(BiTree bt, elemtype x, BiTree parent)
```

```
{ BiTree p;
```

```
  if (parent==NULL)
```

```
    return NULL;
```

```
  if ((p=(BiTNode *)malloc(sizeof(BiTNode)))==NULL)
```

```
    return NULL;
```

```
  p->data=x;
```

```
  p->lchild=NULL; p->rchild=NULL;
```

```
  if (parent->lchild==NULL)
```

```
    parent->lchild=p;
```

```
  else
```

```
  { // 先让P的lchild 指向parent的左子树
```

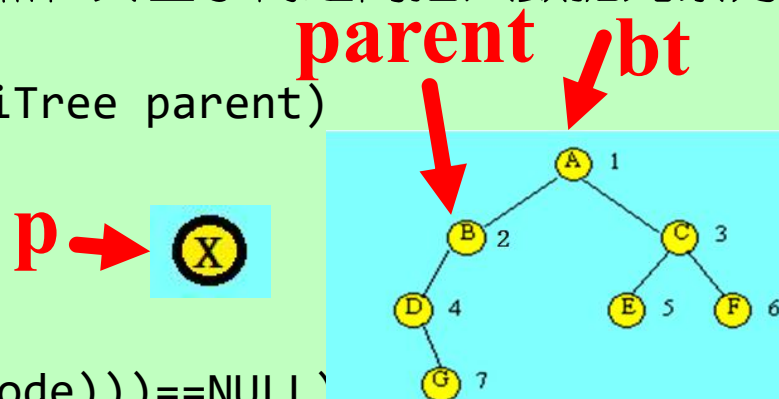
```
    p->lchild=parent->lchild;
```

```
    parent->lchild=p; // 再将 parent的lchild指向p
```

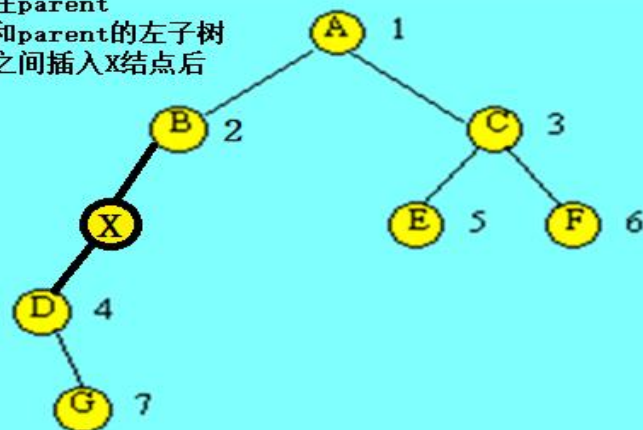
```
  }
```

```
  return bt;
```

```
}
```



在parent
和parent的左子树
之间插入x结点后



(5) DeleteL(bt, parent): 在二叉树bt中删除parent的左子树

6.2 二叉树

【算法 6-4】 在二叉树bt中删除parent的左子树

```
BiTree DeleteL(BiTree bt, BiTree parent)
```

```
{ BiTree p;
```

```
  if (parent==NULL || parent->lchild==NULL)
```

```
  { //删除出错;
```

```
    return NULL;
```

```
  }
```

```
  p = parent->lchild;
```

```
  parent->lchild=NULL;
```

```
  // 不能直接free(p);
```

```
  //当*p为非叶子结点时，这样删除仅释放了所删子树根结点的空间，
```

```
  //子树中除了根节点之外的其他结点，未释放。
```

```
  //此时需要采用遍历的方法找出子树分支中的每个结点，挨个释放
```

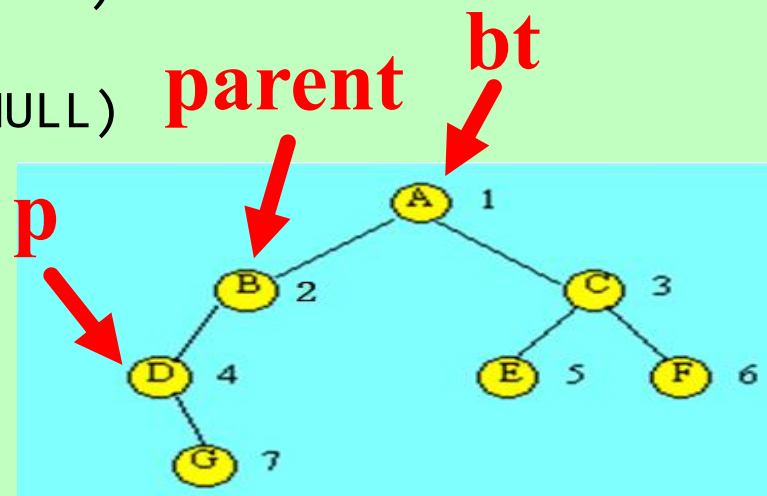
```
  ClearTree(p);
```

```
  //假设ClearTree函数是用来实现上述用途的
```

```
  //p为根的树中所有结点都被释放掉。
```

```
  return bt;
```

```
}
```



遍历二叉树



6.3 遍历

遍历定义——指按某条搜索路线遍访每个结点且不重复（又称周游）。

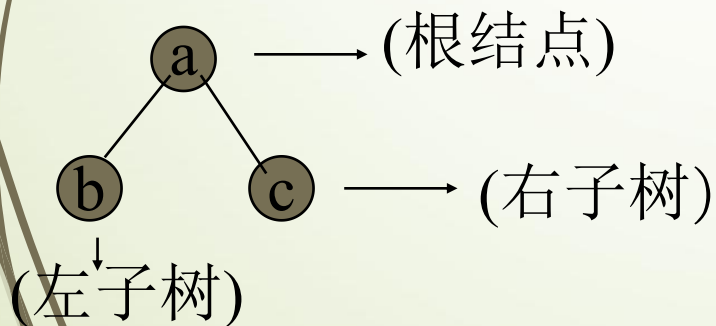
遍历用途——它是树结构插入、删除、修改、查找和排序运算的前提，是二叉树一切运算的基础和核心。

二叉树的遍历:

6.3 遍历

在二叉树的一些应用中，常常要求在树中查找具有某种特征的结点，或者对树中全部结点逐一进行某种处理。这就引入了遍历二叉树的问题，即如何按某条搜索路径巡访树中的每一个结点，使得每一个结点均被访问一次，而且仅被访问一次。

遍历对线性结构是容易解决的，而二叉树是非线性的，因而需要寻找一种规律，以便使二叉树上的结点能排列在一个线性队列上，从而便于遍历。

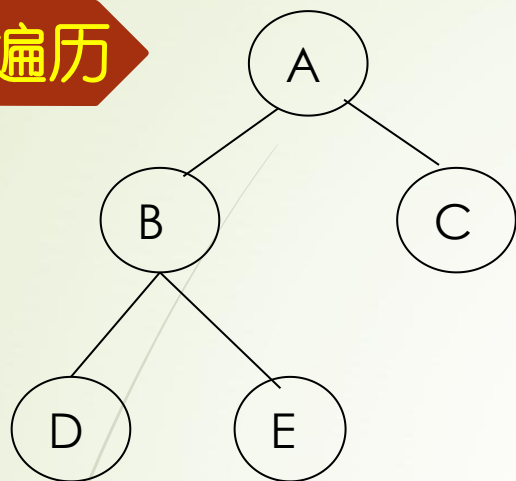


由二叉树的递归定义，二叉树的三个基本组成单元是：

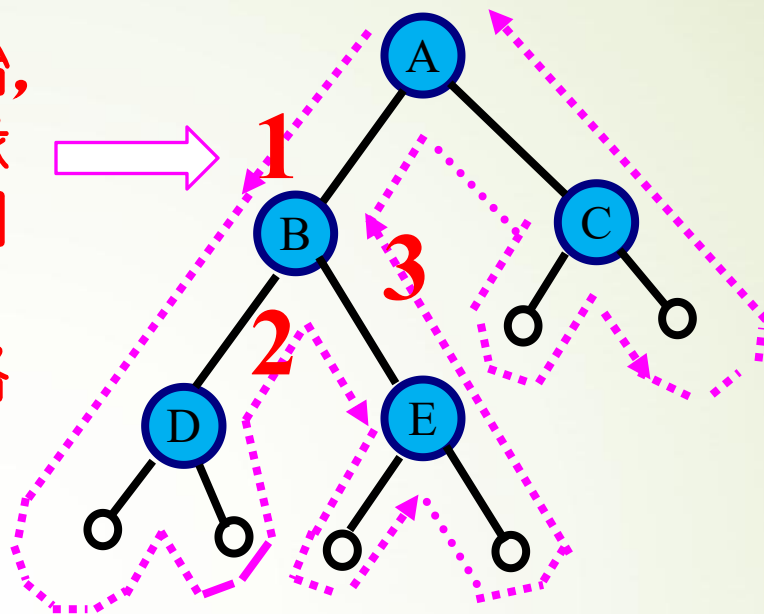
根结点、左子树和右子树。

二叉树遍历规则

6.3 遍历



从根开始,
沿着边缘
走一圈回
到根
沿着包络
线走。



在包络线上，每个结点经过**3次**。

B结点的3次经过在包络线上的位置

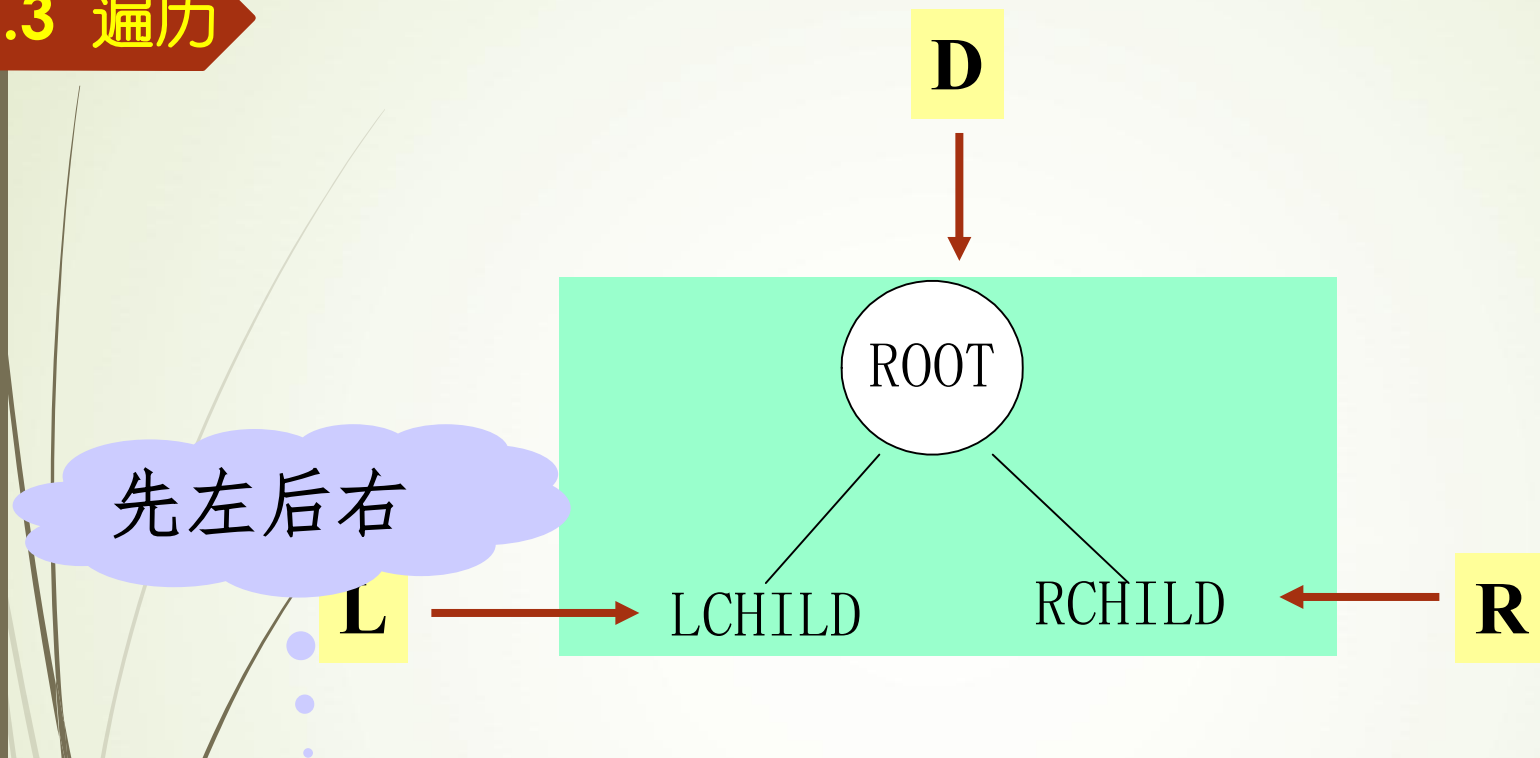
第**1次**经过时访问 = **先序**遍历: ABDEC

第**2次**经过时访问 = **中序**遍历: DBEAC

第**3次**经过时访问 = **后序**遍历: DEBCA

二叉树遍历规则

6.3 遍历



DLR

LDR

LRD

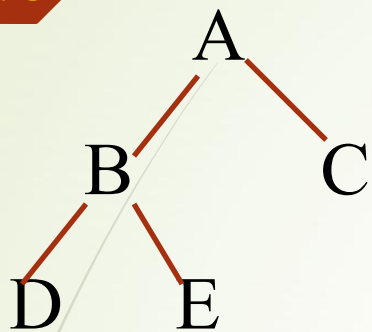
DRL

RDL

RLD

三个逆序方式很少使用

6.3 遍历



先序遍历: A B D E C

中序遍历: D B E A C

后序遍历: D E B C A

口诀: 以根的位置为准算先后

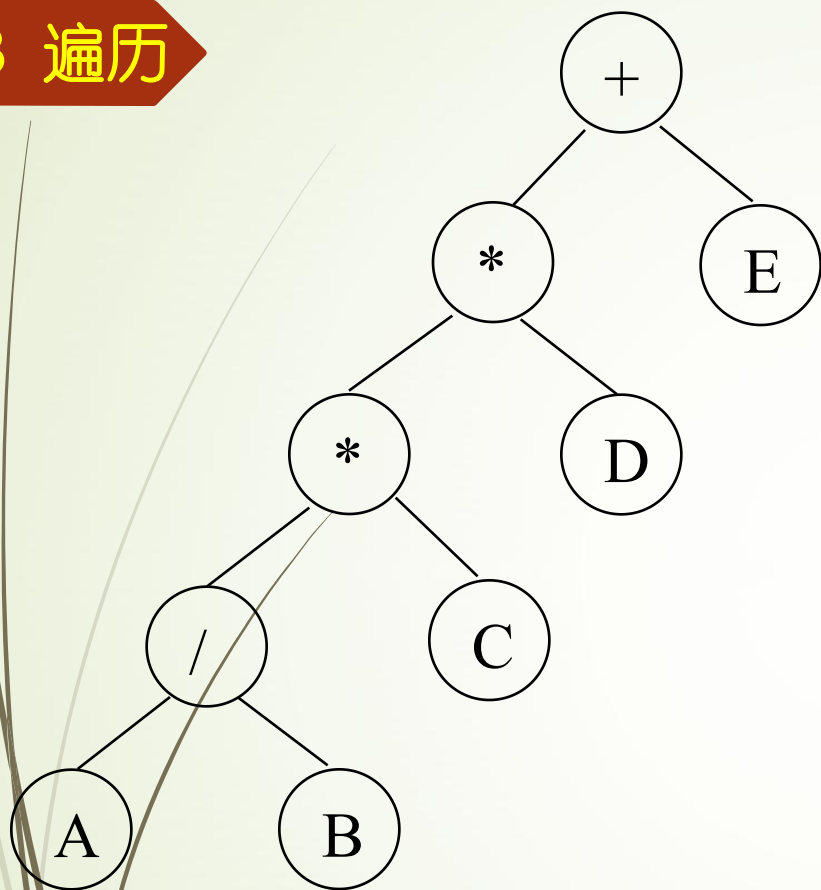
DLR—先序遍历, 即先根再左再右, 先根序

LDR—中序遍历, 即先左再根再右, 中根序

LRD—后序遍历, 即先左再右再根, 后根序

用二叉树表示算术表达式

6.3 遍历



先序遍历

$+ * * / A B C D E$

前缀表示

中序遍历

$A / B * C * D + E$

中缀表示

后序遍历

$A B / C * D * E +$

后缀表示

层序遍历

$+ * E * D / C A B$

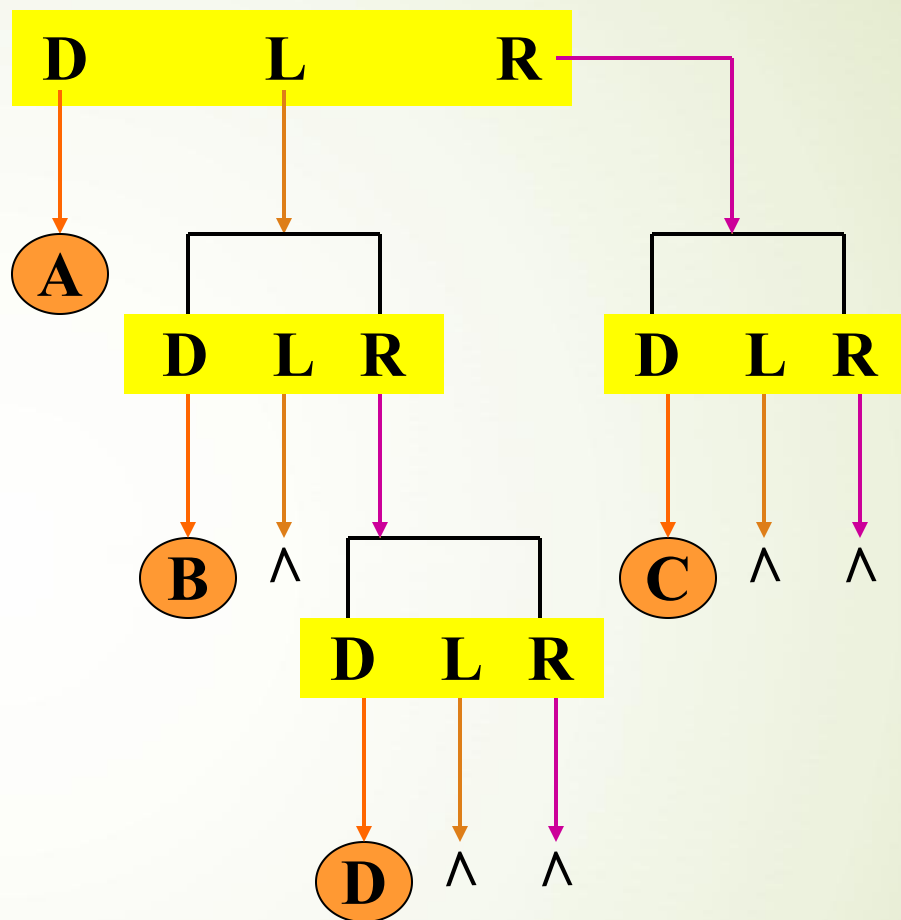
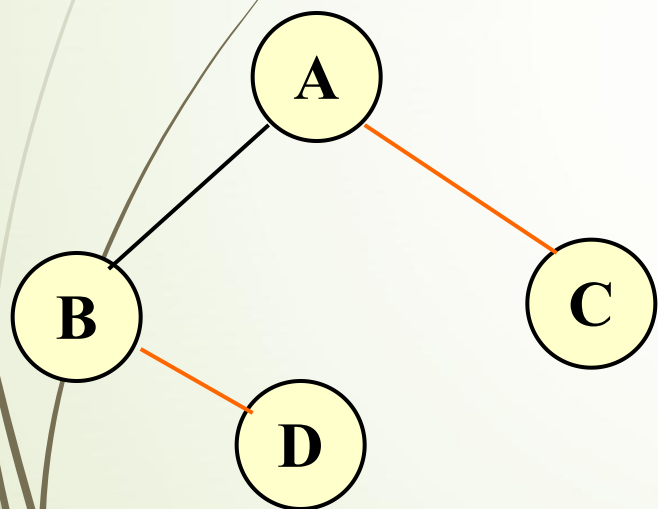
遍历的算法思路—先序遍历递归算法

若二叉树为空，则空操作
否则

访问根结点 (D)

前序遍历左子树 (L)

前序遍历右子树 (R)



先序遍历序列: A B D C

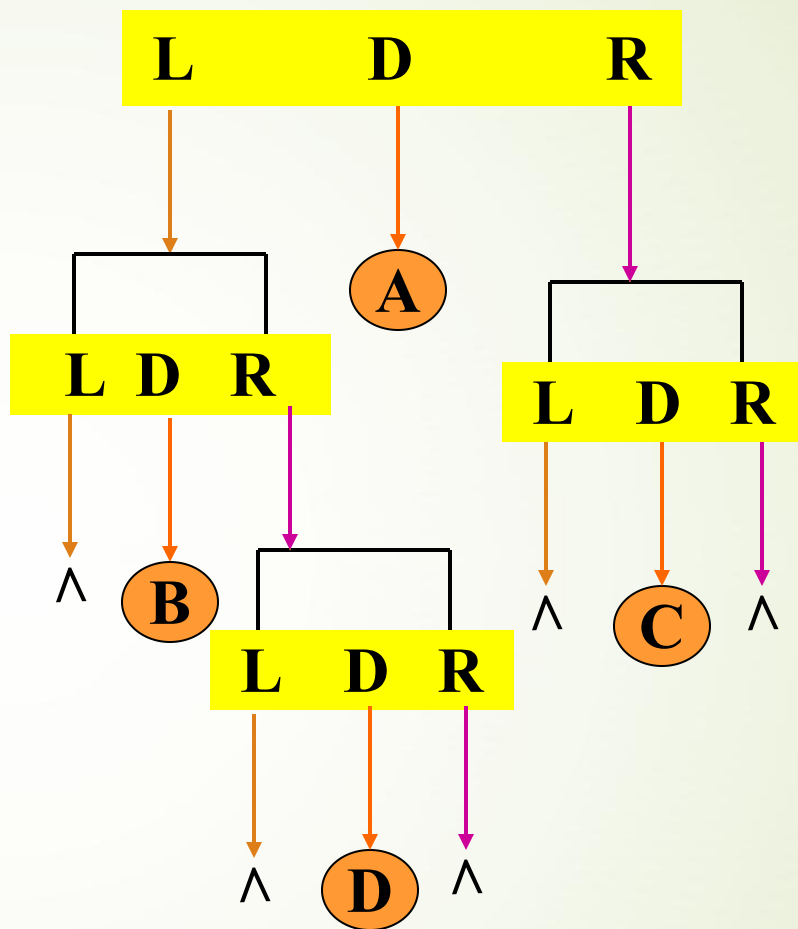
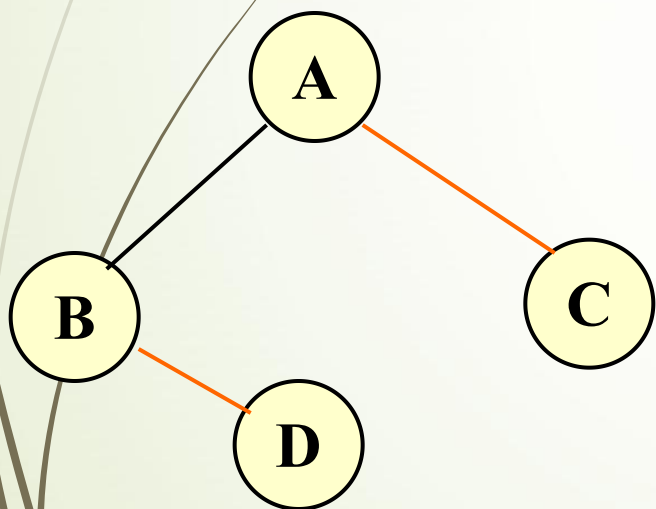
遍历的算法思路—中序遍历递归算法

若二叉树为空，则空操作
否则：

中序遍历左子树 (L)

访问根结点 (D)

中序遍历右子树 (R)



中序遍历序列: B D A C

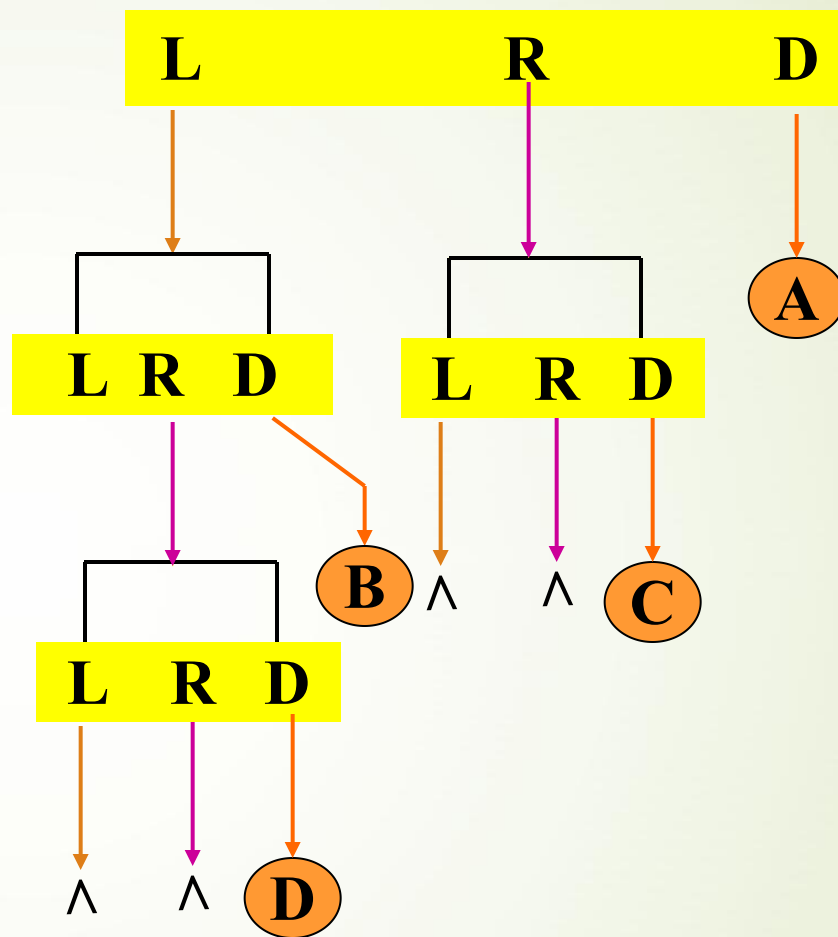
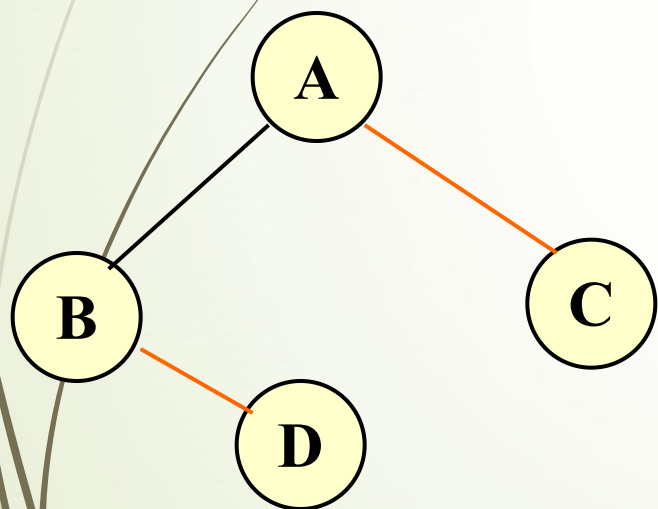
遍历的算法思路—后序遍历递归算法

若二叉树为空，则空操作
否则

后序遍历左子树 (L)

后序遍历右子树 (R)

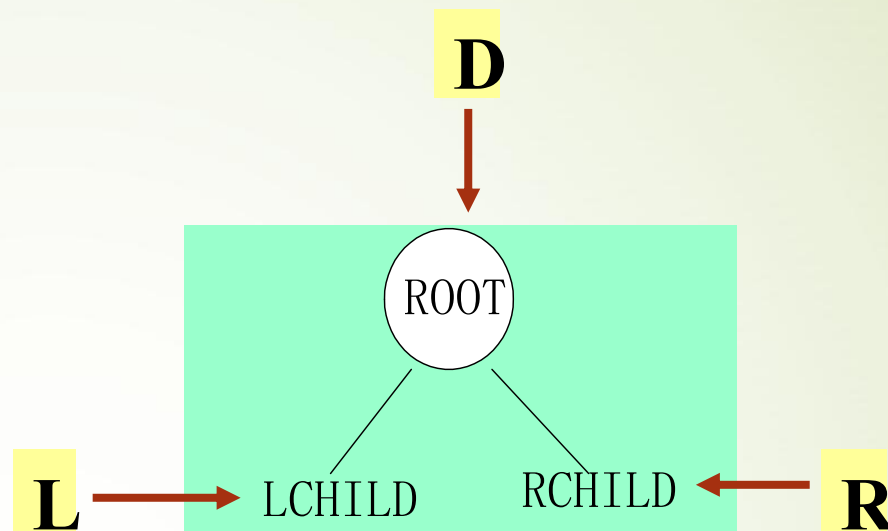
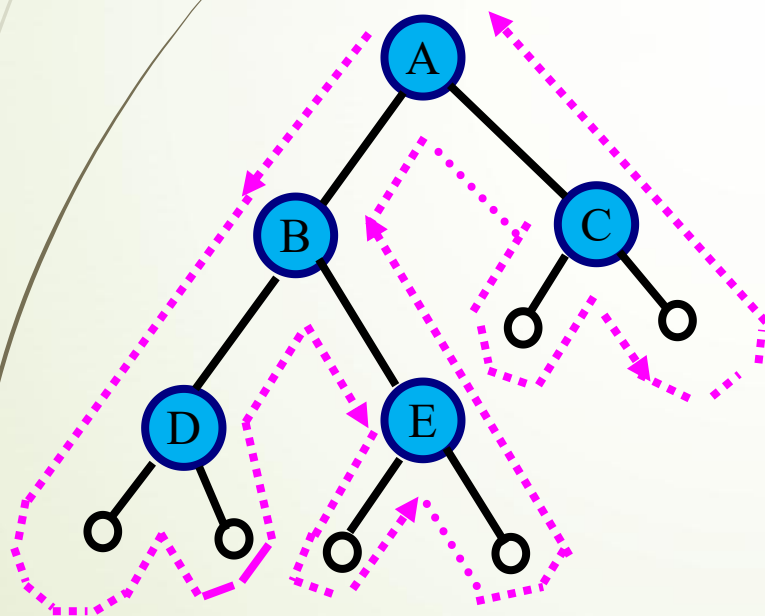
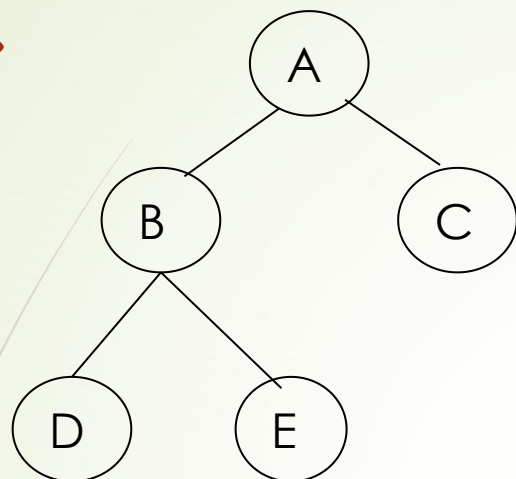
访问根结点 (D)



后序遍历序列: D B C A

二叉树遍历规则

6.3 遍历



先序遍历 **DLR**: ABDEC

中序遍历 **LDR**: DBEAC

后序遍历 **LRD**: DEBCA

遍历的算法实现——用递归形式格外简单！

6.3 遍历

回忆：**long** Factorial (**long** n)

{

if (n == 0) **return** 1; //基本项

else return n * Factorial (n-1); //归纳项

}

则三种遍历算法可写出：

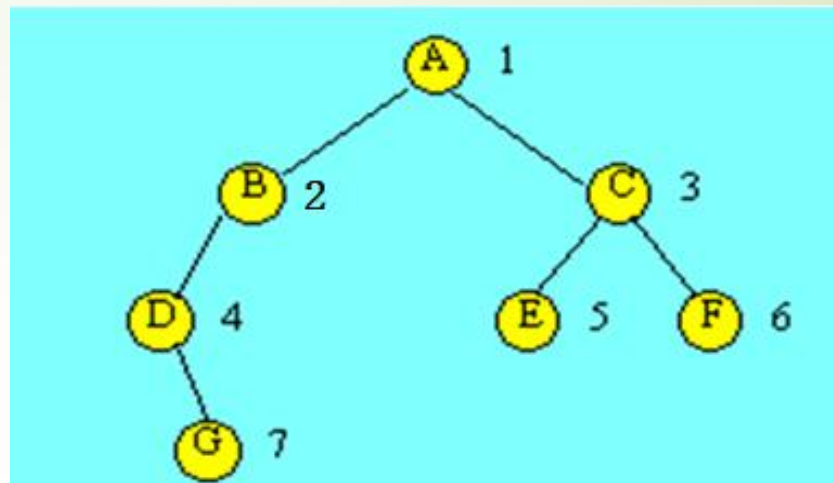
遍历的算法实现—先序遍历递归算法

若二叉树为空，则空操作
否则

访问根结点 (D)

前序遍历左子树 (L)

前序遍历右子树 (R)



先序遍历所得到的结点序列为：

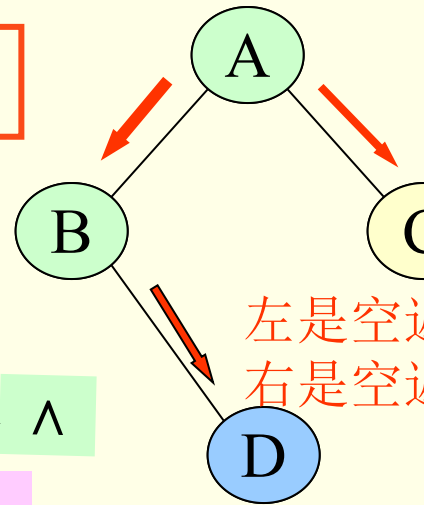
A B D G C E F

```
Status PreOrderTraverse(BiTree T)
{
    if(T==NULL) return OK; //空二叉树
    else
    {
        cout<<T->data;
        PreOrderTraverse(T->lchild); //递归遍历左子树
        PreOrderTraverse(T->rchild); //递归遍历右子树
        return OK;
    }
}
```

cout<<T->data; //访问根结点
PreOrderTraverse(T->lchild); //递归遍历左子树
PreOrderTraverse(T->rchild); //递归遍历右子树
return OK;

```
Status PreOrderTraverse(BiTree T)
{
    if(T==NULL) return OK;
    else{
        cout<<T->data;
        PreOrderTraverse(T->lchild);
        PreOrderTraverse(T->rchild);
        return OK;
    }
}
```

先序序列: **ABDC**



左是空返回

左是空返回
右是空返回

左是空返回
右是空返回

主程序

Pre(T)

T → A

cout << A;

pre(T → L);

pre(T → R);

T → B

cout << B;

pre(T → L);

pre(T → R);

T → C

cout << C;

pre(T → L);

pre(T → R);

T → Λ

返回

T → D

cout << D;

pre(T → L);

pre(T → R);

T → Λ

返回

T → Λ

返回

T → A

返回

T → A

返回

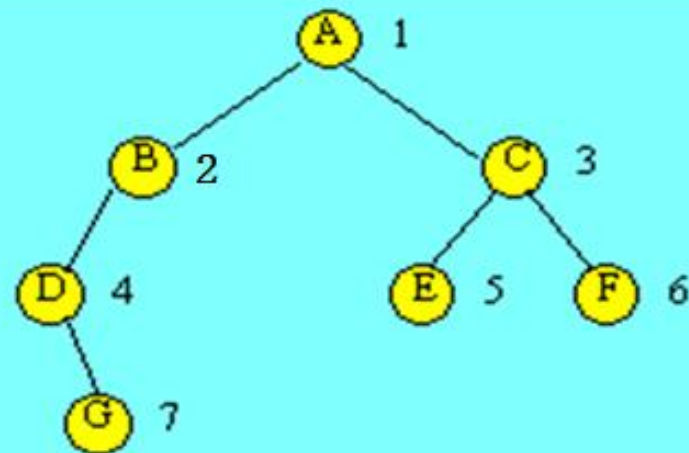
遍历的算法实现—中序遍历递归算法

若二叉树为空，则空操作
否则

中序遍历左子树 (L)

访问根结点 (D)

中序遍历右子树 (R)



中序: D G B A E C F

```
Status InOrderTraverse(BiTree T)
{
    if(T==NULL) return OK; //空二叉树
    else
    {
        InOrderTraverse(T->lchild); //递归遍历左子树
        cout<<T->data; //访问根结点
        InOrderTraverse(T->rchild); //递归遍历右子树
        return OK;
    }
}
```

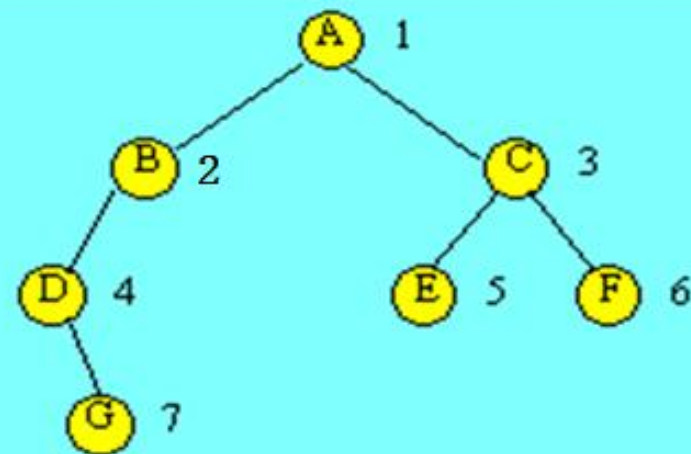
遍历的算法实现—后序遍历递归算法

若二叉树为空，则空操作
否则

后序遍历左子树 (L)

后序遍历右子树 (R)

访问根结点 (D)



```
Status PostOrderTraverse(BiTree T)
{
```

```
    if(T==NULL) return OK; //空二叉树
```

```
    else
```

```
    {
```

```
        PostOrderTraverse(T->lchild); //递归遍历左子树
```

```
        PostOrderTraverse(T->rchild); //递归遍历右子树
```

```
        cout<<T->data; //访问根结点
```

```
        return OK;
```

```
    }
```

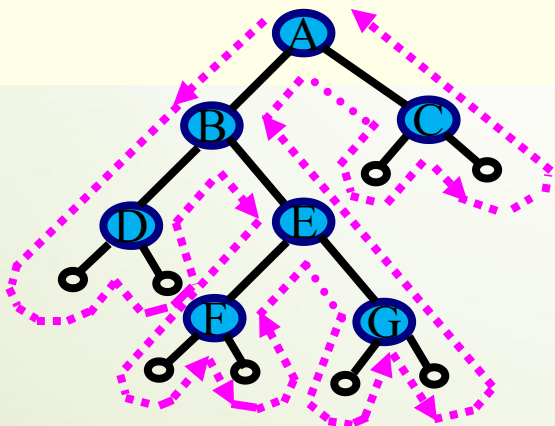
```
}
```

后序: G D B E F C A

遍历算法的分析

6.3 遍历

```
Status PreOrderTraverse(BiTree T)
{
    if(T==NULL) return OK;
    else
    {
        cout<<T->data;
        PreOrderTraverse(T->lchild);
        PreOrderTraverse(T->rchild);
        return OK;
    }
}
```



```
Status InOrderTraverse(BiTree T)
{
    if(T==NULL) return OK;
    else
    {
        InOrderTraverse(T->lchild);
        cout<<T->data;
        InOrderTraverse(T->rchild);
        return OK;
    }
}
```

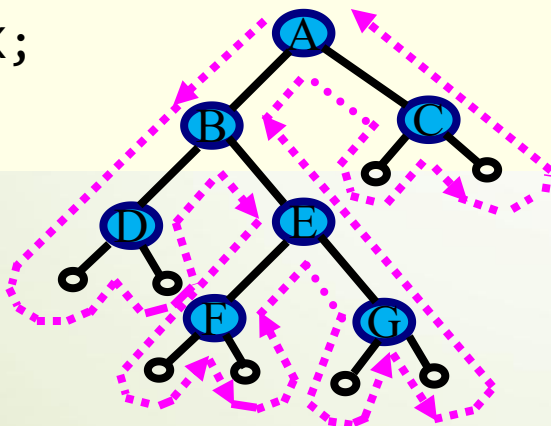
```
Status PostOrderTraverse(BiTree T)
{
    if(T==NULL) return OK;
    else
    {
        PostOrderTraverse(T->lchild);
        PostOrderTraverse(T->rchild);
        cout<<T->data;
        return OK;
    }
}
```

遍历算法的分析---改进

6.3 遍历

目的：减少对空二叉树的递归调用。

```
Status PreOrderTraverse(BiTree T)
{
    if(T)
    {
        cout<<T->data;
        if(T->lChild)
            PreOrderTraverse(T->lchild);
        if(T->rChild)
            PreOrderTraverse(T->rchild);
    }
    return OK;
}
```



```
Status InOrderTraverse(BiTree T)
{
    if(T)
    {
        if(T->lChild)
            InOrderTraverse(T->lchild);
        cout<<T->data;
        if(T->rChild)
            InOrderTraverse(T->rchild);
    }
    return OK;
}

Status PostOrderTraverse(BiTree T)
{
    if(T)
    {
        if(T->lChild)
            PostOrderTraverse(T->lchild);
        if(T->rChild)
            PostOrderTraverse(T->rchild);
        cout<<T->data;
    }
}
```

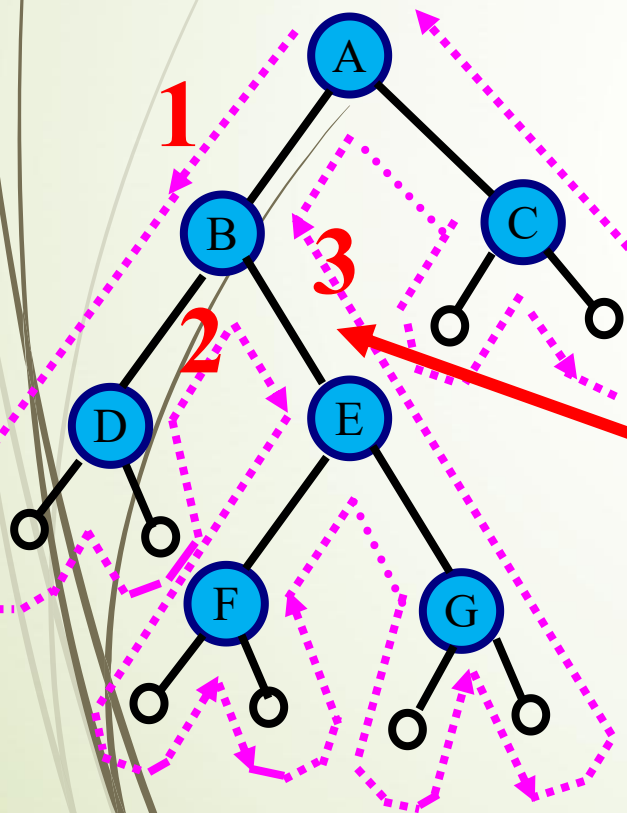
遍历算法的分析

如果去掉输出语句，从递归的角度看，三种算法的访问路径是相同的，只是访问结点的时机不同。即从根结点左外侧开始，由根结点右外侧结束的曲线，称其为**该二叉树的包络线**。

```
void Order(BiTree bt)
{if (bt==NULL) return; //递归调用的结束条件
  Order(bt->lchild); //递归遍历bt的左子树
  Order(bt->rchild); //递归遍历bt的右子树
}
```

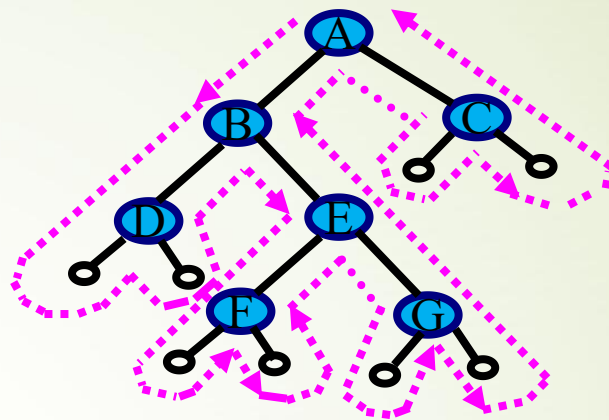
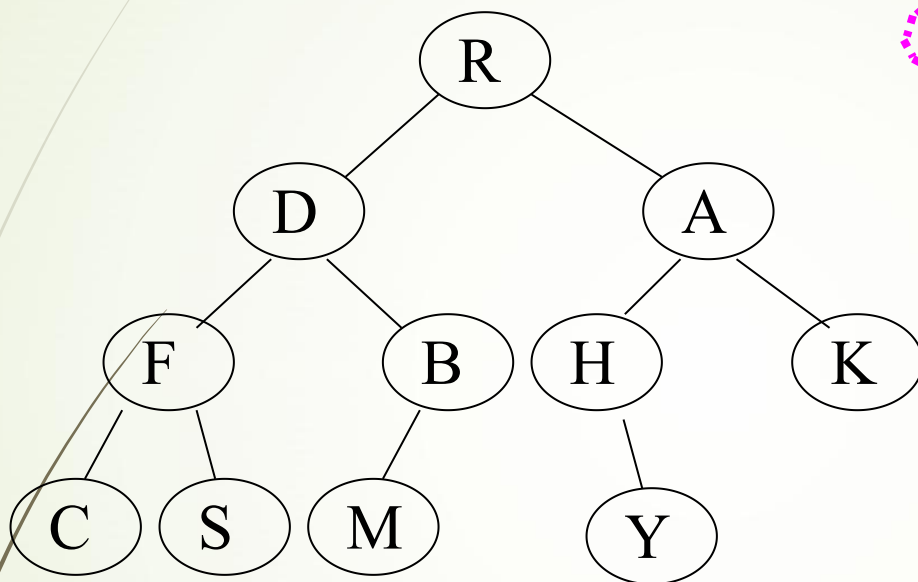
在包络线上，每个结点经过**3次**。
B结点的3次经过在包络线上的位置

第1次经过时访问 = **先序**遍历
第2次经过时访问 = **中序**遍历
第3次经过时访问 = **后序**遍历



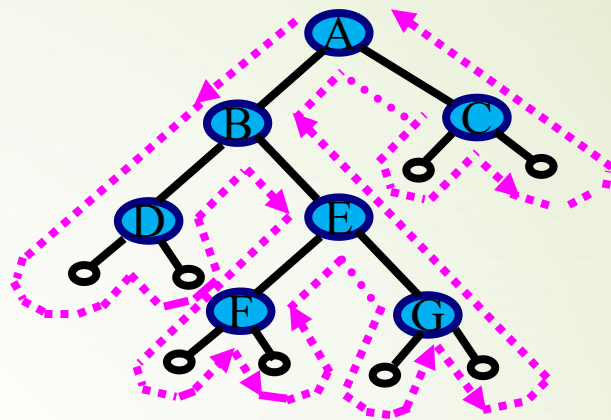
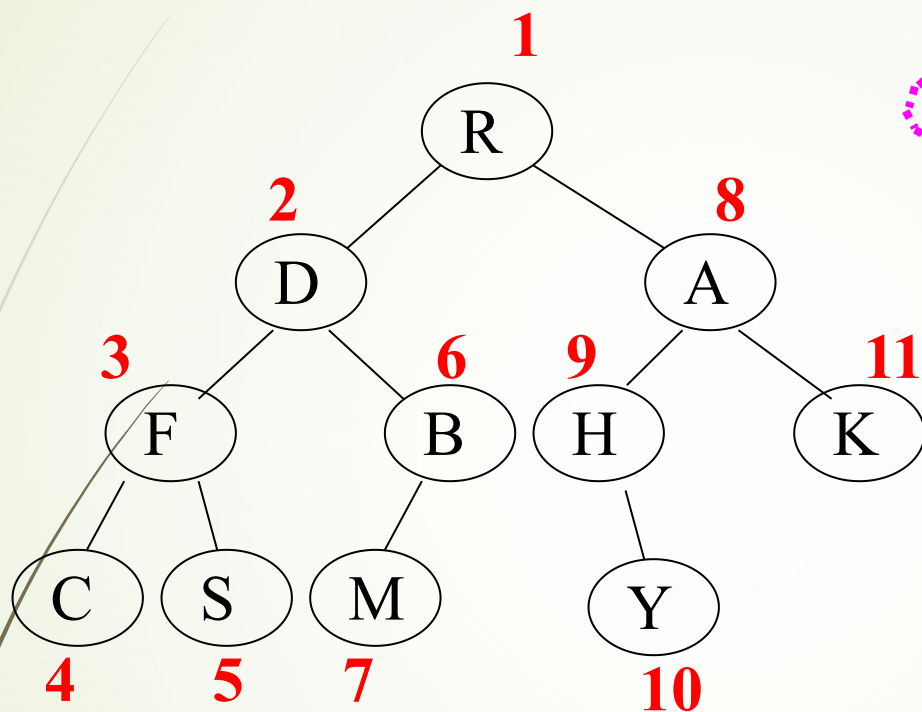
二叉树的遍历示例：

6.3 遍历



二叉树的遍历示例：前序

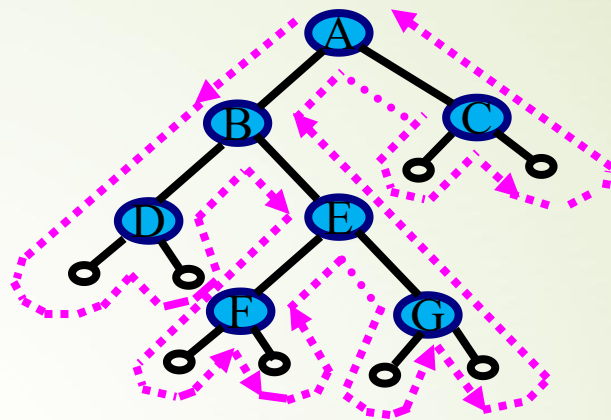
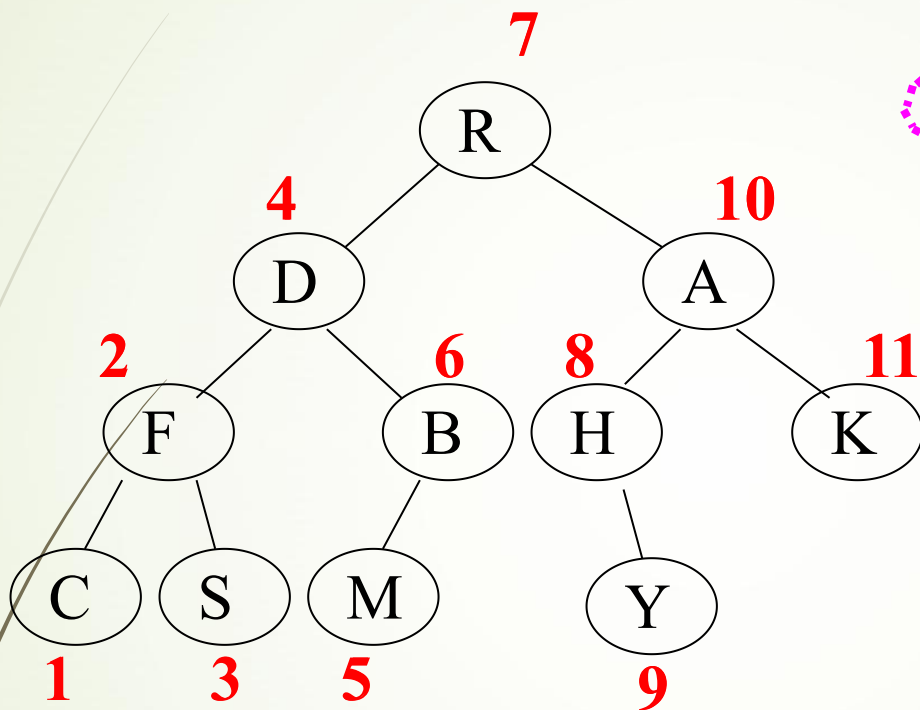
6.3 遍历



R D F C S B M A H Y K

二叉树的遍历示例：中序

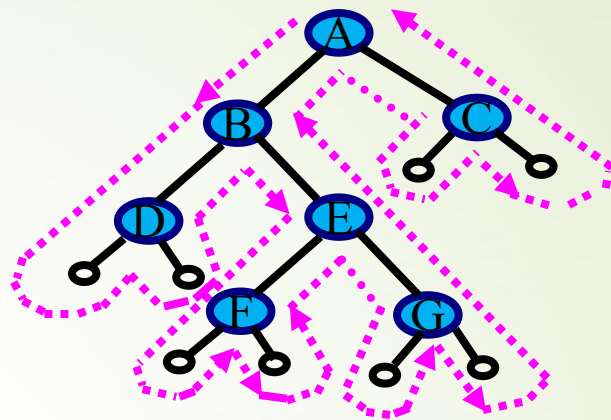
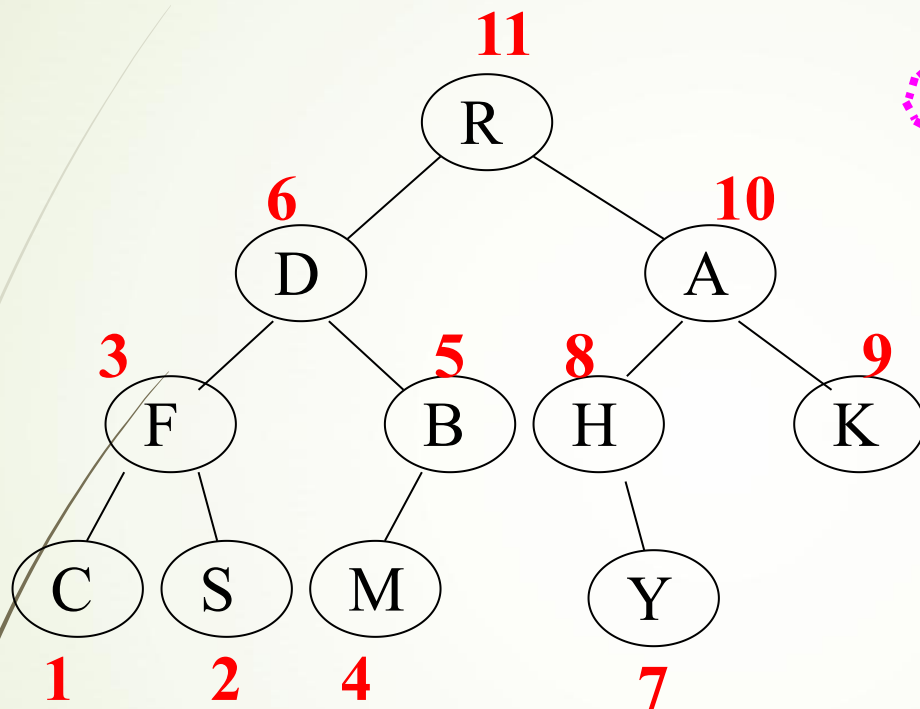
6.3 遍历



C F S D M B R H Y A K

二叉树的遍历示例：后序

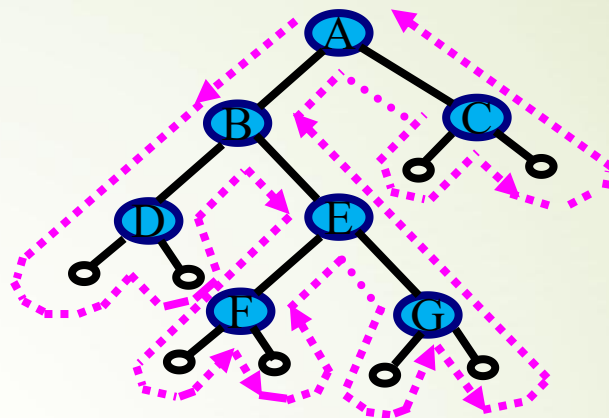
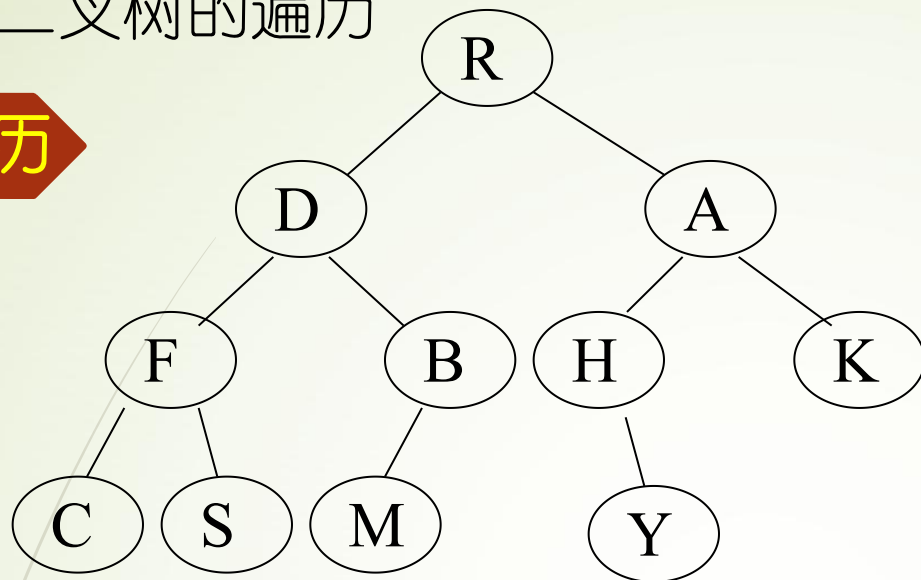
6.3 遍历



C S F M B D Y H K A R

二叉树的遍历

6.3 遍历



DLR: R D F C S B M A H Y K

LD**R**: C F S D M B R H Y A K

LR**D**: C S F M B D Y H K A R

第1次经过时访问=先(根)序遍历

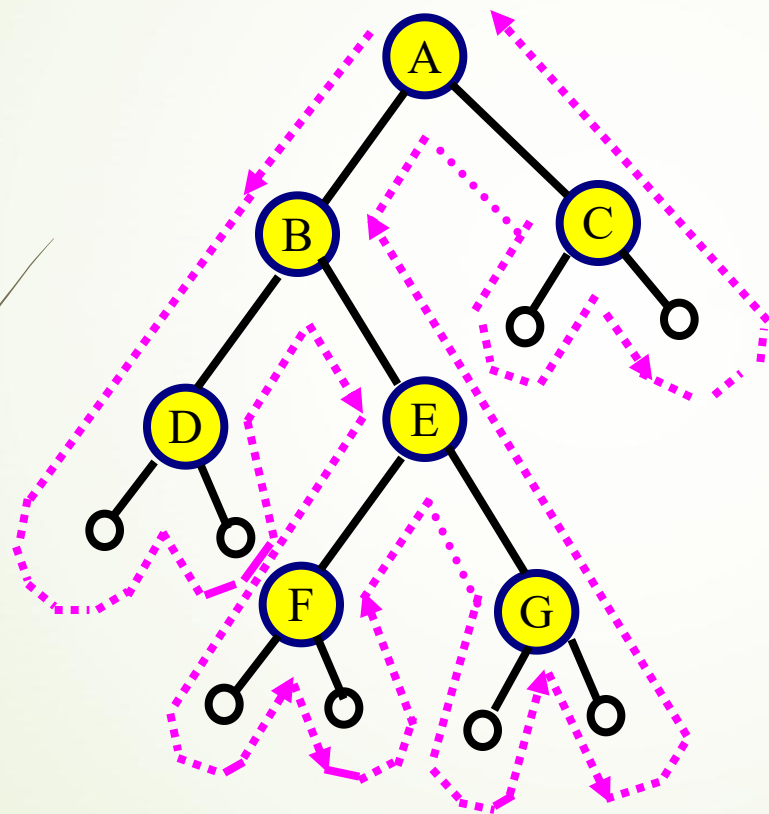
第2次经过时访问=中(根)序遍历

第3次经过时访问=后(根)序遍历

遍历的递归算法的分析

6.3 遍历

时间效率: $O(n)$ // 每个结点只访问一次
空间效率: $O(n)$ // 栈占用的最大辅助空间



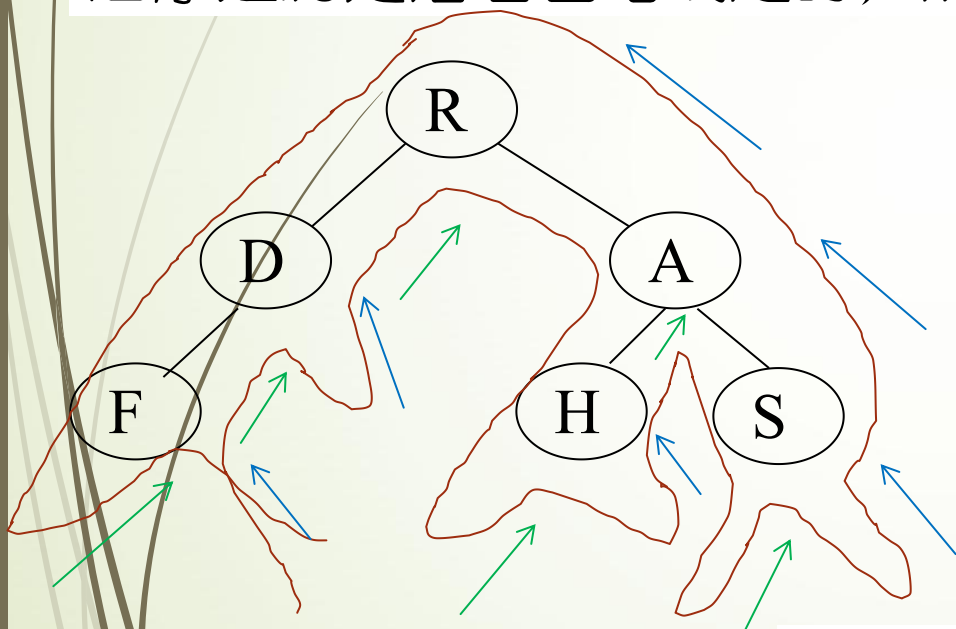
非递归方法实现二叉树的三种遍历

6.3 遍历

先序遍历是沿着包络线走时，遇到结点就访问；

中序遍历是沿着包络线走时，从左子树返回时遇到结点访问；

后序遍历是沿着包络线走时，从右子树返回时遇到结点访问。



DLR: R D F A H S

L**D**R: F D R H A S

LR**D**: F D H S A R

要解决的问题：

如何用非递归方式沿着包络线走

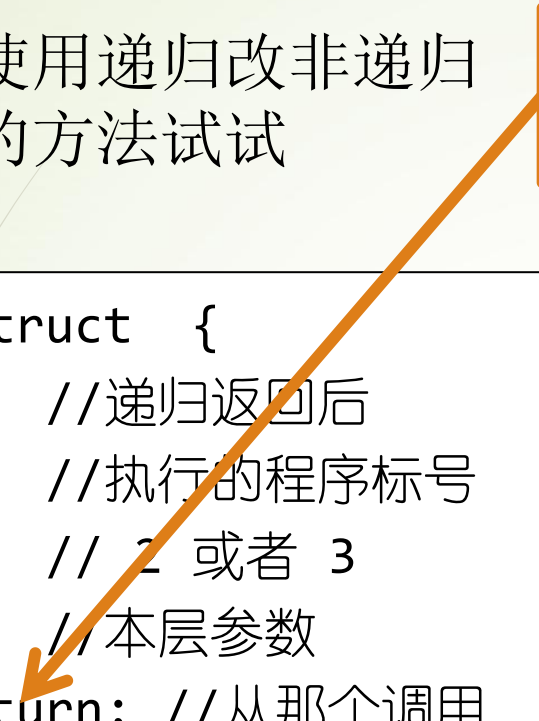
非递归遍历-改递归算法

6.3 遍历

使用递归改非递归的方法试试

标号值和进入的分支有关，可以代替 LEFT、RIGHT

```
typedef struct {  
    int rd1 //递归返回后  
            //执行的程序标号  
            // 2 或者 3  
    int n; //本层参数  
    int nReturn; //从那个调用  
                //分支返回，  
                //LEFT 或者 RIGHT  
    int fL; //保存左返回值用  
    int fR; //保存右返回值用  
}SElemType;
```



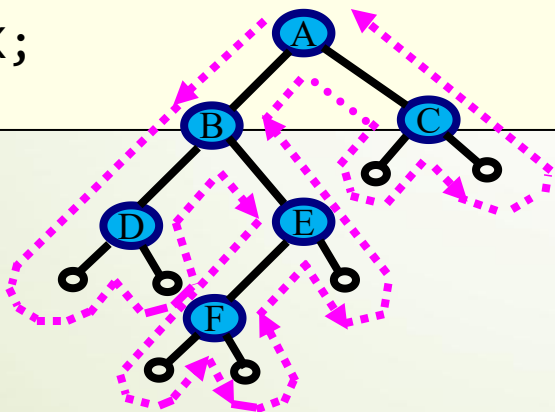
```
typedef struct {  
    BiTree pTree; //本层参数  
  
    int rd; //递归返回后  
            //执行的程序标号  
            // 2 或者 3  
    int f; //fL, fR, 先保留1个  
           //需要时再增加  
}STreeElemType;
```

非递归遍历-改递归算法

6.3 遍历

使用递归改非递归的方法试试

```
Status PreOrderTraverse(BiTree T)
{
0:   if(T)
    {
        cout<<T->data;
1:   if(T->lChild)
        PreOrderTraverse(T->lchild);
2:   if(T->rChild)
        PreOrderTraverse(T->rchild);
    }
3:   return OK;
}
```



```
Status PreOrderTraverse(BiTree T)
{
    InitStack(S);
L0:  if(T){
        cout<<T->data;
        if(T->lChild){
            Push(S, T, 2, f);
            T = T->lChild;
            goto L0;
        }
L2:  if(T->rChild){
        Push(S, T, 3, f);
        T=T->rChild;
        goto L0;
    }
L3:  if(!StackEmpty(S)){
        //修改返回值: S[top-1].f = ...
        Pop(S, T, n, f);
        Switch(n){
            case 2: goto L2;break;
            case 3: goto L3;break;
        }
    }
    DestroyStack(S);
    return OK;
}
```

非递归遍历-改递归算法

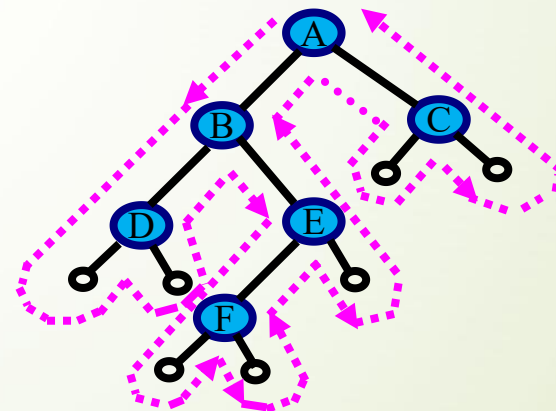
6.3 遍历 使用递归改非递归

```
Status InOrderTraverse(BiTree T)
```

```
{
    InitStack(S);
L0: if(T){
        if(T->lChild){
            Push(S, T, 2, f);
            T = T->lChild;
            goto L0;
        }
L2: cout<<T->data;
        if(T->rChild){
            Push(S, T, 3, f);
            T=T->rChild;
            goto L0;
        }
L3: if(!StackEmpty(S)){
        //修改返回值: S[top-1].f = ...
        Pop(S, T, n, f);
        Switch(n){
            case 2: goto L2;break;
            case 3: goto L3;break;
        }
    }
    DestroyStack(S);
    return OK;
}
```

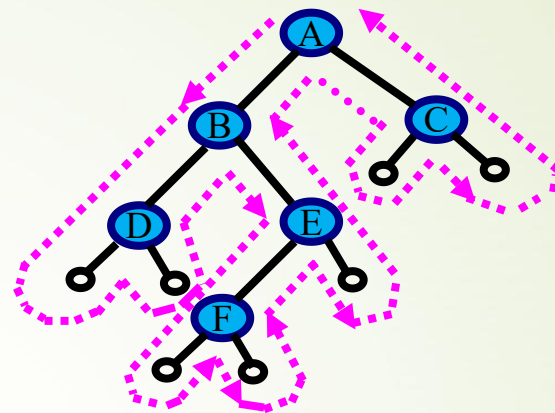
```
Status InOrderTraverse(BiTree T)
```

```
{
0: if(T)
    {
1: if(T->lChild)
        InOrderTraverse(T->lchild);
2: cout<<T->data;
        if(T->rChild)
            InOrderTraverse(T->rchild);
    }
3: return OK;
}
```



非递归遍历-改递归算法

```
Status PostOrderTraverse(BiTree T)
{
    InitStack(S);
L0: if(T){
        if(T->lChild){
            Push(S, T, 2, f);
            T = T->lChild;
            goto L0;
        }
L2: if(T->rChild){
            Push(S, T, 4, f);
            T=T->rChild;
            goto L0;
        }
L4: cout<<T->data;
    }
L3: if(!StackEmpty(S)){
        //修改返回值: S[top-1].f = ...
        Pop(S, T, n, f);
        Switch(n){
            case 2: goto L2;break;
            case 4: goto L4;break;
        }
    }
    DestroyStack(S);
    return OK;
}
```



```
Status PostOrderTraverse(BiTree T)
{
    0: (T)
    {
        1: if(T->lChild)
            PostOrderTraverse(T->lchild);
        2: if(T->rChild)
            PostOrderTraverse(T->rchild);
        4: cout<<T->data;
    }
    3: return OK;
}
```

非递归遍历-借助栈沿着包络线走

按包络线行走的核心思路：

从根结点开始沿左子树深入下去，当深入到最左端，无法再深入下去时，则返回，再逐一进入刚才深入时遇到结点的右子树，再进行如此的深入和返回，直到结点为止。

在这个沿着包络线走的过程中，
结点入栈几次？

1个结点入栈1次，用于未来找右子树用。

在这一过程中，返回结点的后深入先返回。符合“栈”结构的特点。

p是否为空

如果不为空，把p指向的结点入栈，然后 $p=p-\text{>lchild}$

如果为空，出栈一个结点， $p=\text{刚出栈的结点的}-\text{>rchild}$

非递归遍历-借助栈沿着包络线走

6.3 遍历

p是否为空

如果不为空，把p指向的结点入栈，然后 $p=p-\>lchild$

如果为空，出栈一个结点， p =刚出栈的结点的 $\rightarrow rchild$

$p=bt$; //p指向根结点

$while(!(p==NULL \& \& top==-1))$ //指针p为空且栈空时结束

{

while($p!=NULL$)

{

top++;

stack[top]=p; //将当前指针p压栈

$p=p-\>lchild$; //指针指向p的左孩子结点

}

if (top<0) return; //栈空时结束

else

{

$p=stack[top]$;

top - -; //从栈中弹出栈顶元素

$p=p-\>rchild$; //到栈顶元素右子树

}

}

(1) 先序遍历的非递归实现

6.3 遍历

【算法 6-8】 先序遍历的非递归算法

```
void NRPreOrder(BiTree bt)
{ //非递归先序遍历二叉树
    BiTNode* stack[MAXNODE], p;    //设足够大的栈空间
    int top=-1;    //栈初始化为空
    if (bt==NULL) return;
    p=bt; //p指向根结点
    while( ! (p==NULL && top==-1) ) //指针p为空且栈空时结束
    {
        while(p!=NULL)
        {
            Visit (p);    //访问当前结点，在左孩子入栈之前访问当前结点
            top++;
            stack[top]=p;    //将当前指针p压栈
            p=p->lchild;    //指针指向p的左孩子结点
        }
        if (top<0) return;    //栈空时结束
        else
        {
            p=stack[top];
            top - -;    //从栈中弹出栈顶元素
            p=p->rchild;    //到栈顶元素右子树
        }
    }
}
```

6.3 遍历

(2) 中序遍历的非递归实现

中序遍历的非递归算法的实现，只需将先序遍历的非递归算法中的 `Visit(p->data)` 移到 `p=stack[top]` 和 `p=p->rchild` 之间即可。

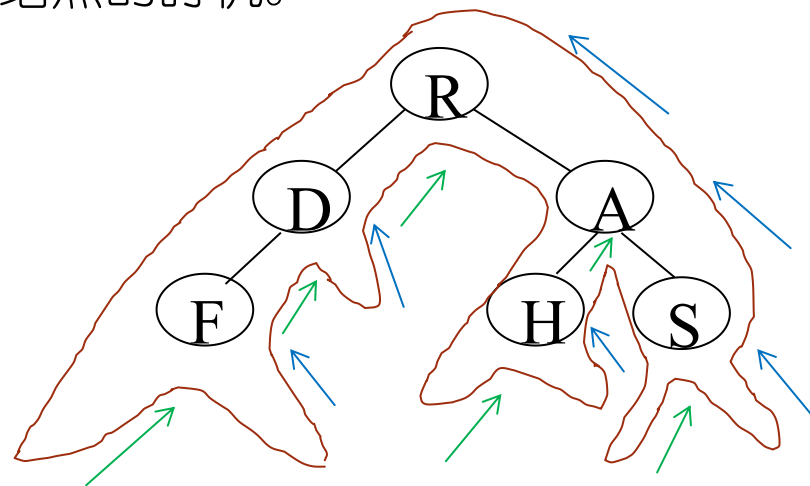
```
p=stack[top];  
top --;      //从栈中弹出栈顶元素  
Visit(p->data); //在从左孩子返回时访问  
p=p->rchild;  //到栈顶元素右子树
```

6.3 遍历

(3) 后序遍历的非递归实现

蓝色箭头方向是后续遍历时访问结点的时机。

对沿着包络线进行的算法而言：



p是否为空

如果不为空，把p指向的结点入栈，然后 $p = p \rightarrow \text{lchild}$

如果为空，出栈一个结点， $p = \text{刚出栈的结点的} \rightarrow \text{rchild}$

应该在完成rchild的访问之后进行对p的访问，此时p已经出栈，找不到了；

改进思路：

将p再次保存起来，待右孩子访问完毕后再访问。

带来2个问题：1) p保存在哪里？ 2) 何时把p弹出来用？

6.3 遍历

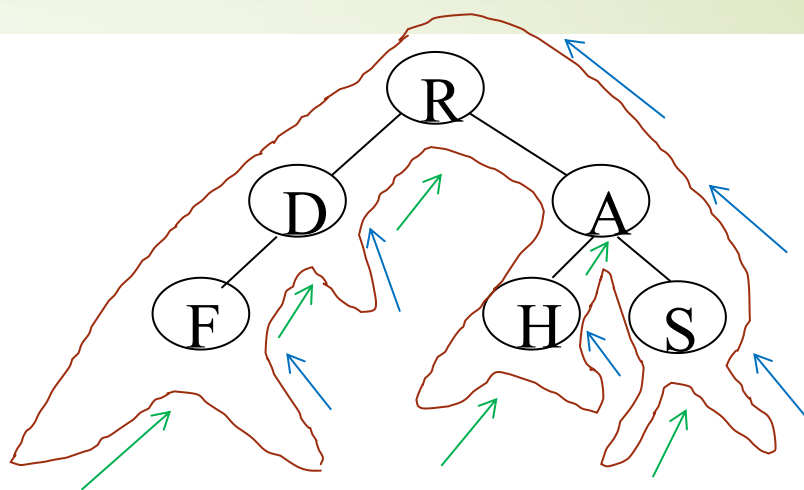
(3) 后序遍历的非递归实现

先看问题2：何时把p弹出来用？

p是否为空

如果不为空，把p指向的结点入栈，然后 $p = p \rightarrow lchild$

如果为空，出栈一个结点， $p = \text{刚出栈的结点的} \rightarrow rchild$



在上述的包络线前进的算法中，看不到什么时候可以用这个保留的p，需要增加一些信息：**即此时进行的是某个结点的右子树的包络线前进**

再看问题1：p保存起来，p的右子树的根结点（ $p \rightarrow rchild$ 非空时）也要保存起来，而且先于p进行访问，需要用栈。

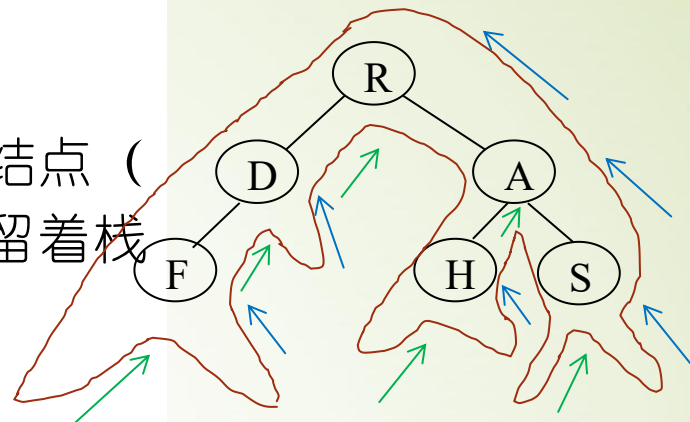
能不能用同一个栈来解决2个问题？

6.3 遍历

(3)后序遍历的非递归实现

解决办法：

当要进入右子树时，将当前栈顶的结点（p结点）做一个特殊标记，并且让其留着栈中



这样从栈顶结点就可以判断出是不是右子树返回了，同时也把p结点保存下来了。

用 $\text{stack}[\text{top}] = -\text{stack}[\text{top}]$ 的方式，将指针置为负值，即做了标记；在使用时，将其再次乘以-1即可恢复。

完成沿包络线深入到最深的左子树后，需要从栈中弹出时，判断栈顶元素的正负值，来判定是从左子树回来的，还是右子树回来的，即可完成我们的2个问题了。

(3) 后序遍历的非递归实现

【算

```
void NRPostOrder(BiTree bt)
```

```
{ BiTNode *stack[MAXNODE];
```

```
  int top; BiTNode *p;
```

```
  if (bt==NULL) return;
```

```
  top=-1; p=bt; //p指向根结点
```

```
  while( !( p==NULL && top == -1) )
```

```
  { while( p!=NULL)
```

```
    { //将当前指针p压栈，设栈空间足够大
```

```
      top++;      stack[top]=p;      p=p->lchild;      //指针指向p的左孩子结点
```

```
    }
```

```
    if (top>-1)      //栈不空
```

```
      if(stack[top]>0)      //第一次在栈中，左子树返回
```

```
      { p=stack[top]->rchild;      //转向右子树
```

```
        stack[top]= -stack[top];      //相当于结点第一次出栈、第二次进栈
```

```
      }
```

```
      else      //第二次在栈中，右子树返回
```

```
      { p=-stack[top];      //真正出栈，访问后不需要再保存了
```

```
        top--;
```

```
        Visit (p);
```

```
        p=NULL;      //用于标记当前子树都访问完了
```

```
      }
```

```
    }
```

```
  }
```

这里用将指针值置为负数来表示算法的意思，实际使用中，需要定义一个结构体，将指针值和标志放在里面，对标志进行正负的设置和判断。如：

```
typedef struct
```

```
{
```

```
    BiTNode * bt;
```

```
    int      nFlag;
```

```
}BiTNodeWithFlag;
```

栈也需要改成保存上述结构体结点的类型。

[教材P131 6.3算法 中序遍历非递归算法二]

LDR

Status **InOrderTraverse2**(BiTree T)

{

InitStack(S); p = T;

while (p || ! StackEmpty (S)) {
 if (p) {Push(S,p); p=p->lchild; }

 else {

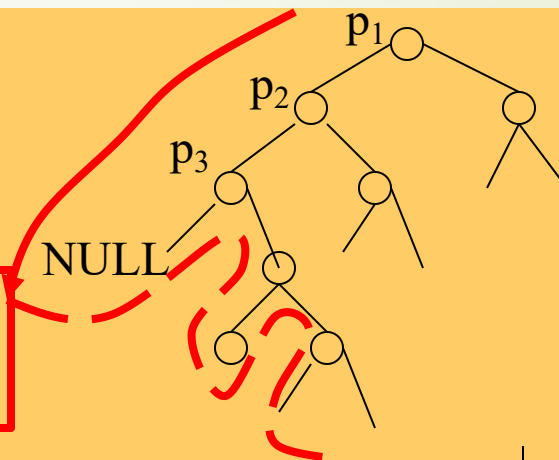
 pop(S, p); if(!Visit(p->data))return ERROR;

 p=p->rchild;

 }

return OK;

}// InOrderTraverse2 P131-6.3



while(p!=NULL)
{
 top++;
 stack[top]=p;
 p=p->lchild; //将当前指针p压栈
 //指针指向p的左孩子结点
}

~~p3~~
p2
p1

[教材P130 6.2算法 中序遍

```
while (p || ! StackEmpty (S)) {  
    if (p) {Push(S,p); p=p->lchild; }  
}
```

Status **InOrderTraverse1**(BiTree T)

```
{ InitStack(S); Push(S, T);  
  while (! StackEmpty (S)) {  
    while (GetTop(S,p) && p)  
      Push(S, p->lchild);
```

```
    Pop(S, p);
```

```
    if (!StackEmpty(S)) {
```

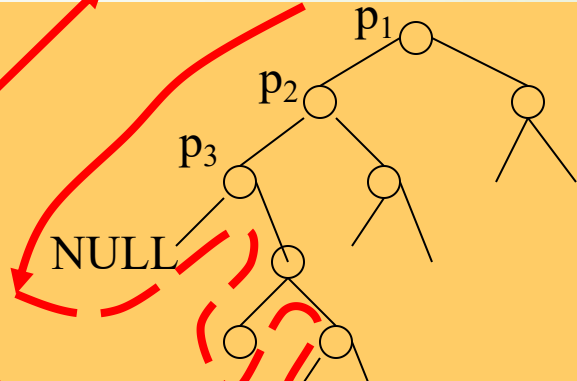
```
      Pop(S, p); if (!Visit(p->data)) return ERROR;
```

```
      Push(S, p->rchild);
```

```
    }
```

```
  }return OK;
```

```
}// InOrderTraverse1    P130-6.2
```



```
while(p!=NULL)  
{  
    top++;  
    stack[top]=p;    //将当前指针p压栈  
    p=p->lchild;    //指针指向p的左孩子结点  
}
```

~~NULL~~

~~p3~~

p2

p1

教材中两种方法的区别是（二）
的处理要简洁些，空指针不需
要入栈，而方法一有多次的空
指针入栈、出栈操作。

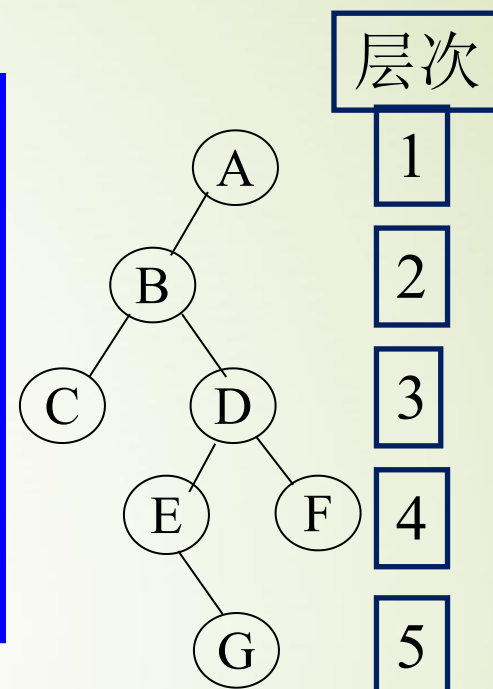
层序遍历算法分析

要求:

从根开始, 第一层从左到右, 第二层从左到右...

思路:

当从第1层开始时, 访问当前点, 还可以知道根的lchild, rchild, 到第二层时, 如何找到自己的兄弟? 堂兄弟?



用队列进行记录。

- 1、从根开始, 访问当前结点, 将lchild和rchild入队列;
- 2、从队列头取出结点访问, 将此结点的lchild和rchild入队列;
- 3、不断重复2, 直到队列空。

A: 在入队列时, 将自己的层加1同时记录

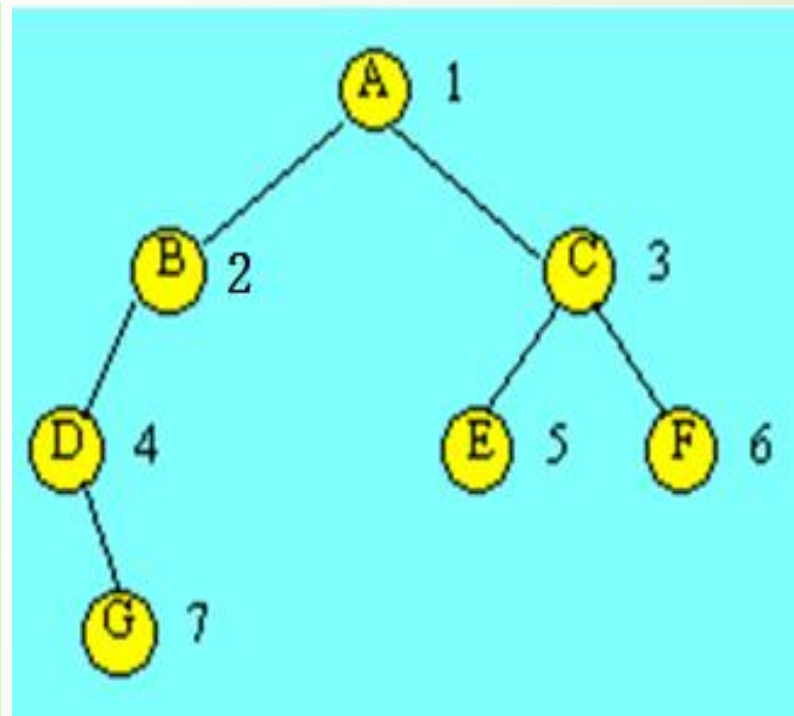
Q: 如何记录当前结点是第几层?

二叉树的 层序遍历 的算法思路

所谓二叉树的层次遍历，是指从二叉树的第一层（根结点）开始，从上至下逐层遍历，在同一层中，则按从左到右的顺序对结点逐个访问。对于下图所示的二叉树，按层次遍历所得到的结果序列为：A B C D E F G

算法思路：

- 1、根是第一层，直接访问
- 2、第二层是根的左右子树，**如果我们保存了根的信息**，此时可以**拿出来**，访问其左、右子树即可；
- 3、第三层是第二层结点的左右子树，**如果我们保存了第二层的结点**，此时可以**拿出来**，依次输出其左右子树
- 4、依次类推，直到所有层均结束



FIFO性质，用**队列**来保存

FIFO性质，用**队列**来保存

入队列：当访问结点时，将其左右子树的根结点依次入队列

出队列：依次从队列头取出结点来进行访问

在进行层次遍历时，可设置一个队列结构，遍历从二叉树的根结点开始：初始化：向将根结点入队列

- 1) 从队列头取出一个元素
- 2) 访问该元素所指结点；
- 3) 若该元素所指结点的左、右孩子结点非空，则将该元素所指结点的左孩子指针和右孩子指针顺序入队。
- 4) 当队列不为空时，回到1) 继续

【算法6-10】 层次遍历二叉树

```
void LevelOrder(BiTree bt)
```

```
{
```

```
    BiTNode * Queue[MAXNODE]; //顺序存储的队列，单向增长的队列
```

```
    if (bt==NULL) return;
```

```
    EnQueue(Queue, bt);
```

```
    //根结点进队列
```

```
    while(!EmptyQueue(Queue))
```

```
        //队列非空时
```

```
{
```

```
    BiTNode * aNode;
```

```
    DeQueue(Queue, aNode);
```

```
    //出队列, aNode是采用引用的方式进行参数传递
```

```
    Visit (aNode);
```

```
    //访问队首结点的数据域
```

```
    if (aNode->lchild!=NULL)
```

```
        //将队首结点的左孩子结点入队列
```

```
{
```

```
    EnQueue(Queue, aNode->lchild);
```

```
}
```

```
    if (aNode->rchild!=NULL)
```

```
        //将队首结点的右孩子结点入队列
```

```
{
```

```
    EnQueue(Queue, aNode->rchild);
```

```
}
```

```
}
```

```
}
```

【算法6-10】层次遍历二叉树-2 (不使用栈的ADT操作函数)

```
void LevelOrder(BiTree bt)
```

```
{
```

```
    BiTNode *Queue[MAXNODE]; //顺序存储的队列，单向增长的队列
```

```
        //未考虑队列存储不下树的情况，请大家改造为循环队列
```

```
    int front,rear;
```

```
    if (bt==NULL) return;
```

```
    front = -1;    rear=0;
```

```
    queue[rear]=bt;
```

```
        //根结点进队列
```

```
    while(front!=rear)
```

```
        //队列非空时
```

```
{
```

```
    front++;
```

```
        //出队列
```

```
    Visit (queue[front]);
```

```
        //访问队首结点的数据域
```

```
    if (queue[front]->lchild!=NULL)
```

```
        //将队首结点的左孩子结点入队列
```

```
    { rear++;
```

```
        queue[rear]=queue[front] ->lchild;
```

```
    }
```

```
    if (queue[front]->rchild!=NULL)
```

```
        //将队首结点的右孩子结点入队列
```

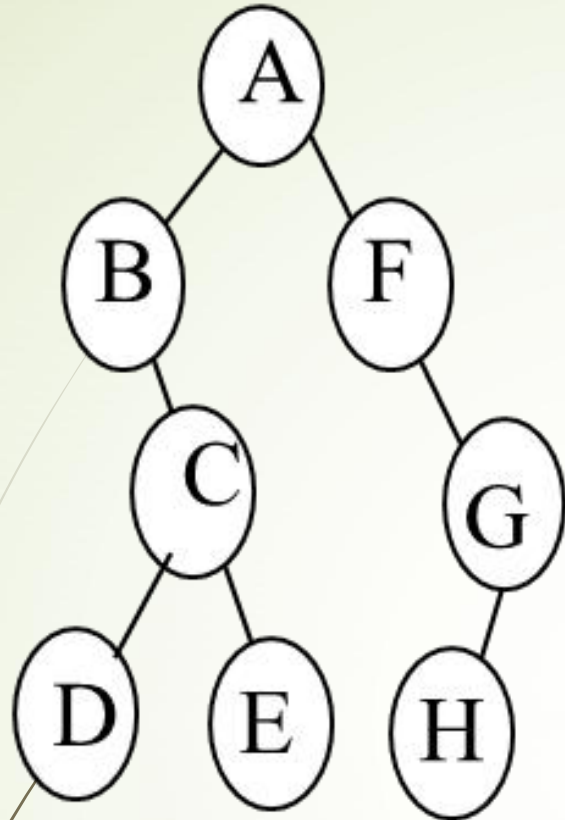
```
    { rear++;
```

```
        queue[rear]=queue[front]->rchild;
```

```
    }
```

```
}
```

```
}
```



中序: BDCEAFHG

后序: DECBHGFA

先序: ABCDEFGH

补充空指针为 ‘!’ 后的先序
AB!CD!!E!!F!GH!!!

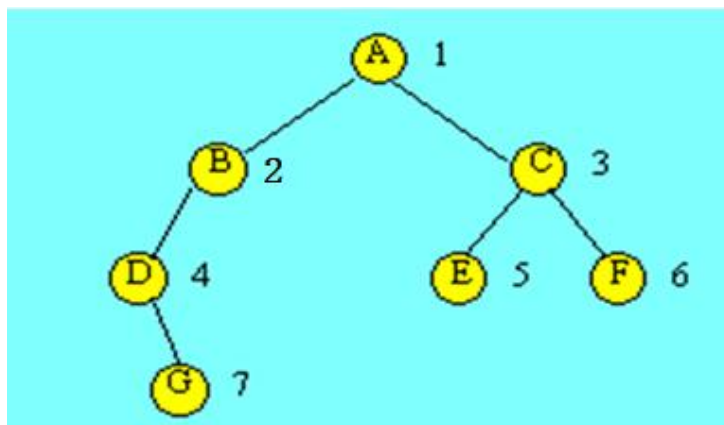
6.4 二叉树遍历的应用

6.4 应用

6.4.1 从字符串构造二叉树的二叉链表存储

构建一棵二叉树的二叉链表也是基于遍历的过程进行的。这里按照先序遍历的过程构建。

首先建立二叉树**带空指针的先序次序**，依此作为构建时结点的输入顺序，如对于下图所示的二叉树，输入序列为带空孩子的先序遍历序列：**ABD0G00CE00F00**（**0表示空孩子**，为了简化问题，设数据元素的类型为字符型）。



【算法6-11】 建立二叉树的二叉链表 (写法1, 返回值为指针)

//用带空孩子的先序遍历序列, 构造二叉链表存储的二叉树

BiTree CreateBinTree()

```
{    BiTree T;
    char ch;
    scanf("%c",&ch);
    if (ch=='0') T=NULL;           //读入0时, 将相应结点置空
    else
    {
        T=(BinTNode*)malloc(sizeof(BinTNode)); //生成结点空间
        T->data=ch;
        T->lchild = CreateBinTree();           //构造二叉树的左子树
        T->rchild = CreateBinTree();           //构造二叉树的右子树
    }
    return T;
}

main()
{
    BiTree bt = CreateBinTree();
}
```

【算法6-11】 建立二叉树的二叉链表（写法2，参数为指向指针的指针，即BiTree的地址；BiTree是指针类型的变量，这里传入的参数即为此变量的地址）
//用带空孩子的先序遍历序列，构造二叉链表存储的二叉树

```
void CreateBinTree(BiTree *T)
{
    char ch;
    scanf("%c", &ch);
    if (ch=='0')
        *T=NULL;          //读入0时，将相应结点置空
    else
    {
        *T=(BiTNode*)malloc(sizeof(BiTNode));          //生成结点空间
        (*T)->data=ch;
        CreateBinTree(&((*T)->lchild));                  //构造二叉树的左子树
        CreateBinTree(&((*T)->rchild));                  //构造二叉树的右子树
    }
}

main()
{
    BiTree bt;
    CreateBinTree(&bt);
}
```

6.4.2 在二叉树中查找值为x的数据元素

6.4 应用

【算法6-12】 在二叉树bt中查找值为x的数据元素

```
BiTree Search(BiTree bt, datatype x)
```

```
{ BiTree p=NULL;
```

```
  if(bt)
```

```
  {
```

```
    if (bt->data==x) return bt;    //查找成功返回，递归结束条件；
```

```
    if (bt->lchild)
```

```
      p=Search(bt->lchild, x);    //若有左子树，在左子树中继续查找
```

```
    if( p) return p;             //左子树中查到了，不需要继续了
```

```
    if (bt->rchild)
```

```
      p=Search(bt->rchild, x);    //若有右子树，在右子树中继续查找
```

```
    if( p) return p;             //右子树查到了，递归结束条件
```

```
  }
```

```
  return NULL;                  //查找失败返回，递归结束条件
```

```
}
```

6.4.3 统计给定二叉树中叶子结点的数目

6.4 应用

【算法 6-13】统计二叉树bt中叶子结点的个数

```
int CountLeaf2(BiTree bt)
```

```
{
```

```
    //bt为根结点的指针，返回值为bt的叶子数
```

```
    if (bt==NULL)
```

```
        return(0);    //当前结点为空，没有叶子结点，递归结束条件
```

```
    if (bt->lchild==NULL && bt->rchild==NULL)
```

```
        return(1);    //当前结点左右子树均没有，即为叶子，返回1；
```

```
        // 递归结束条件
```

```
    //非空，非叶子结点，即为分支结点
```

```
    //分支结点的叶子数目 = 左子树中的叶子数 + 右子树中的叶子数
```

```
    int nLeaf = CountLeaf2(bt->lchild);
```

```
    nLeaf += CountLeaf2(bt->rchild);
```

```
    Return nLeaf;
```

```
}
```

释放二叉树存储空间 ClearTree

6.4 应用

思路：找到二叉树的每个结点，释放掉即可。

问题1：如何找？用遍历方式找。

问题2：用那个遍历方式？前序、中序、后序、层序
当选择前序遍历方式时：

```
void ClearTree(BiTree bt)  // 前序
{
    if (bt==NULL) return; //递归调用的结束条件
    //bt 此时是D，如何做？
```

~~free(bt);~~

X 直接free(bt)，指针无效了，bt->lchild 和 bt->rchild都是无效的操作

```
ClearTree(bt->lchild); //先序递归遍历bt的左子树
ClearTree(bt->rchild); //先序递归遍历bt的右子树
}
```

释放二叉树存储空间 ClearTree

6.4 应用

思路：找到二叉树的每个结点，释放掉即可。

问题1：如何找？用遍历方式找。

问题2：用那个遍历方式？前序、中序、后序、层序
当选择前序遍历方式时：

```
void ClearTree(BiTree bt)  // 前序
{
    if (bt==NULL) return; //递归调用的结束条件
    //bt 此时是D，如何做？
    // 先保存bt->lchild 和 bt->rchild，用于后面释放左右子树；再释放bt
    BiTree lchild = tree->lchild;
    BiTree rchild = tree->rchild;
    free(bt);
    ClearTree(lchild); //先序递归遍历bt的左子树
    ClearTree(rchild); //先序递归遍历bt的右子树
}
```

释放二叉树存储空间 ClearTree

6.4 应用

思路：找到二叉树的每个结点，释放掉即可。

问题1：如何找？用遍历方式找。

问题2：用那个遍历方式？前序、中序、后序、层序
当选择前序遍历方式时：

```
void ClearTree(BiTree bt)    // 后序
{
    if (bt==NULL) return; //递归调用的结束条件
    ClearTree(bt->lchild)
    ClearTree(bt->rchild)
    free(bt);
}
```

中序，需用类似前序的方式进行保存；

层序？是否需要保存被释放结点的左孩子和右孩子？

不需要了，入队列的时候保存过了啊

【算法】层次遍历释放二叉树

```
void LevelOrder(BiTree bt)
```

```
{
```

```
    BiTNode * Queue[MAXNODE]; //顺序存储的队列，单向增长的队列
```

```
    if (bt==NULL) return;
```

```
    EnQueue(Queue, bt);
```

```
    //根结点进队列
```

```
    while(!EmptyQueue(Queue))
```

```
        //队列非空时
```

```
{
```

```
    BiTNode * aNode;
```

```
    DeQueue(Queue, aNode);
```

```
    //出队列, aNode是采用引用的方式进行参数传递
```

```
    Visit (aNode);
```

当用层序方式释放时，不在此处释放

```
    if (aNode->lchild!=NULL) //将队首结点的左孩子结点入队列
```

```
{
```

```
        EnQueue(Queue, aNode->lchild);
```

```
}
```

```
    if (aNode->rchild!=NULL) //将队首结点的右孩子结点入队列
```

```
{
```

```
        EnQueue(Queue, aNode->rchild);
```

```
}
```

```
}
```

在这里释放，增加一条语句：free(aNode);

【算法】层次遍历释放二叉树-2 (不使用栈的ADT操作函数)

```
void ClearTree(BiTree bt)
```

```
{  BiTNode *Queue[MAXNODE];      //顺序存储的队列，单向队列
   int front,rear;
   if (bt==NULL) return;
   front = -1;    rear=0;
   queue[rear]=bt;      //根结点进队列
   while(front!=rear)    //队列非空时
   {  front++;           //出队列
      //Visit (queue[front]); 当用层序方式释放时，不在这里释放
      if (queue[front]->lchild!=NULL) //将队首结点的左孩子结点入队列
      {  rear++;
         queue[rear]=queue[front] ->lchild;
      }
      if (queue[front]->rchild!=NULL) //将队首结点的右孩子结点入队列
      {  rear++;
         queue[rear]=queue[front]->rchild;
      }
   }
}
```

在这里释放，增加一条语句：
free(queue[front]);

6.4.4 由遍历序列恢复二叉树

6.4 应用

从前面讨论的二叉树的遍历知道，任意一棵二叉树结点的先序序列和中序序列都是唯一的。

反过来，若已知结点的先序序列和中序序列，能否确定这棵二叉树呢？这样确定的二叉树是否是唯一的呢？

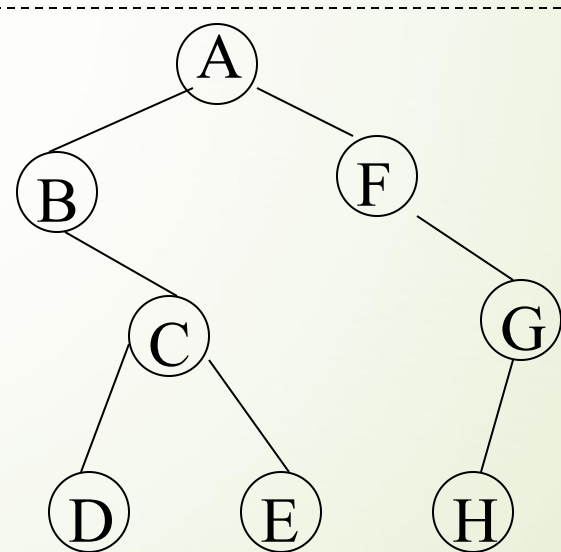
先序序列+中序序列 = 二叉树

中序序列+后序序列 = 二叉树

先序序列+后序序列 ❌

先序序列: **A**BCDEFGH

中序序列: BDC**E**AFHG

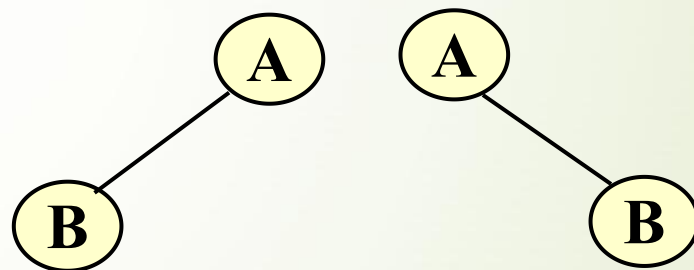


重要结论

6.4 应用

- 若二叉树中各结点的值均不相同（前提条件），则
- 由二叉树的前序序列和中序序列，
- 或由其后序序列和中序序列，均能唯一地确定一棵二叉树

- 但由前序序列和后序序列却不一定能唯一地确定一棵二叉树。



前序均为AB，后序均为BA。

- Q: 什么情况下，由前序序列和后序序列能唯一确定一棵二叉树。

6.4 应用

1、依据遍历定义：

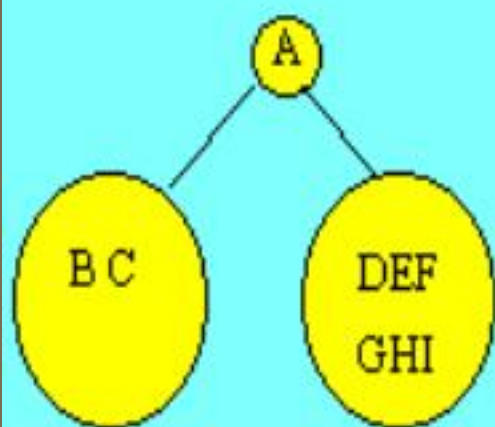
由二叉树的先序序列和中序序列可唯一地确定该二叉树。

由二叉树的后序序列和中序序列也可唯一地确定该二叉树。

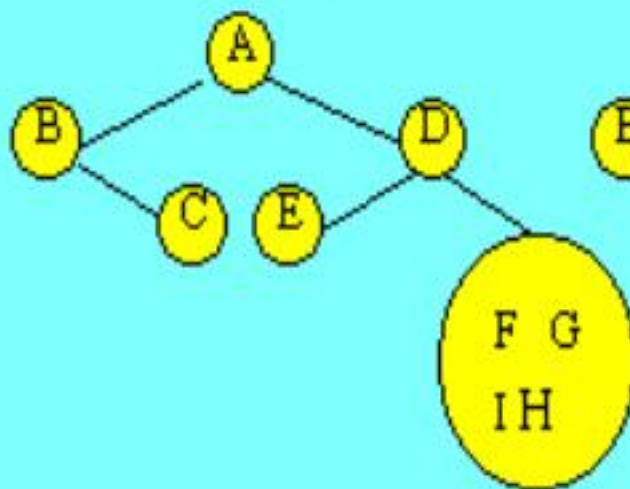
由二叉树的先序序列和后序序列不能唯一地确定该二叉树。

2、分隔过程：

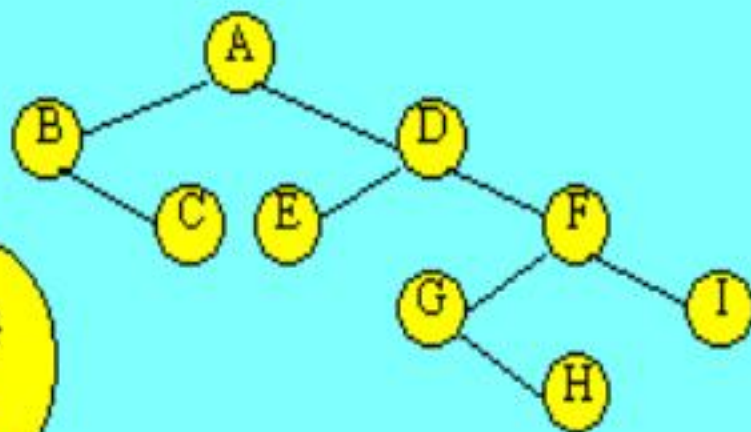
已知一棵二叉树的先序序列与中序序列分别为：A B C D E F G H I，B C A E D G H F I，试恢复该二叉树。



(a)



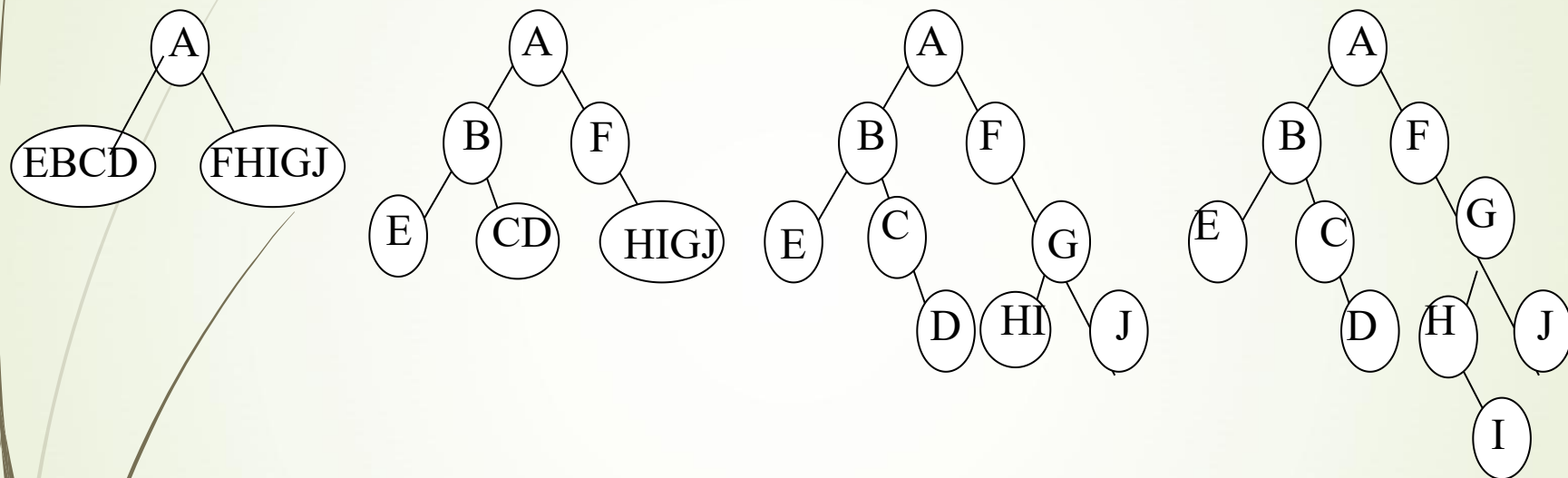
(b)



(c)

6.4 应用

当前序序列为ABECDFGHIJ，中序序列为EBCDAFHIGJ时，逐步形成二叉树的过程如下图所示：



核心思路：先根据先序序列的第一个元素建立根结点，然后在中序序列中找到该元素，确定根结点的左、右子树的中序序列，再在先序序列中确定左、右子树的先序序列；最后递归调用分别恢复左子树和右子树。

练习

6.4 应用

已知一棵二叉树的中序序列和后序序列分别是 **BDCEAFHG** 和 **DECBHGFA**，请画出这棵二叉树。

- ①由后序遍历特征，根结点必在后序序列尾部 (**A**)；
- ②由中序遍历特征，根结点必在其中间，而且其左部必全部是左子树子孙 (**BDCE**)，其右部必全部是右子树子孙 (**FHG**)；
- ③继而，根据后序中的DECB子树可确定B为A的左孩子，根据HGF子串可确定F为A的右孩子；以此类推。

6.4 应用

1) 中序: BDCE A FHG 后序: DECBHGF A

2) 中序: B DCE 后序: DEC B

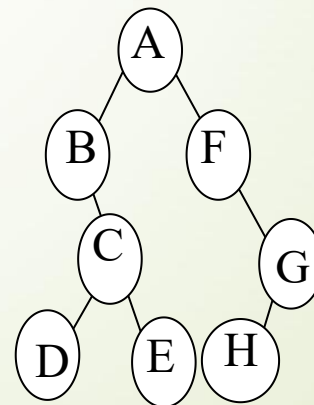
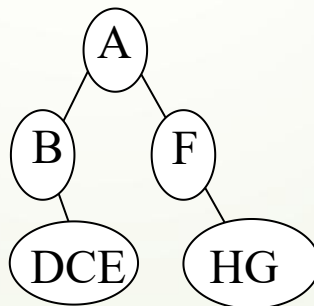
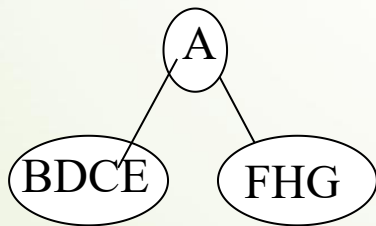
中序: F HG 后序: HG F

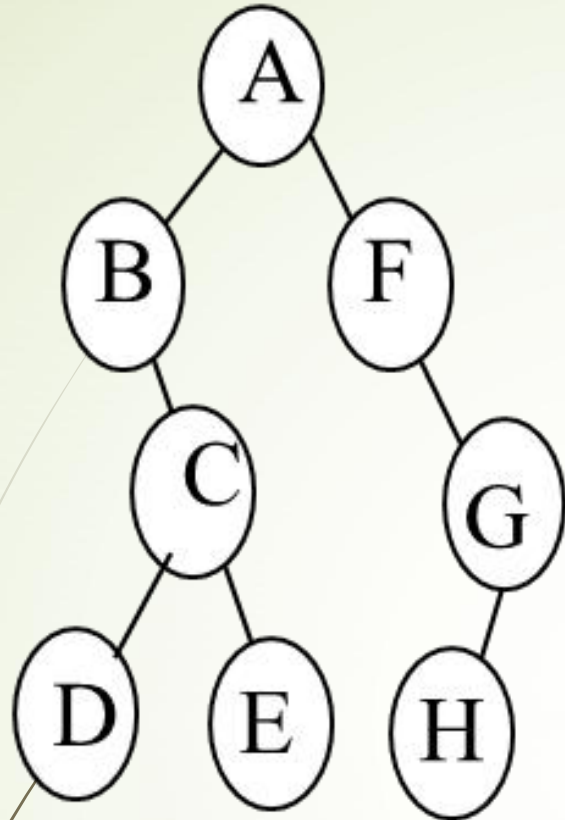
3) 中序: D C E 后序: DE C

中序: H G 后序: H G

先序:

ABCDEFGH



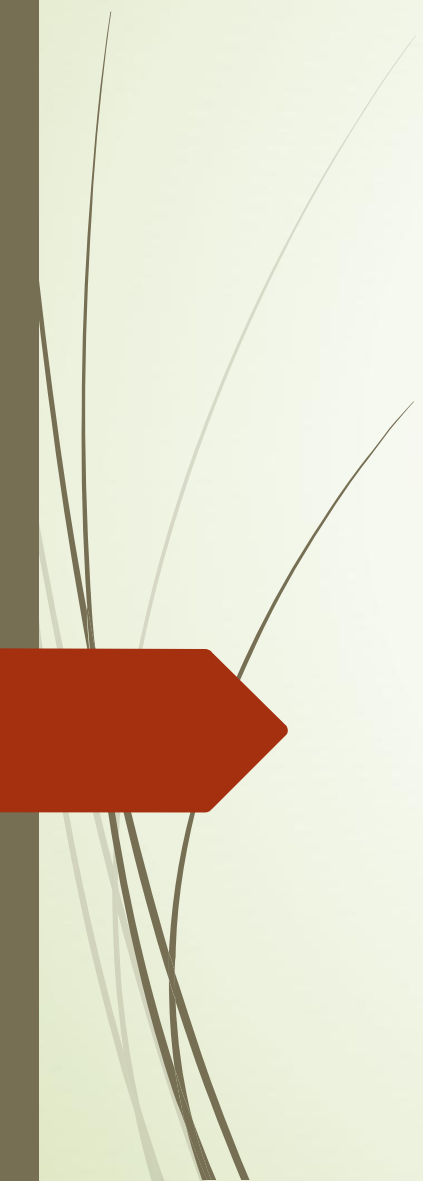


中序: BDCEAFHG

后序: DECBHGFA

先序: ABCDEFGH

补充空指针为 ‘!’ 后的先序
AB!CD!!E!!F!GH!!!



第6章 树和二叉树

赫夫曼树和线索二叉树

6.6 赫夫曼树（最优二叉树）

赫夫曼树

1问题的引入

一个将百分制转换为五级分制的判定程序可以用以下条件语句完成：

if ($a < 60$) $b = \text{"bad"}$;

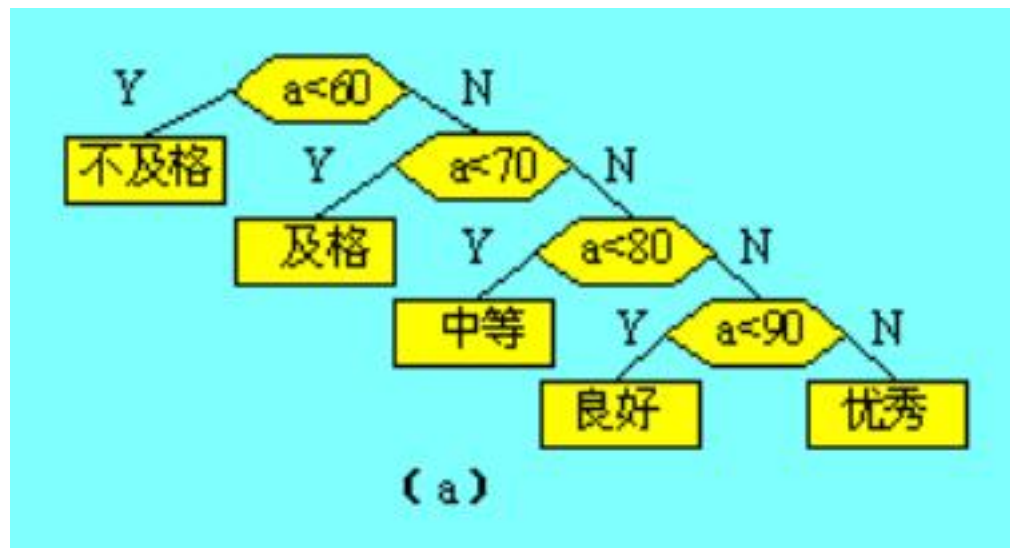
else if ($a < 70$) $b = \text{"pass"}$

else if ($a < 80$) $b = \text{"general"}$

else if ($a < 90$) $b = \text{"good"}$

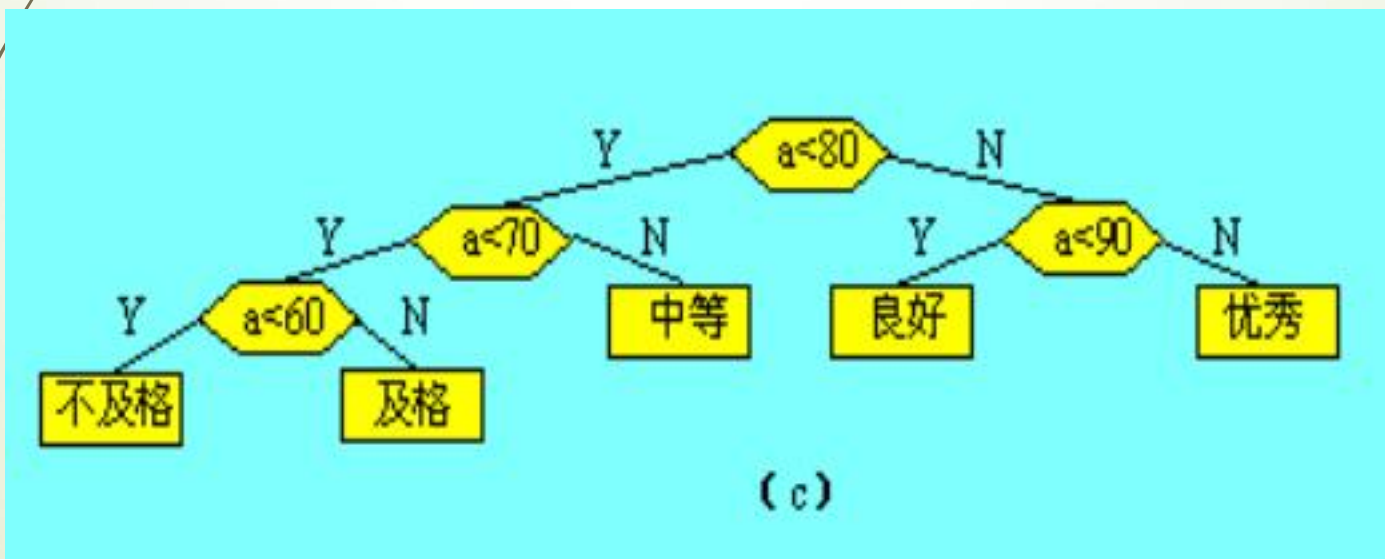
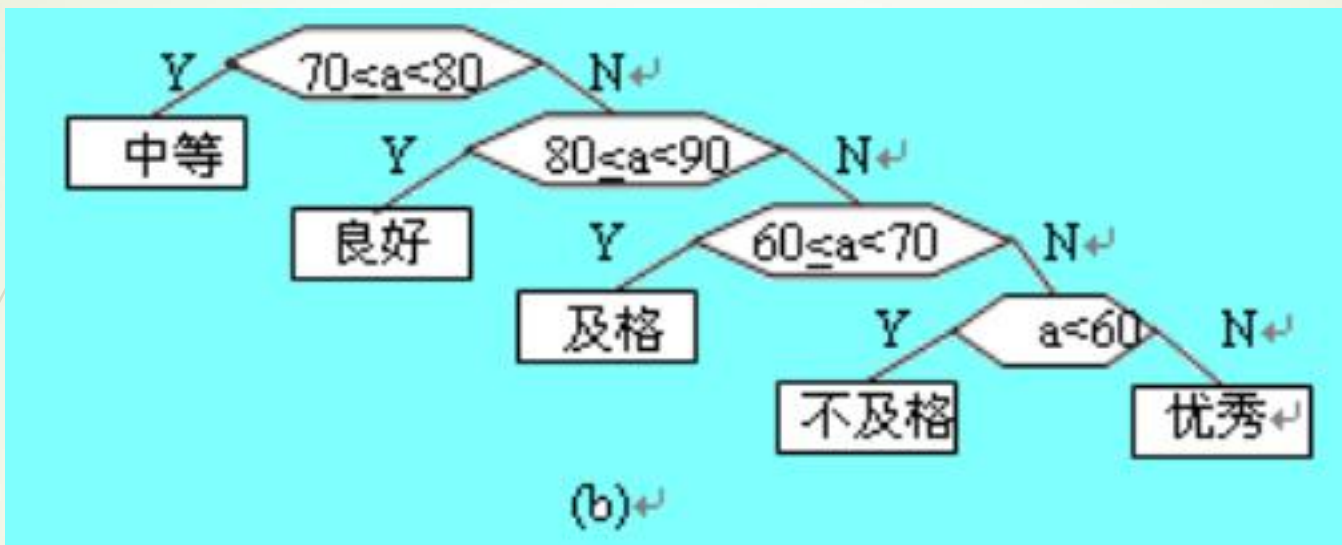
else $b = \text{"excellent"}$;

判定过程如图所示。



赫夫曼树

判定方法是多样的，也可以采用下面的判定过程来完成。

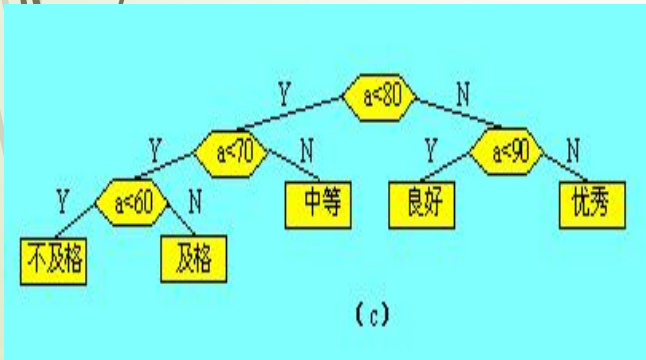
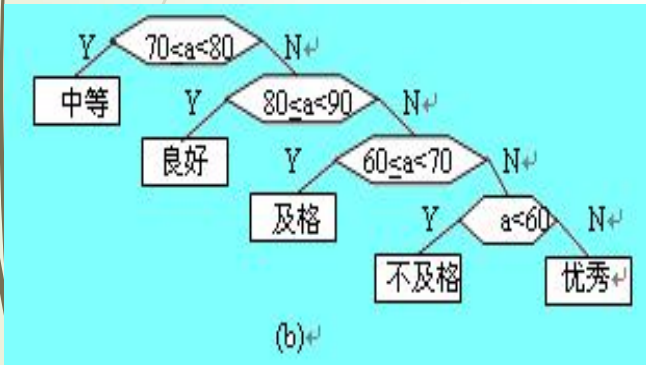
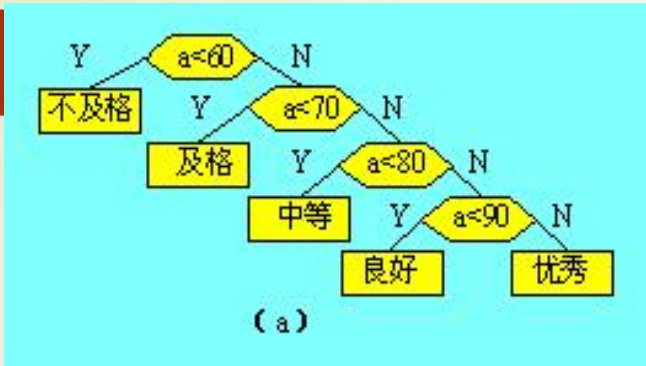


如果数据的输入量很大，则应考虑上述程序的**性能问题**，即其**计算所需要的时间**。因为在实际中，学生的成绩在五个等级上的分布是不均匀的，假设其分布规律（正态分布情况下）如下表所示：

分数区间:	0 - 59	60 - 69	70 - 79	80 - 89	90 - 100
出现的百分比:	5%	15%	40%	30%	10%

a的比较次数:	1	2	3	4	4
b的比较次数:	4	3	1	2	4
c的比较次数:	3	3	2	2	2

a的平均比较次数	3.15
b的平均比较次数	2.05
c的平均比较次数	2.2



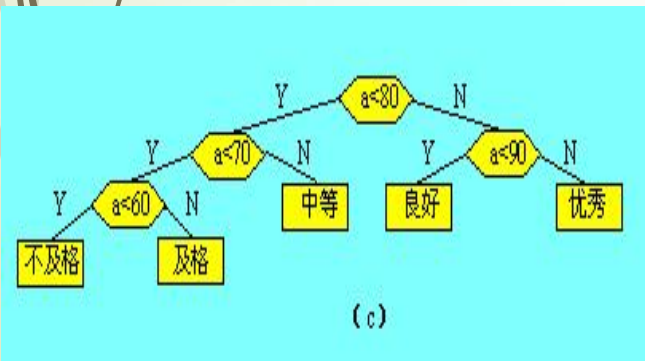
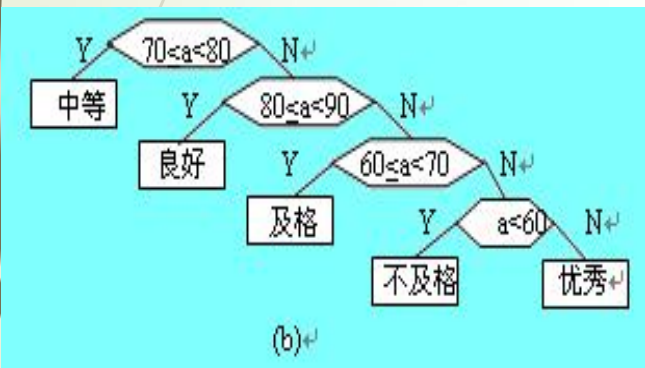
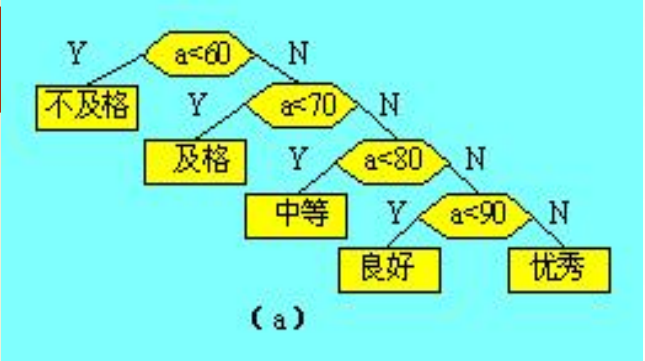
如果数据的输入量很大，则应考虑上述程序的**性能问题**，即其**计算所需要的时间**。因为在实际中，学生的成绩在五个等级上的分布是不均匀的，假设其分布规律（正态分布情况下）如下表所示：

分数区间:	0 - 59	60 - 69	70 - 79	80 - 89	90 - 100
出现的百分比:	5%	15%	40%	30%	10%

a的比较次数:	1	2	3	4	4
b的比较次数:	4	3	1	2	4
c的比较次数:	2	2	2	2	2

- 1、搜索树/比较树的**效率和数据有关系**
2、可以**根据数据来生成高效的树**
3、**高效的树，是和树的深度有关的**

b的平均比较次数	2.05
c的平均比较次数	2.2



实际上我们考虑的是树的某种平均路径长度：

赫夫曼树

结点权值：和叶子结点对应的一个有某种意义的实数(W_i)

树的路径长度：从树根到每一个结点的路径上的分支数之和。

带权路径长度：叶子结点的路径长度与该结点的权之积。

树的带权路径长度： 树中所有叶子结点的带权路径长度之和。

记作：
$$wpl = \sum_{k=1}^n w_k l_k$$

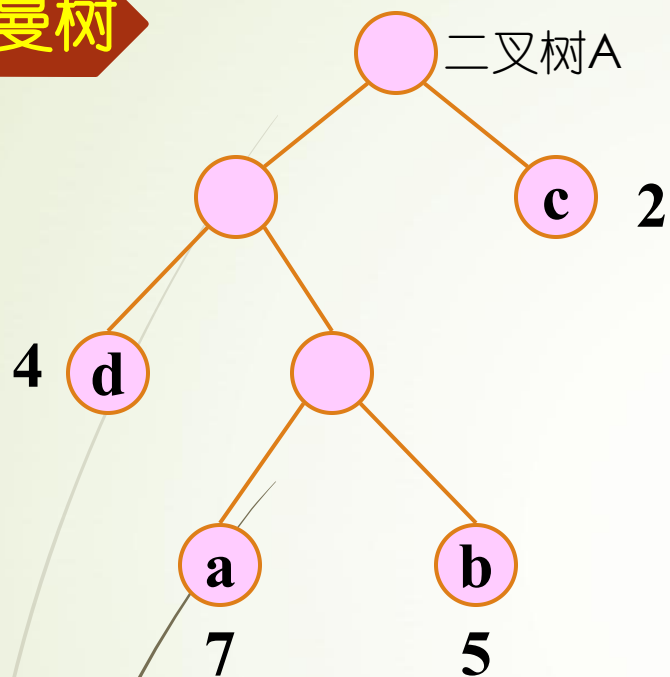
其中： w_k — 权值

l_k — 结点到根的路径长度

最优二叉树(Huffman, 赫夫曼/哈夫曼/霍夫曼树)：带权路径长度WPL最小的二叉树。

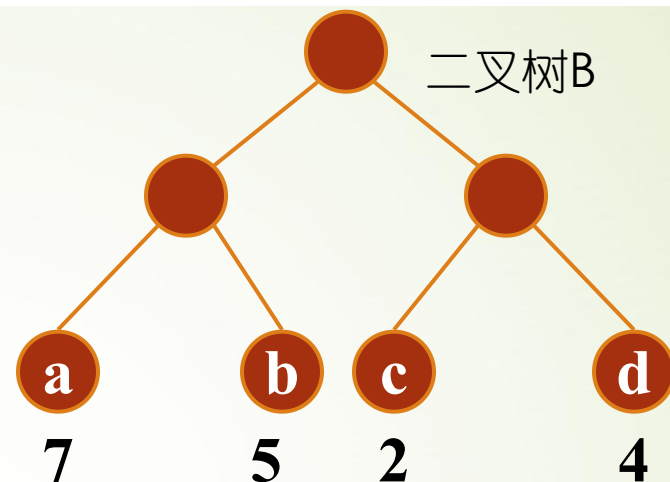
【例】 有4个结点，权值分别为7，5，2，4，构造有4个叶子结点的二叉树。

赫夫曼树



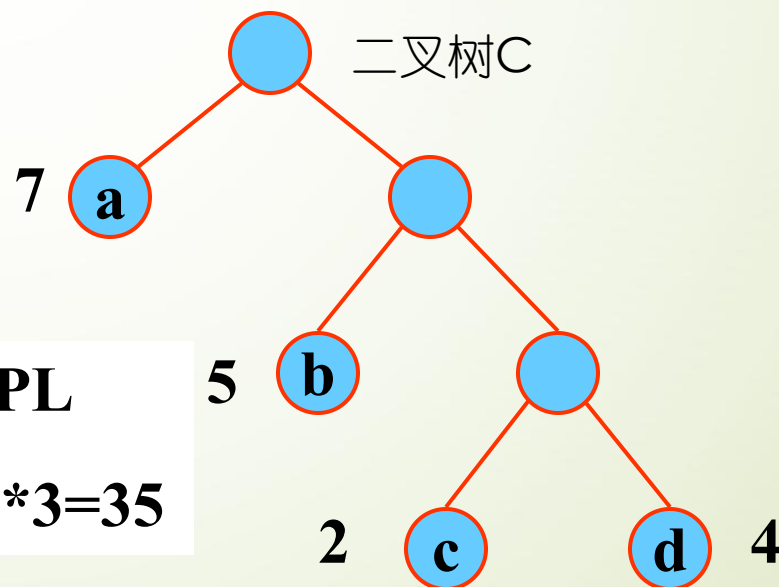
二叉树A的WPL

$$=7*3+5*3+2*1+4*2=46$$



二叉树B的WPL

$$=7*2+5*2+2*2+4*2=36$$



二叉树C的WPL

$$=7*1+5*2+2*3+4*3=35$$

赫夫曼树/哈夫曼树（最优二叉树）

赫夫曼树

在权值为 $w_1, w_2, \dots, w_n$ 的 **n 个叶子**所构成的所有二叉树中，带权路径长度最小的二叉树，称为**最优二叉树(赫夫曼树)**。

那么如何对于给定的叶子数目及其权值来构造一棵最优二叉树呢？从前图可以看出，最优二叉树中，**权值越大的叶子距离根越近**。赫夫曼首先给出了构造最优二叉树的方法，故我们称其为**赫夫曼算法**。

建立赫夫曼树（最优二叉树）的方法

赫夫曼树

[基本思想] 使权大的结点靠近根。

[算法思路]

(1) 将给定权值从小到大排序成 $\{w_1, w_2, \dots, w_m\}$ ，生成一个森林

$F = \{T_1, T_2, \dots, T_m\}$ ，其中 T_i 是一个带权 w_i 的根结点，它的左右子树均空。

(2) 把 F 中根的权值最小的两棵二叉树 T_1 和 T_2 合并成一棵新的二叉树 T ： T 的左子树是 T_1 ，右子树是 T_2 ， T 的根的权值是 T_1 、 T_2 树根结点权值之和。

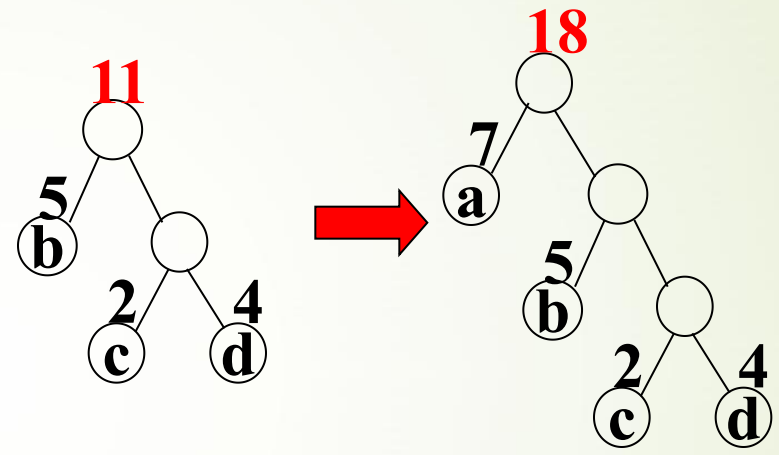
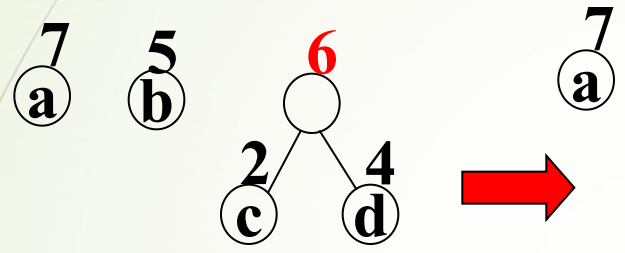
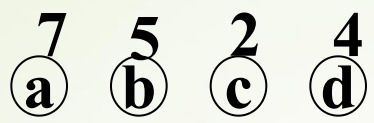
(3) 将 T 按权值大小加入 F 中，同时从 F 中删去 T_1 和 T_2

(4) 重复(2)和(3)，直到 F 中只含一棵树为止，该树即为所求。

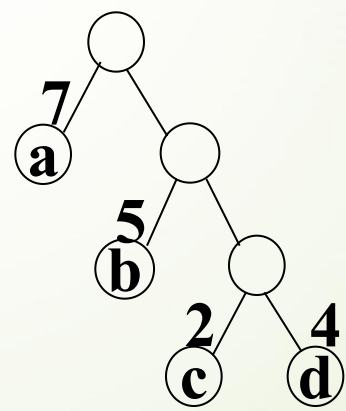
赫夫曼树的构造过程

赫夫曼树

基本思想：使权大的结点靠近根

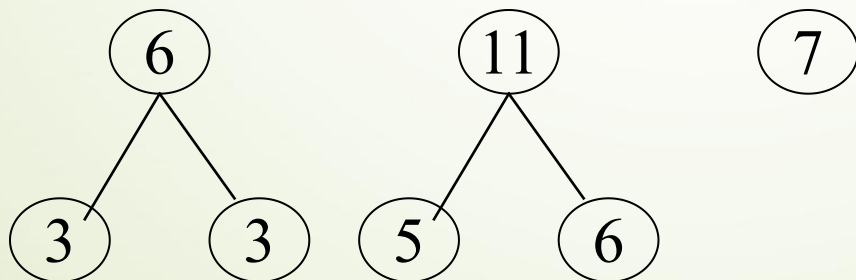
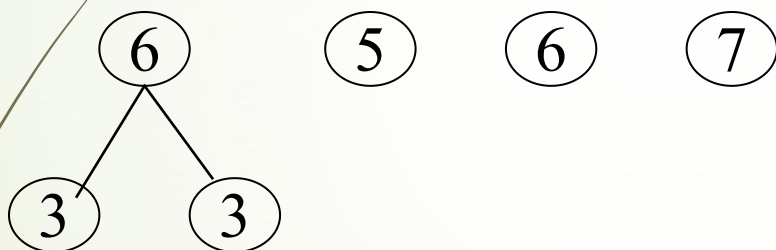


操作要点：对权值的合并、删除与替换，总是合并当前值最小的两个



赫夫曼树

例1: 3, 3, 5, 6, 7



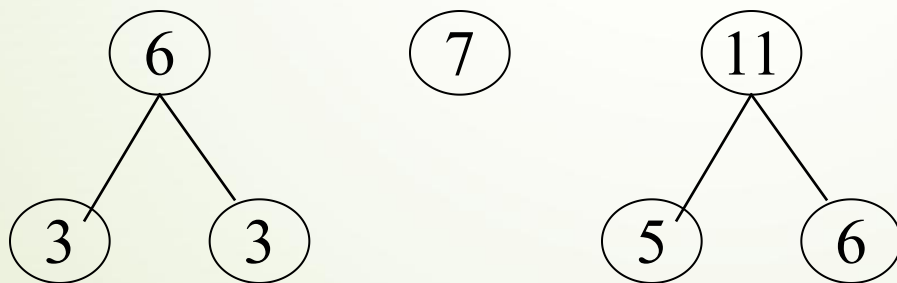
基本思想: 使权大的结点靠近根

操作要点: 对权值的合并、删除与替换, 总是合并当前值最小的两个

赫夫曼树

例1: 3, 3, 5, 6, 7

基本思想: 使
权大的结点靠
近根

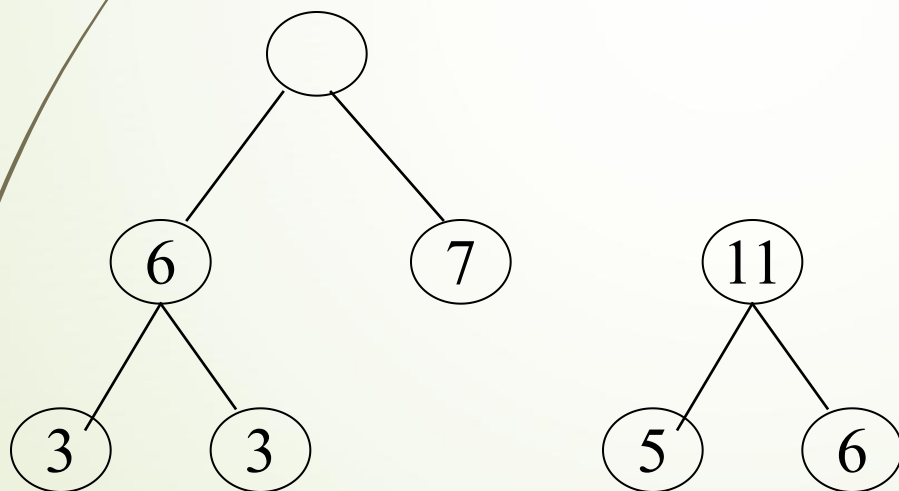


操作要点: 对权
值的**合并、删除
与替换**, 总是合
并当前值最小的
两个

赫夫曼树

例1: 3, 3, 5, 6, 7

基本思想: 使权大的结点靠近根

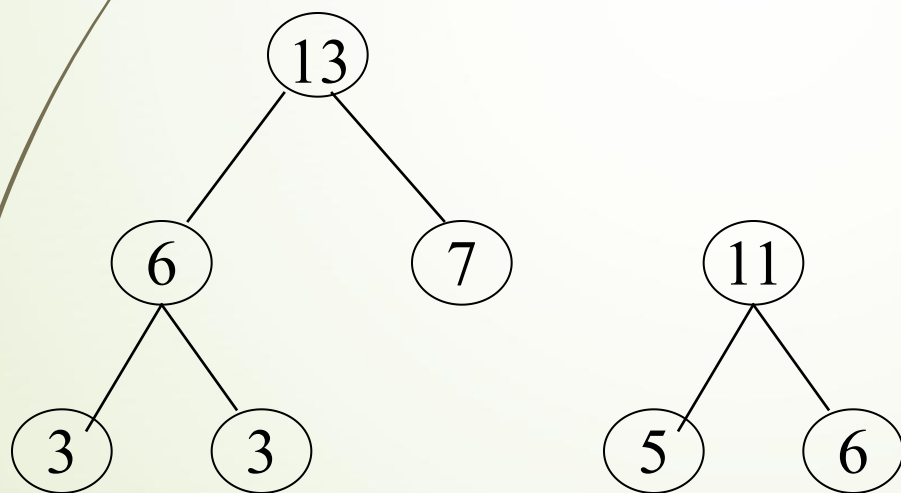


操作要点: 对权值的合并、删除与替换, 总是合并当前值最小的两个

赫夫曼树

例1: 3, 3, 5, 6, 7

基本思想: 使权大的结点靠近根

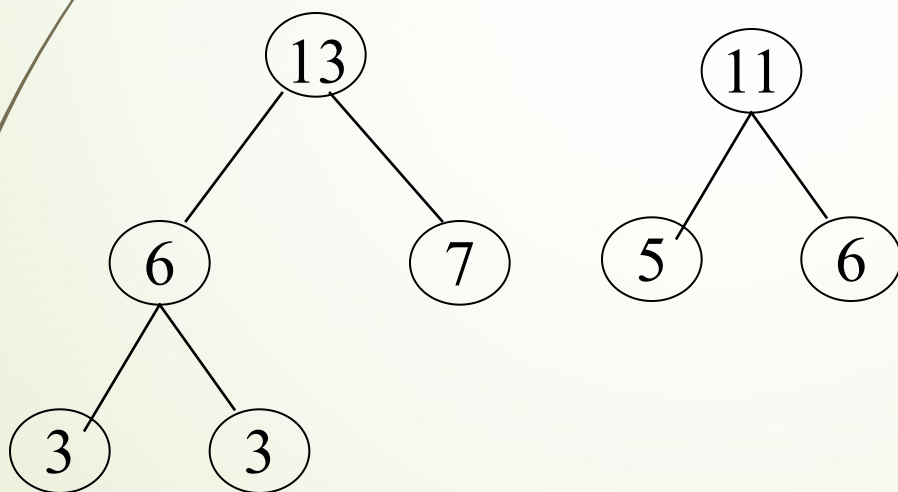


操作要点: 对权值的合并、删除与替换, 总是合并当前值最小的两个

赫夫曼树

例1: 3, 3, 5, 6, 7

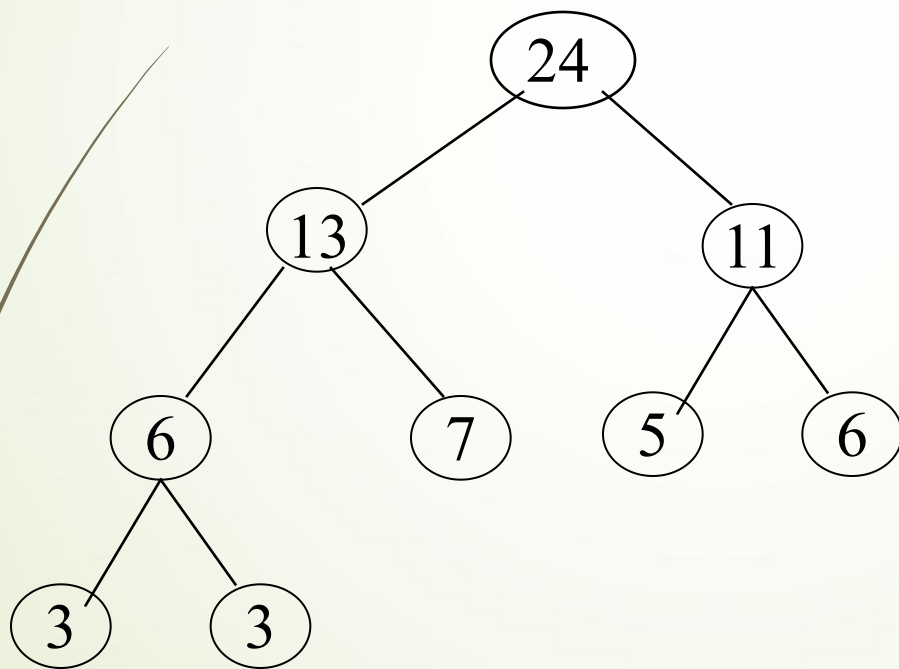
基本思想: 使权大的结点靠近根



操作要点: 对权值的合并、删除与替换, 总是合并当前值最小的两个

赫夫曼树

例1: 3, 3, 5, 6, 7



基本思想：使权大的结点靠近根

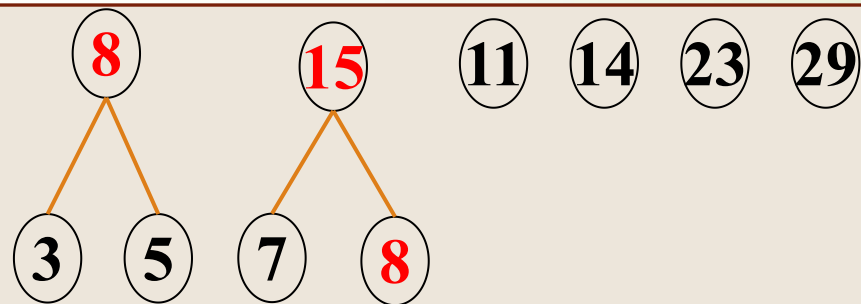
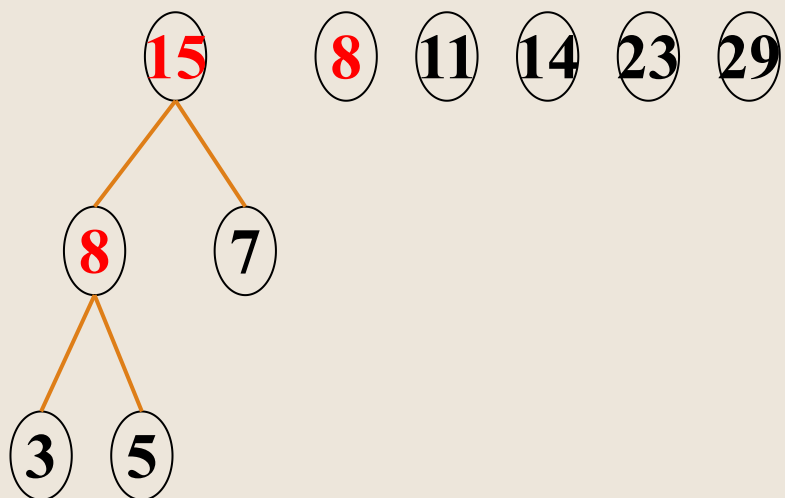
操作要点：对权值的合并、删除与替换，总是合并当前值最小的两个

赫夫曼树的构造过程---例5.2

赫夫曼树

原始数据	5	29	7	8	14	23	3	11
排序后	3	5	7	8	11	14	23	29

③ ⑤ ⑦ ⑧ ⑪ ⑭ ⑳ ㉑



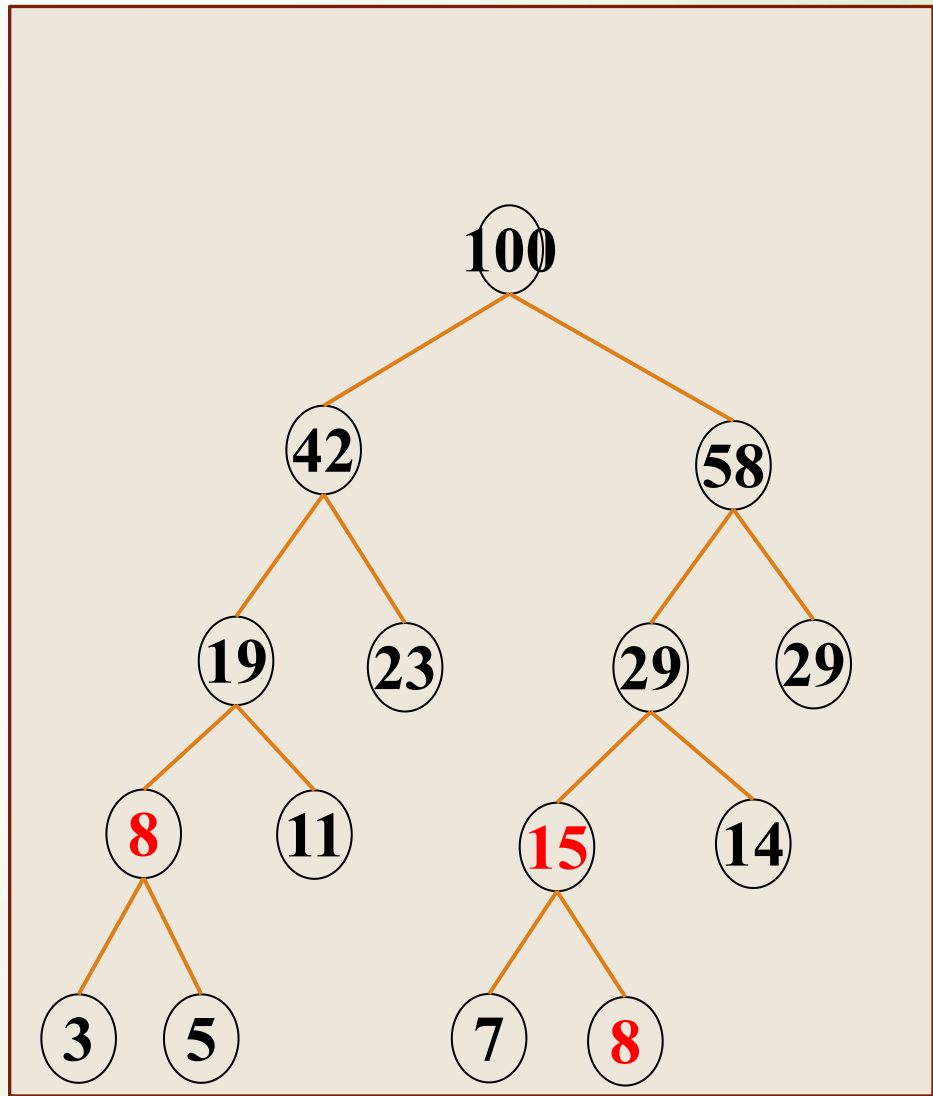
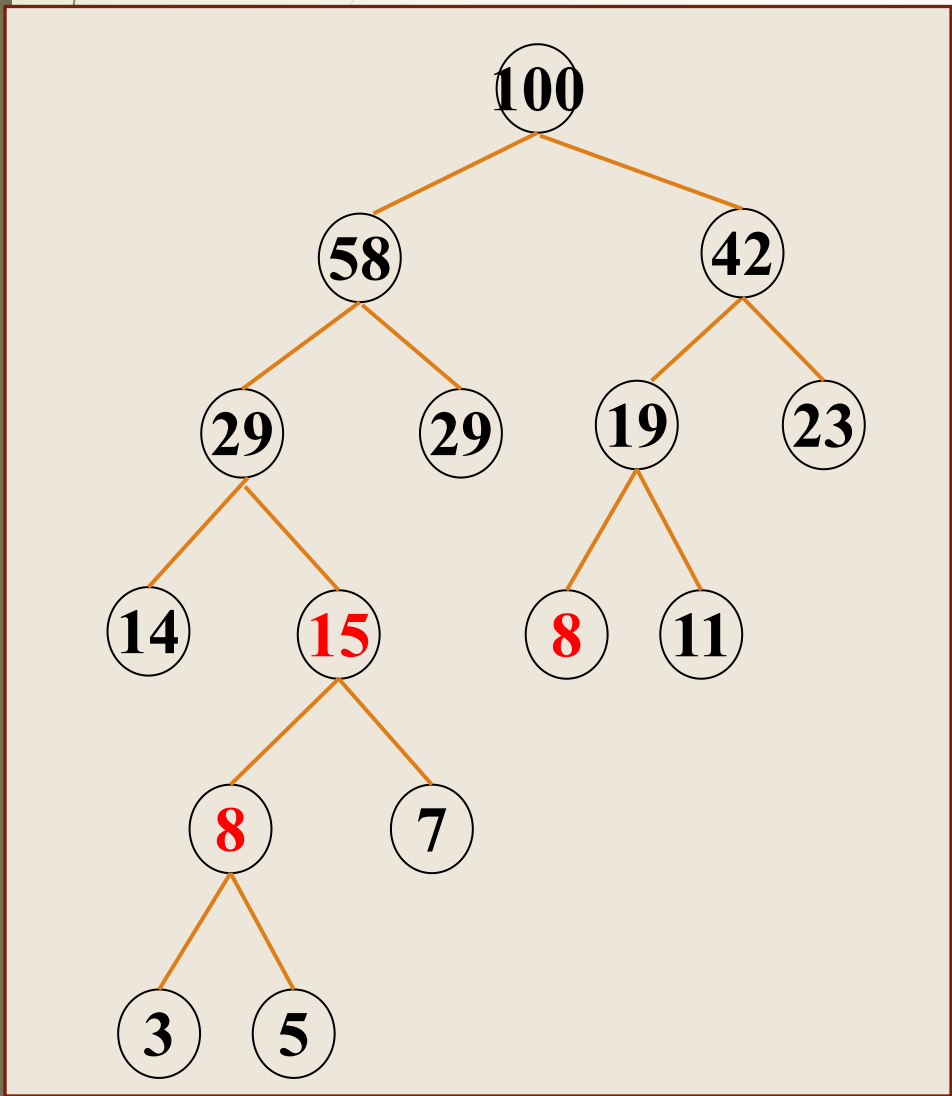
Q1: 树的根节点相同的权重情况下, 怎么选?

赫夫曼树的构造过程---例5.2

赫夫曼树

	5	29	7	8	14	23	3	11
排序	3	5	7	8	11	14	23	29

Q1: 树的根节点相同的权重情况下，怎么选？
任选1个



赫夫曼树的存储结构

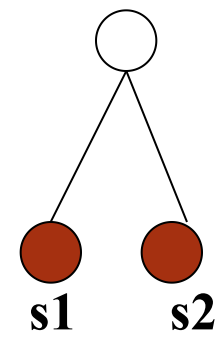
```
typedef struct {
    unsigned int  weight;
    unsigned int  parent, lchild, rchild;
}HTNode, *HuffmanTree;
//在数组中构造链式关系，便于从叶节点构造到根
//采用动态分配存储空间方式构造一个数组方式的存储
```

赫夫曼树构造过程决定了没有度为1的节点。
由性质3: $n_0 = n_2 + 1$, 得到全部结点数为 $2n-1$ 个

- ➡ 初始有n个叶子结点，则构造出的赫夫曼树共有 **$2n-1$** 个结点。
- ➡ 用大小为 $2n$ 的向量存储赫夫曼树——顺序存储。

	0	1	2	s1	s2	n	n+1				2n-1
weight		3	4		1		1		2		
lchild		0	0	...	0		0	...	0		
rchild		0	0	...	0		0	...	0		
parent											

注：用0表示空指针



赫夫曼树的存储结构

HuffmanTree HT;

- 1) 按 $2n$ 个存储单位申请空间并初始化 $HT[1..2n-1]$:
 $weight=0$, $lchild=rchild=parent=0$
将 n 个结点合并为1个二叉树需要 $n-1$ 个非终端节点。
- 2) 将 n 个节点值置于 $HT[1..n]$ 的 $weight$ 字段
- 3) 进行以下 $n-1$ 次合并, 依次产生 $HT[i]$, $i=n+1..2n-1$:
 - 3.1) 在 $HT[1..i-1]$ 中选两个需要合并的子树, 并且子树权重最小: 即 $parent==0$ 并且 $weight$ 最小的两个子树根结点 $HT[s1]$ 和 $HT[s2]$
 - 3.2) 给子树 $HT[s1]$ 和 $HT[s2]$ 合并后的 $parent$ 找个位置 i , 即 $HT[s1]$ 和 $HT[s2]$ 的 $parent=i$ 。
 - 3.3) 置 $HT[i]$ 的 $weight=HT[s1].weight + HT[s2].weight$, $lchild=s1$, $rchild=s2$ 。回到3.1) 继续, 直到 $i>2n-1$

赫夫曼树的构造算法

```
bool BuildHT(HuffmanTree &HT, int *w, int n)
// w存放n个叶结点的权值(均>0), 构造赫夫曼树HT
{ if ( n<=1 ) { HT=NULL; return false; }
  m=2*n-1;      //计算总结点个数
  HT=(HuffmanTree)malloc((m+1)sizeof(HTNode));
  if(!HT) return false;
  //初始化HT
  for (p=HT+1, i=1; i<=n; ++i, ++p, ++w) //前n个有权重
    *p={*w,0,0,0};
  for (; i<=m; ++i, ++p) //后n-1个无权重, 待合并结点时填上
    *p={0,0,0,0};
  for (i=n+1; i<=m; ++i) { //建赫夫曼树
    Select(HT, i-1, s1, s2);
    //在HT[1..i-1]中选择parent为0且权值最小的两个结点
    HT[s1].parent=i; HT[s2].parent=i;
    HT[i].lchild=s1; HT[i].rchild=s2;
    HT[i].weight=HT[s1].weight+HT[s2].weight;
  }
  return true;
} //BuildHT
```

赫夫曼树应用实例——赫夫曼编码

赫夫曼树

在远程通讯中，要将待传字符转换成二进制的字符串，怎样编码才能使它们组成的报文在网络中传得最快？

字符	编码
A	00
B	01
C	10
D	11

ABACCDAA

字符	编码
A	0
B	00
C	1
D	01

00010010101100

000011010

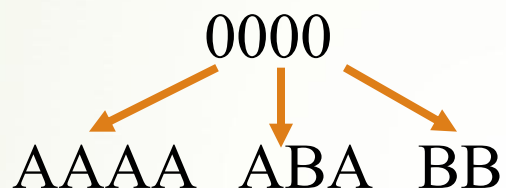
出现次数较多的字符采用尽可能短的编码

赫夫曼树应用实例——赫夫曼编码

赫夫曼树

ABACCDAA

A	0
B	00
C	1
D	01



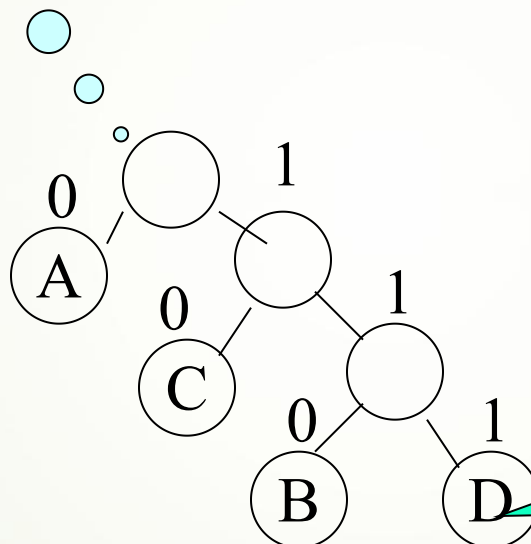
重码

000011010

关键：要设计长度不等的编码，则必须使任一字符的编码都不是另一个字符的编码的**前缀**——**前缀编码**

赫夫曼树

采用二叉树设计
前缀编码



ABACCD A

A—0

B—110

C—10

D—111



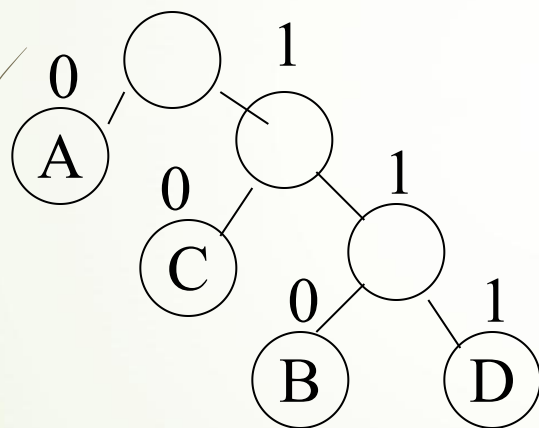
0110010101110

左分支用“0”
右分支用“1”

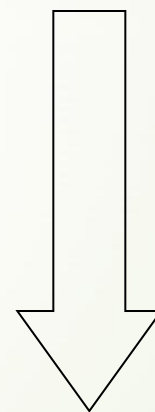
赫夫曼编码的译码过程

赫夫曼树

分解接收字符串：遇“0”向左，遇“1”向右；一旦到达叶子结点，则译出一个字符，反复由根出发，直到译码完成。



0110010101110



ABACCD A

特点：每一码都不是另一码的前缀，绝不会错译！称为前缀码

赫夫曼编码的特点

赫夫曼树

- 赫夫曼编码是不等长编码
- 赫夫曼编码是**前缀编码**，即任一字符的编码都不是另一字符编码的前缀
- 赫夫曼编码树中没有度为1的结点。若叶子结点的个数为 n ，则赫夫曼编码树的结点总数为 $2n-1$
- 发送过程：根据由赫夫曼树得到的编码表送出字符数据
- 接收过程：按左0、右1的规定，从根结点走到一个叶结点，完成一个字符的译码。反复此过程，直到接收数据结束

赫夫曼编码：

赫夫曼树

- 用于通信和数据传送中字符的二进制编码，可以使电文编码总长度最短。
 - 1、构造最优树，用需编码**字符的出现频率做权重**生成最优树(赫夫曼树)。
 - 2、根据最优树进行编解码：
 - 规定赫夫曼树中的**左分支代表0，右分支代表1**，则从**根结点到每个叶结点所经过的路径**分支组成的0和1的序列便为**该结点对应字符的编码**，我们称之为赫夫曼编码。
 - 3、解码
 - 按左0、右1的规定，从根结点走到一个叶结点，完成一个字符的译码

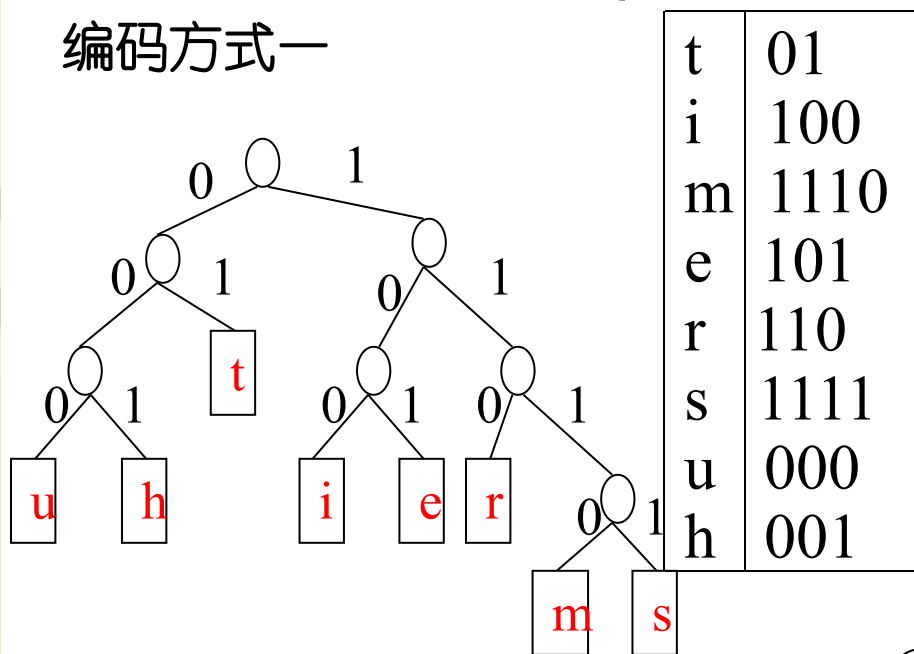
赤

【例】对time tries truth赫夫曼编码

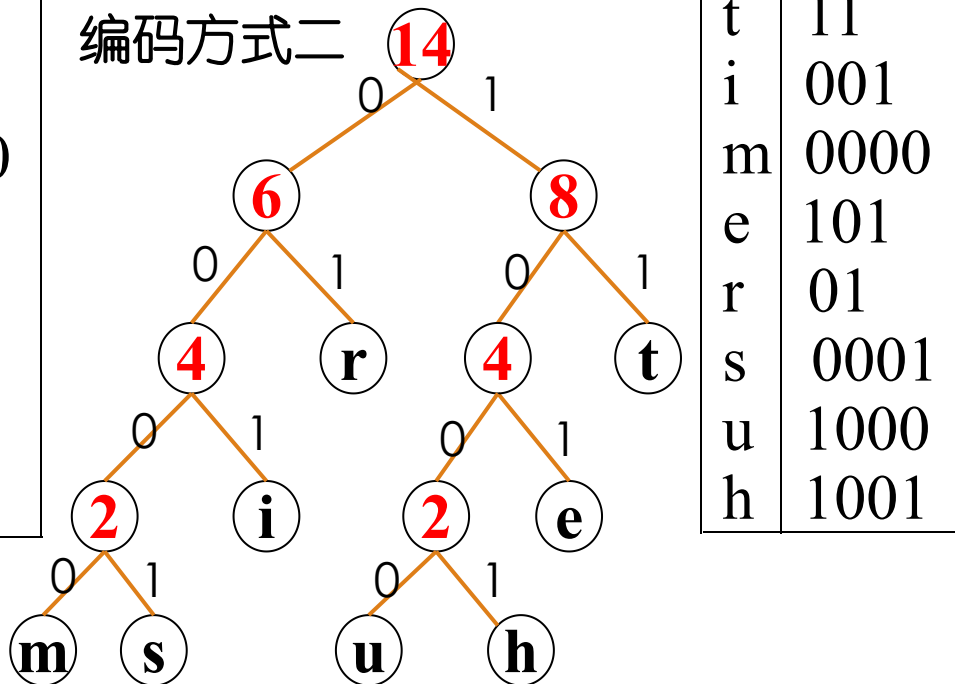
字符集 $D = \{t, i, m, e, r, s, u, h\}$

出现频次 $W=\{4, 2, 1, 2, 2, 1, 1, 1\}$

编码方式一



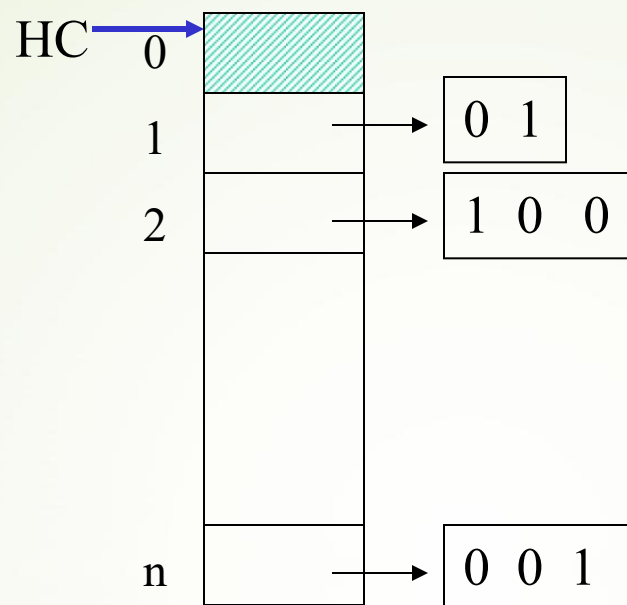
编码方式二



两个方式WPL均为40

赫夫曼编码码表构造

赫夫曼树



```
typedef char * * HuffmanCode;
```

```
HuffmanCode HC;
```

```
//HC是一个指针类型变量，
```

```
//先进行一次分配内存构造n个char*类型数据元素的线性表
```

```
//再对每个线性表的数据元素分配内存构造编码串
```

```
//使用完毕释放时，先依次释放每个数据元素，再释放表
```

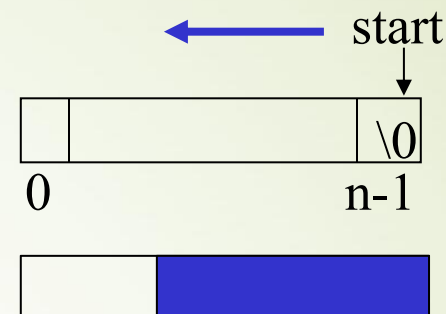
赫夫曼编码码表构造-算法步骤

赫夫曼树

主数据结构：赫夫曼树HT

赫夫曼编码表HC

辅助数据结构：cd {编码工作空间}



- 1) 分配HC这个线性表的空间;
- 2) 分配cd空间（大小为n），置 $cd[n-1] = "\0"$;
- 3) 依次求叶子HT[i]的编码HC[i], $i=1..n$
 - 3.1) 置cd中记录编码位置的初值: $start=n-1$;
置上溯的起点: $c=i$;
取c的父结点: $f=HT[c].parent$;
 - 3.2) 根据HT, c、f上溯直到根结点($f=0$), 依次产生的编码(c是f的左孩子编码为0; 否则为1)置入cd的相应编码空间 $cd[start-1]$ 并调整start(减1)
 - 3.3) 为第i个字符编码申请 $(n-start)$ 大小的字符空间HC[i], cd复制给HC[i]
- 4) 释放cd空间

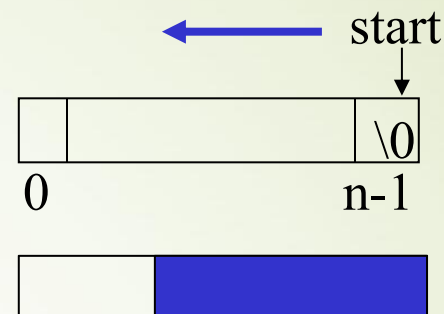
赫夫曼编码表构造-算法步骤

赫夫曼树

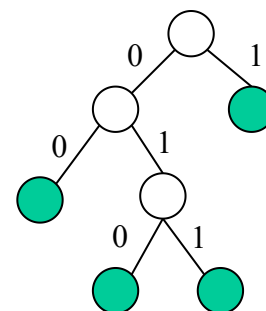
主数据结构：赫夫曼树HT

赫夫曼编码表HC

辅助数据结构：cd {编码工作空间}



```
void HuffmanCoding(HuffmanTree HT, int n, HuffmanCode &HC)
// 求赫夫曼树HT的n个字符的赫夫曼编码HC
{
    HC=(HuffmanCode)malloc((n+1)sizeof(char *));
    cd=(char *) malloc(n*sizeof(char)) ; //分配编码的工作空间
    cd[n-1]=0 ;
    for (i=1; i<=n; ++i){ //为每个字符求编码
        start=n-1;
        for (c=i, f=HT[i].parent; f!=0; c=f, f=HT[f].parent ) {
            if(HT[f].lchild==c) cd[--start]='0';
            else cd[--start]='1';
        }
        HC[i]= (char *)malloc((n-start)*sizeof(char));
        strcpy(HC[i], &cd[start]);
    }
    free(cd);
} //HuffmanCoding
```



线索二叉树---概念

线索二叉树

6.4.1 引入线索二叉树的意义

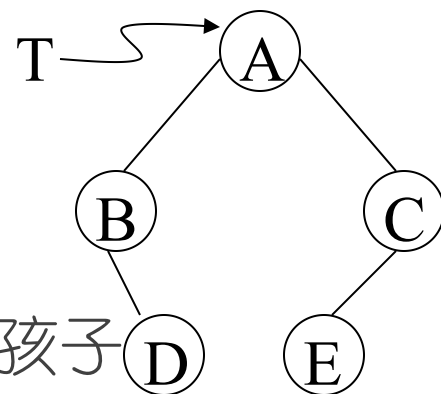
利用二叉链表的空指针域，建立指向该结点的前驱/后继(某种遍历方式下)结点的指针，方便二叉树的线性化使用。
(n 个结点的二叉树含有 $n+1$ 空指针域)

6.4.2 线索二叉树的存储结构

LTag	lchild d	data	rchil d	RTag
------	-------------	------	------------	------

左特征位 LTag = $\begin{cases} 0 & \text{lchild域指示结点的左孩子} \\ 1 & \text{lchild域指示结点的前驱结点} \end{cases}$

右特征位 RTag = $\begin{cases} 0 & \text{rchild域指示结点的右孩子} \\ 1 & \text{rchild域指示结点的后继结点} \end{cases}$



线索二叉树---以中序线索化后的中序遍历为例

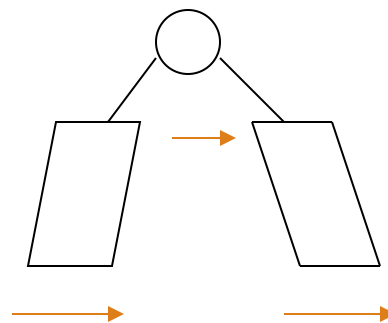
线索二叉树

二叉线索树的遍历不需要递归，在先序/中序线索二叉树中，利用线索可方便地找到当前结点的后继结点。因此，对有 n 个结点的二叉线索树，可以在 $O(n)$ 时间内遍历完该树。

[算法步骤]

若二叉树 p 非空，

- 1) 找到中序序列的第一个结点；
- 2) 访问当前结点；
- 3) 将当前结点的后继作为新的当前结点
- 4) 当前结点存在，则转2)



— p 的后继

若 $p \rightarrow RTag = 1$ ，则 $p \rightarrow rchild$ 即为所求；

若 $p \rightarrow RTag = 0$ ，则从其右子沿着左链走到 $LTag = 1$ 的那个结点就是。(右子树的最左的左孩子)

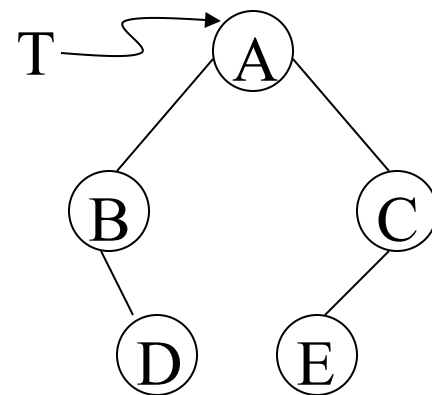
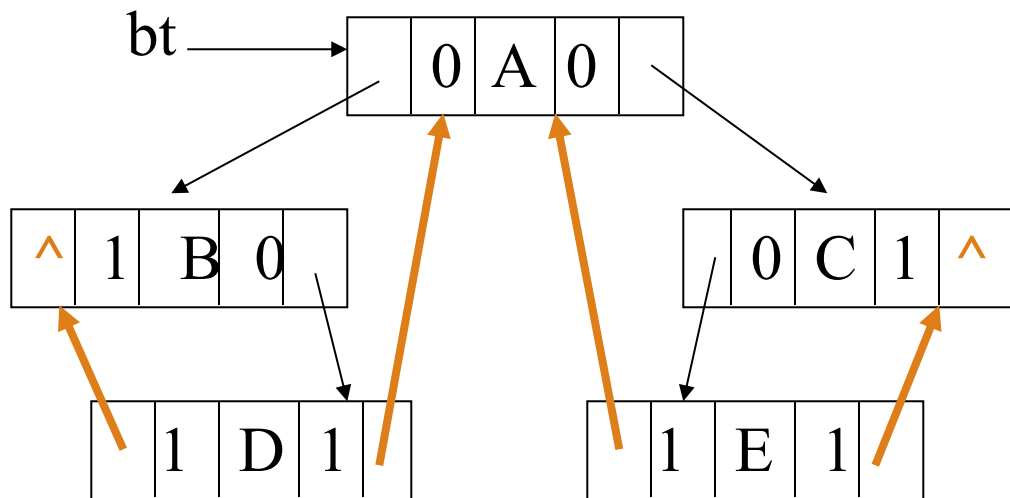
线索二叉树---数据结构定义

线索
二叉树

[例]

中序线索二叉树

BDAEC



[类型定义]

```
typedef enum PointerTag {Link, Thread};  
// Link==0: 指针, Thread==1: 线索  
typedef struct BiThrNode {  
    TElemType data;  
    Struct BiThrNode *lchild, *rchild;  
    PointerTag LTag, RTag;  
}BiThrNode, *BiThrTree;
```

6.4.3.1 查找结点p的前驱 /后继结点

(1) 中序线索二叉树

– p的后继

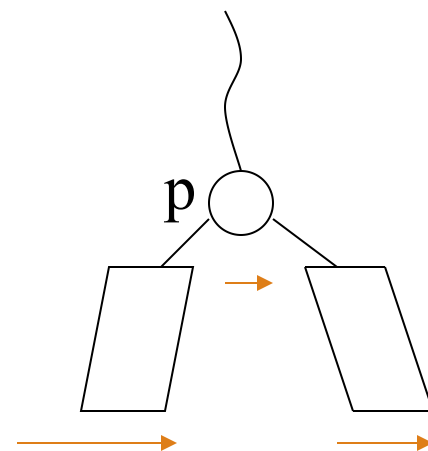
若 $p \rightarrow RTag = 1$ ，则 $p \rightarrow rchild$ 即为所求；

若 $p \rightarrow RTag = 0$ ，则从其右子沿着左链走到 $LTag = 1$ 的那个结点就是。(右子树的最左的左孩子)

– p的前驱

若 $p \rightarrow LTag = 1$ ，则 $p \rightarrow lchild$ 即为所求；

若 $p \rightarrow LTag = 0$ ，则从其左子沿着右链走到 $RTag = 1$ 的那个结点就是。(左子树的最右的右孩子)



L D R
LDR LDR

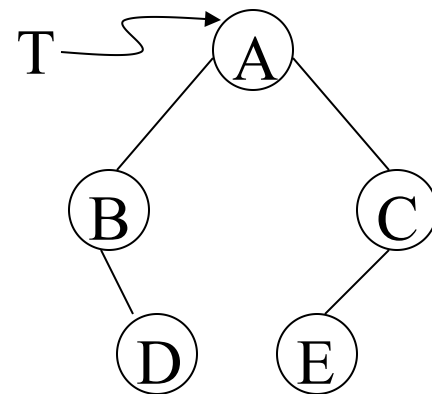
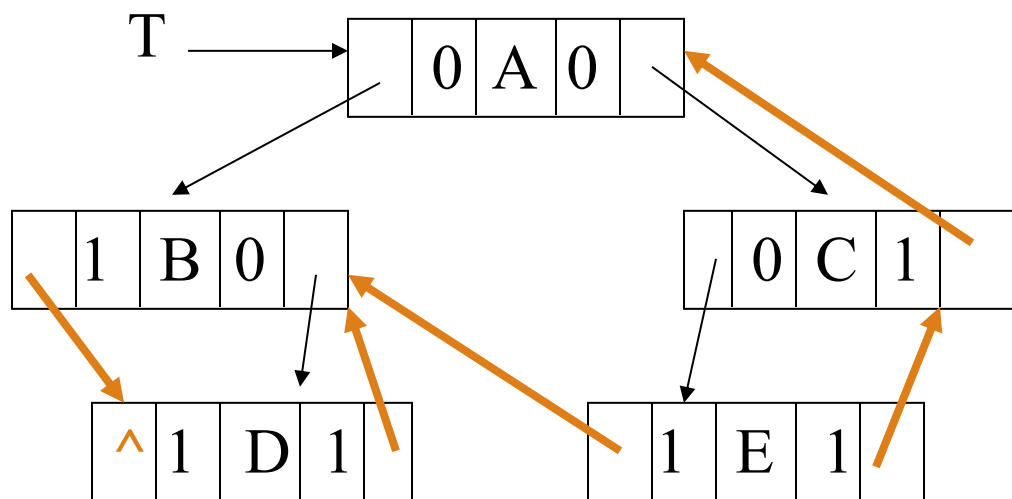
线索二叉树---数据结构定义

线索
二叉树

[例]

后序线索二叉树

DBECA



[类型定义]

```
typedef enum PointerTag {Link, Thread};  
// Link==0: 指针, Thread==1: 线索  
typedef struct BiThrNode {  
    TElemType data;  
    Struct BiThrNode *lchild, *rchild;  
    PointerTag LTag, RTag;  
}BiThrNode, *BiThrTree;
```

线索二叉树---操作(查找前驱、后继)

线索
二叉树

(2) 后序线索二叉树

– p的前驱

若 $p \rightarrow \text{LTag} = 1$, 则 $p \rightarrow \text{lchild}$ 即为所求;

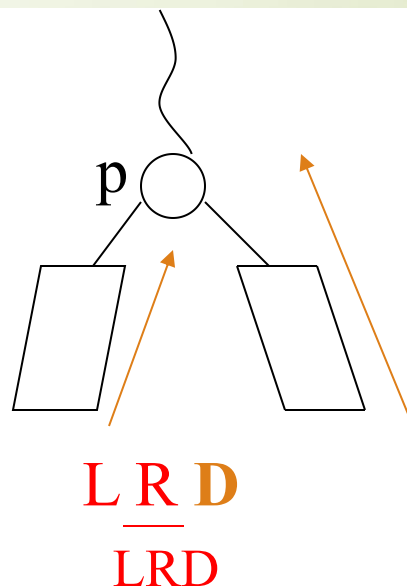
若 $p \rightarrow \text{LTag} = 0$,

则若 $p \rightarrow \text{RTag} = 0$, 则 $p \rightarrow \text{rchild}$ 即为所求

若 $p \rightarrow \text{RTag} = 1$, 则 $p \rightarrow \text{lchild}$ 即为所求

– p的后继

与双亲结点有关, 因二叉链表中没有指向双亲结点的指针, 就可能需通过二叉树的后序遍历才可确定, 因而后序线索二叉树在此问题上并不比普通二叉树有效。



(3) 先序线索二叉树

与后序线索二叉树对偶

中序线索二叉树可以方便地查找一个结点的前驱/后继结点。

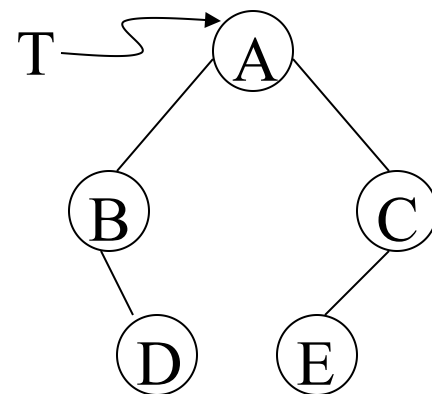
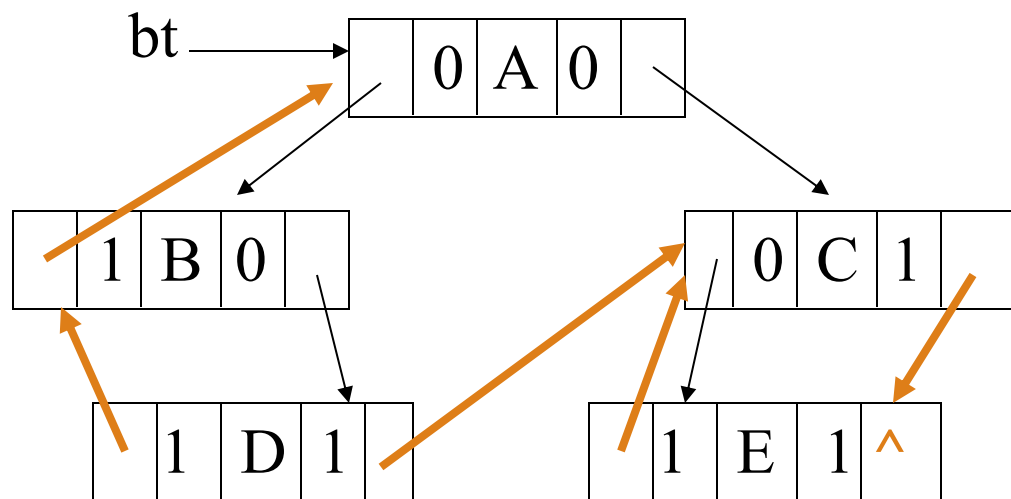
线索二叉树---数据结构定义

线索
二叉树

[例]

先序线索二叉树

ABDCE



[类型定义]

```
typedef enum PointerTag {Link, Thread};  
// Link==0: 指针, Thread==1: 线索  
typedef struct BiThrNode {  
    TElemType data;  
    Struct BiThrNode *lchild, *rchild;  
    PointerTag LTag, RTag;  
}BiThrNode, *BiThrTree;
```

线索二叉树---中序线索化

线索二叉树

遍历:

—p的后继

若 $p \rightarrow RTag = 1$, 则 $p \rightarrow rchild$ 即为所求;

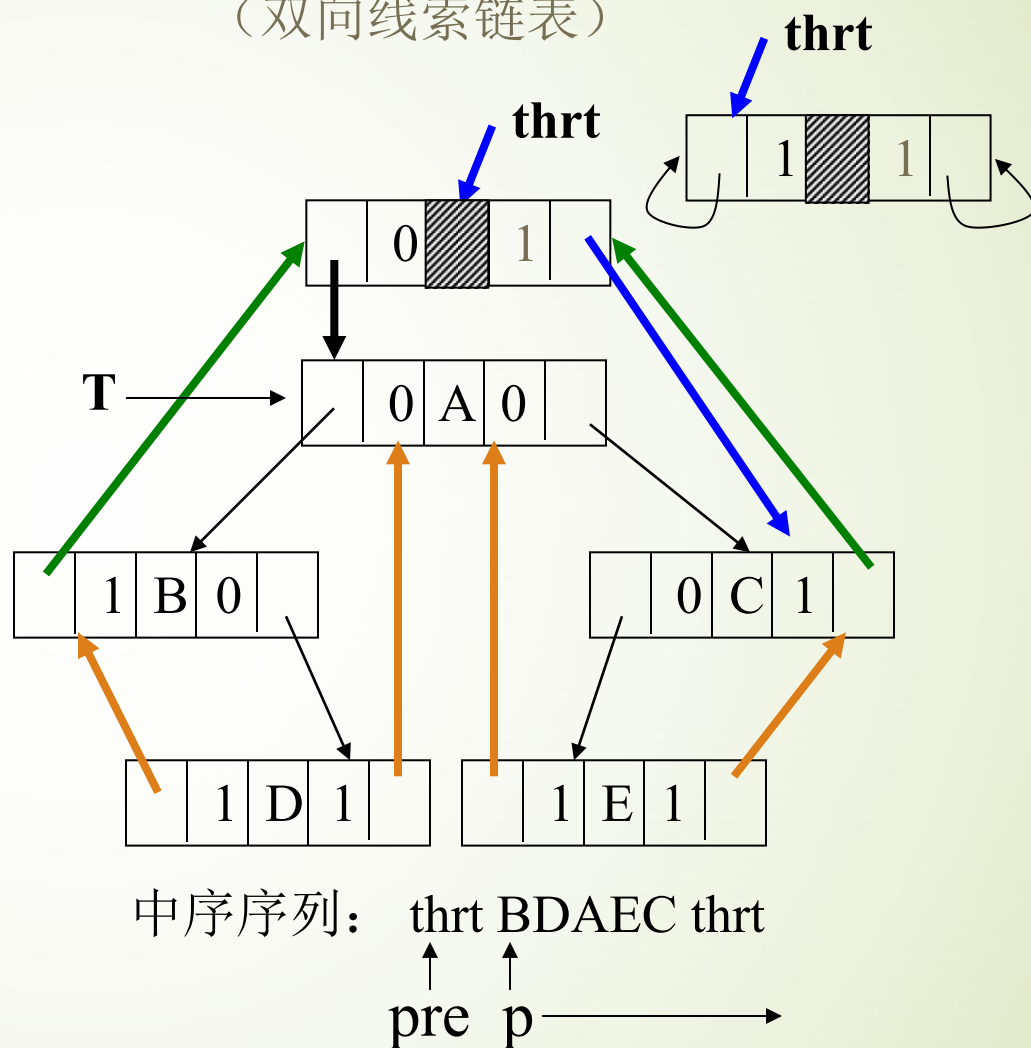
若 $p \rightarrow RTag = 0$, 则从其右子沿着左链走到 $LTag = 1$ 的那个结点就是。(右子树的最左的左孩子)

—p的前驱

若 $p \rightarrow LTag = 1$, 则 $p \rightarrow lchild$ 即为所求;

若 $p \rightarrow LTag = 0$, 则从其左子沿着右链走到 $RTag = 1$ 的那个结点就是。(左子树的最右的右孩子)

带头结点的中序线索二叉树 (双向线索链表)



算法描述参教材P134-135 算法6.6、6.7

线索二叉树---中序线索化

线索
二叉树

[算法步骤]

若二叉树p非空，

1 左子树线索化；

2 访问当前结点p：

2.1 置 LTag和RTag；

2.2 建立p的前趋线索。

```
if ( ! p->lchild ) {  
    p->LTag=Thread; p->lchild=pre;  
}
```

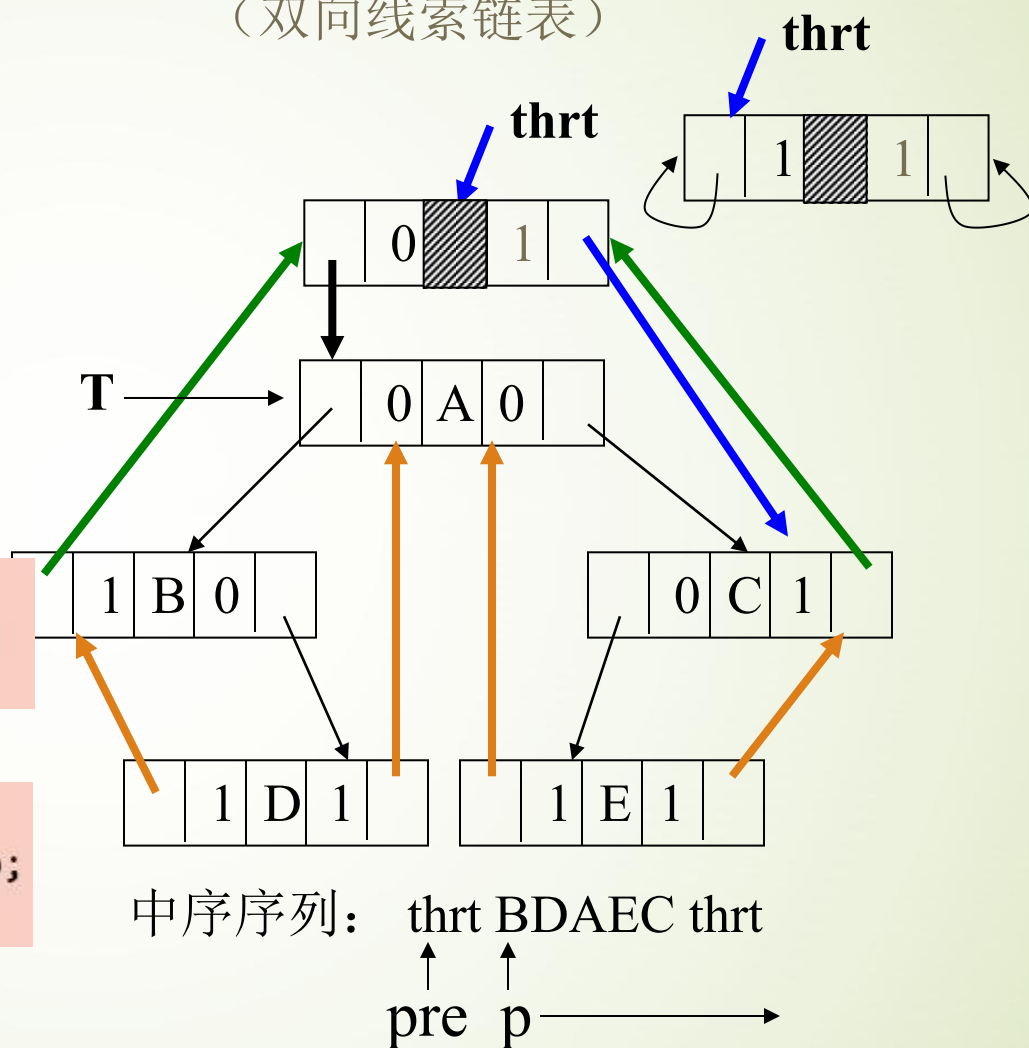
2.3 建立p的前趋pre的

```
if ( ! pre->rchild ) {  
    pre->RTag=Thread; pre->rchild=p;  
}
```

2.4 将p作为下一个被访问结点的前趋
`pre=p;`

3 右子树线索化；

带头结点的中序线索二叉树
(双向线索链表)



算法描述参教材P134-135 算法6.6、6.7

线索二叉树---中序线索化

线索
二叉树

```
BiThrTree pre;
```

```
Status InOrderThreading( BiThrTree &Thrt,  
                          BiThrTree T)
```

```
{ if(!(Thrt=(BiThrTree)malloc(sizeof(BiThrNode))))
```

```
    exit(OVERFLOW);
```

```
Thrt->LTag=Link;
```

```
Thrt->Rtag=Thread;
```

```
Thrt->rchild=Thrt; // 中序遍历的最后一个结点
```

```
if (!T){
```

```
    Thrt->LTag=Thread;
```

```
    Thrt->lchild= T;
```

```
}
```

```
else{
```

```
    Thrt->lchild=T;
```

```
    InThreading(T);
```

```
    pre->Rtag=Thread;
```

```
    pre->rchild=Thrt;
```

```
    Thrt->rchild=pre;
```

```
}
```

```
return OK;
```

```
// InOrderThreading
```

```
void InThreading( BiThrTree p)
```

```
{ if (p) {
```

```
    InThreading(p->lchild);
```

```
    p->LTag=p->Rtag=Link;
```

```
    if ( ! p->lchild) {
```

```
        p->LTag=Thread;  p->lchild=pre;
```

```
    }
```

```
    if ( ! pre->rchild) {
```

```
        pre->Rtag=Thread;  pre->rchild=p;
```

```
    }
```

```
    pre=p;
```

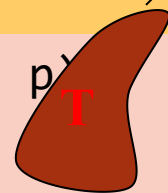
```
    InThreading(p->rchild);
```

```
    }//if(p)
```

```
// InThreading
```

P135-6.7

Thrt →



线索二叉树---中序线索化后的中序遍历

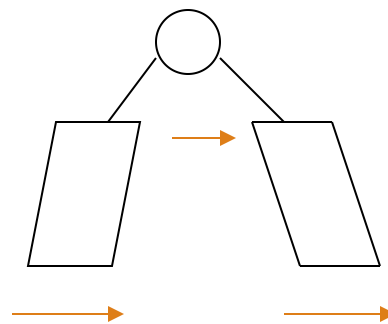
线索 二叉树

二叉线索树的遍历不需要递归，在先序/中序线索二叉树中，利用线索可方便地找到当前结点的后继结点。因此，对有 n 个结点的二叉线索树，可以在 $O(n)$ 时间内遍历完该树。

[算法步骤]

若二叉树 p 非空，

- 1) 找到中序序列的第一个结点；
- 2) 访问当前结点；
- 3) 将当前结点的后继作为新的当前结点
- 4) 当前结点存在，则转2)



- p 的后继

若 $p \rightarrow RTag = 1$ ，则 $p \rightarrow rchild$ 即为所求；

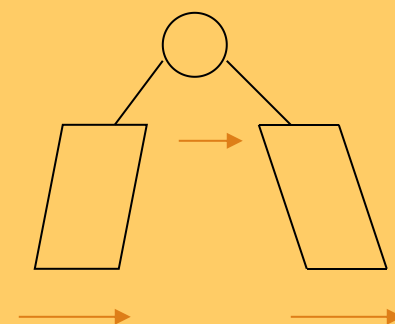
若 $p \rightarrow RTag = 0$ ，则从其右子沿着左链走到 $RTag = 1$ 的那个结点就是。(右子树的最左的左孩子)

线索二叉树---中序线索化后的中序遍历

线索
二叉树

[算法描述]

```
Status InOrderTraverse_Thr(BiThrTree T,  
                           Status (* Visit)(TElemType e) )  
{  
    p=T->lchild;  
    while( p!=T )  
    {  
        while ( p->LTag==Link)  
            p=p->lchild;  
        if ( !Visit(p->data) )  
            return ERROR;  
        while (p->Rtag==Thread && p->rchild!=T)  
        {  
            p=p->rchild;  Visit(p->data);  
        }  
        p=p->rchild;  
    }//while(p!=T)  
    return OK;  
}// InOrderTraverse_Thr
```



线索二叉树---二叉树的结点插入删除后

线索 二叉树

*6.4.3.4 线索二叉树的更新操作

【问题点】

在线索二叉树上插入或删除一棵子树或者结点时，指针和线索都需作相应的改动。

【解决方法】

- 1) 还原为普通二叉树，再重新将其线索化。
- 2) 讨论可能出现的各种情况，寻找规律。

```
typedef char TElemType;
typedef enum PointerTag {Link, Thread};
// Link==0: 指针, Thread==1: 线索
typedef struct BiThrNode {
    TElemType data;
    struct BiThrNode *lchild, *rchild;
    PointerTag LTag, RTag;
}BiThrNode, *BiThrTree;
```

```
BiThrNode biThr;
```

```
cout << "sizeof(BiThrNode) : " << sizeof(BiThrNode) << "\n"
      << "sizeof(PointerTag) : " << sizeof(PointerTag) << "\n"
      << "sizeof(BiThrTree) : " << sizeof(BiThrTree) << "\n"
      << "sizeof(int) : " << sizeof(int) << "\n";
```

32位未压缩编译情况下:

```
sizeof(BiThrNode) : 20
sizeof(PointerTag) : 4
sizeof(BiThrTree) : 4
sizeof(int) : 4
```

64位未压缩编译情况下:

```
sizeof(BiThrNode) : 32
sizeof(PointerTag) : 4
sizeof(BiThrTree) : 8
sizeof(int) : 4
```

在现代计算机的大内存情况下, 不使用Ltag和Rtag, 直接使用前驱指针和后继指针更方便

#pragma pack

编译指令：这条指令主要用作改变**编译器**的默认对齐方式。

#pragma pack(show) //显示当前内存对齐字节数，编译器默认按当前环境位数对准

#pragma pack(n) //设置编辑器按照n个字节对齐，n可以取值

1,2,4,8,16

#pragma pack(push) //将当前的对齐字节数压入栈顶，不改变对齐字节数

#pragma pack(push,n) //将当前的对齐字节数压入栈顶，并按照n字节对齐

#pragma pack(pop) //弹出栈顶对齐字节数，不改变对齐字节数

#pragma pack(pop,n) //弹出栈顶并直接丢弃，按照n字节对齐

例子：

#pragma pack(push) //保存下面行代码编译前的pack设置

#pragma pack(1) //设置下列代码按照内存1个字节对齐

typedef struct //声明结构体，此结构体在编译时按1字节对齐

{

{

char SeriesNumber[16]; //结构体成员

unsigned byte minDelta; //结构体成员

float FilterGauss; //结构体成员

} Param; //结构体名称为Param

#pragma pack(pop) //恢复之前的字节对齐设置

默认32位未压缩
 $4+4*2+4*2=20$
字节

按1字节对准后
32位环境
 $1+4*2+4*2=17$
字节

```
typedef char TElemType;
typedef enum PointerTag {Link, Thread};
// Link==0: 指针, Thread==1: 线索
typedef struct BiThrNode {
    TElemType data;
    struct BiThrNode *lchild, *rchild;
    PointerTag LTag, RTag;
}BiThrNode, *BiThrTree;
```

默认64位未压缩
 $8+8*2+8=32$
字节

按1字节对照后
64位环境
 $1+8*2+4*2=25$
字节

```
BiThrNode biThr;
cout << "sizeof(BiThrNode) : " << sizeof(BiThrNode) << "\n"
    << "sizeof(PointerTag) : " << sizeof(PointerTag) << "\n"
    << "sizeof(BiThrTree) : " << sizeof(BiThrTree) << "\n"
    << "sizeof(int) : " << sizeof(int) << "\n";
```

32位未压缩编译情况下:

```
sizeof(BiThrNode) : 20
sizeof(PointerTag) : 4
sizeof(BiThrTree) : 4
sizeof(int) : 4
```

64位未压缩编译情况下:

```
sizeof(BiThrNode) : 32
sizeof(PointerTag) : 4
sizeof(BiThrTree) : 8
sizeof(int) : 4
```

在现代计算机的大内存情况下, 不使用Ltag和Rtag,
直接使用前驱指针和后继指针更方便

树和森林

树 森林

普通树（多叉树）若不转化为二叉树，则运算很难实现

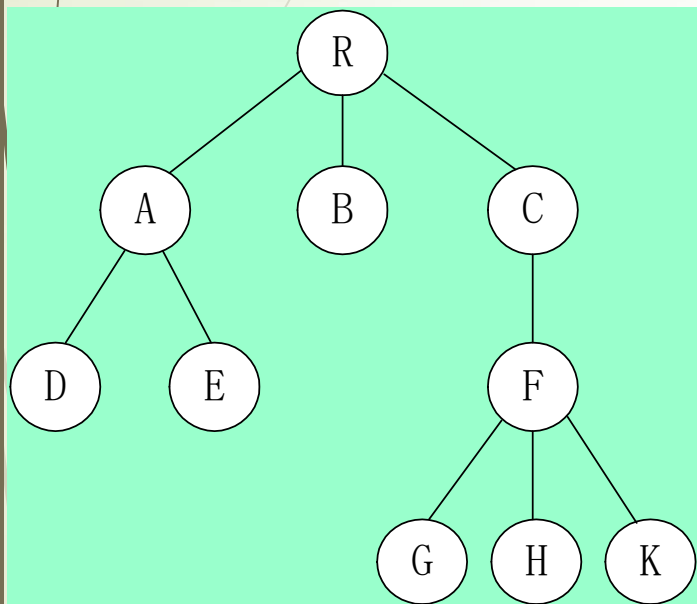
为何要重点研究每结点最多只有两个“叉”的树？

- ✓ 二叉树的结构最简单，规律性最强；
- ✓ 可以证明，所有树都能转为唯一对应的二叉树，不失一般性。
- ✓ 如何转化？

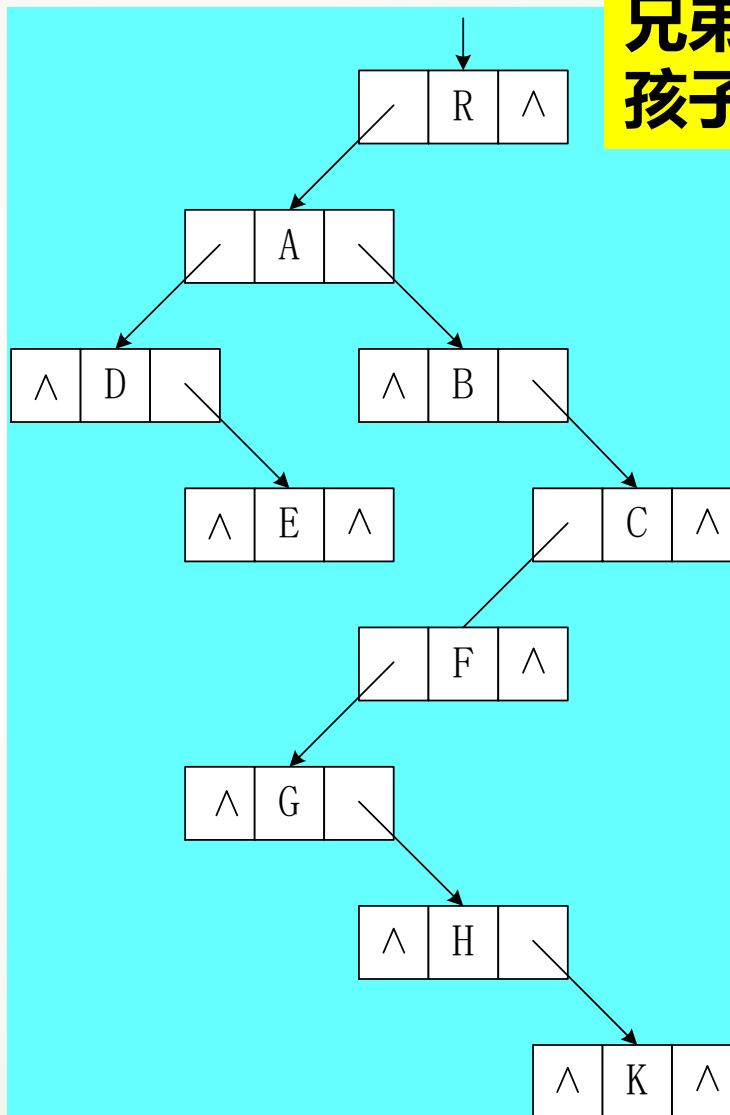
树到二叉树方法：
兄弟在右
孩子在左

树到二叉树的存储结构——二叉链表表示法

树 森林

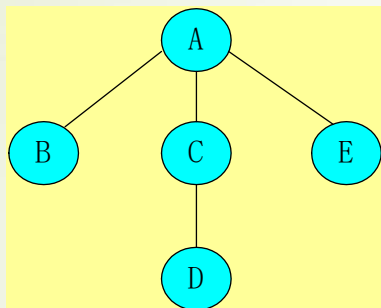


树到二叉树方法：
兄弟在右
孩子在左



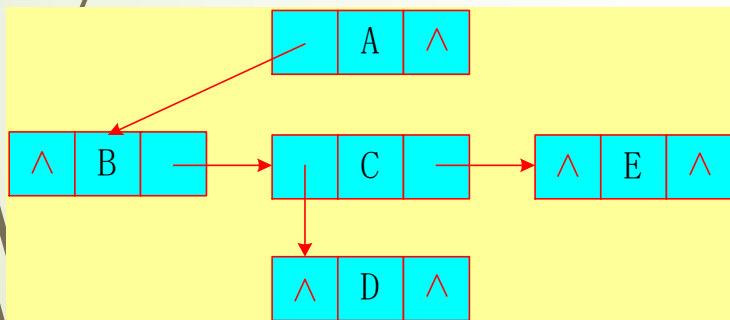
树森林

树



存储

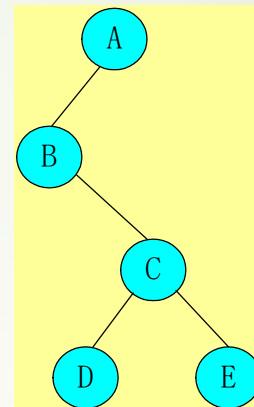
解释



对应

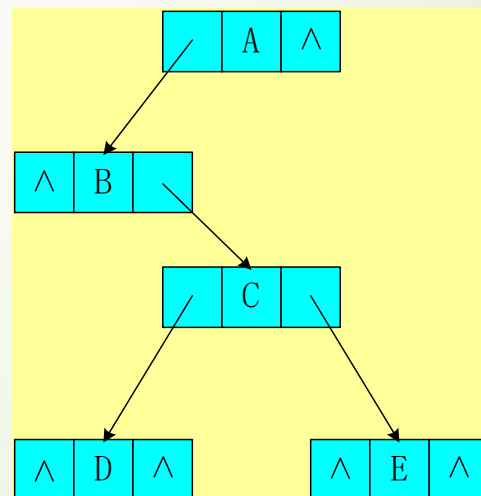


二叉树



存储

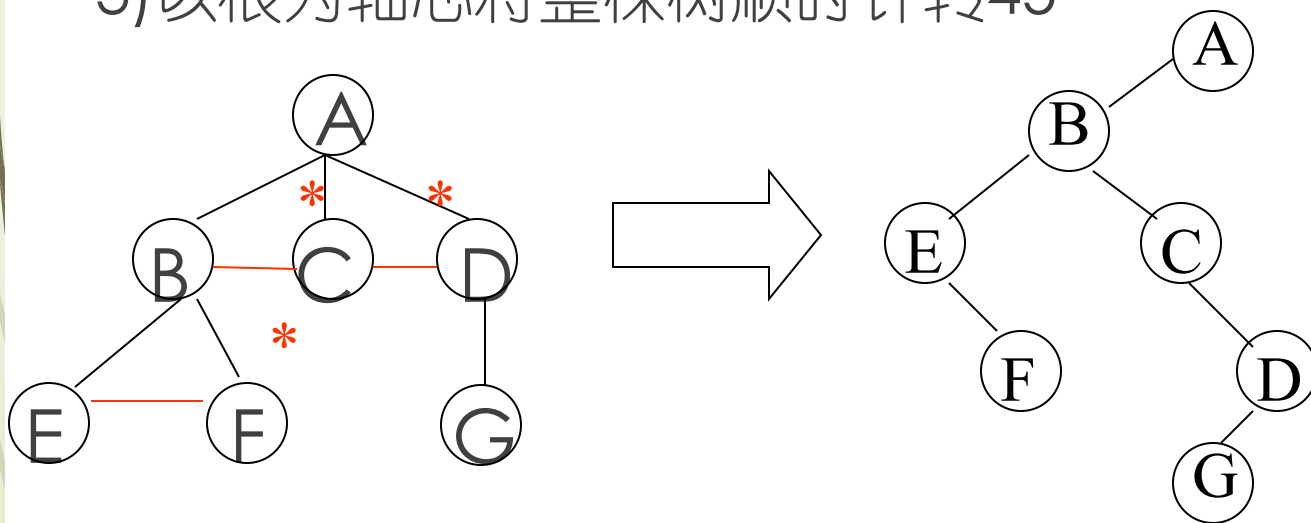
解释



森林与二叉树的转换

[树转换为二叉树]

- 1) 在兄弟间加一连线
- 2) 对每一结点，去掉它与孩子的连线(最左子除外)
- 3) 以根为轴心将整棵树顺时针转 45°



特点:

根无右子树

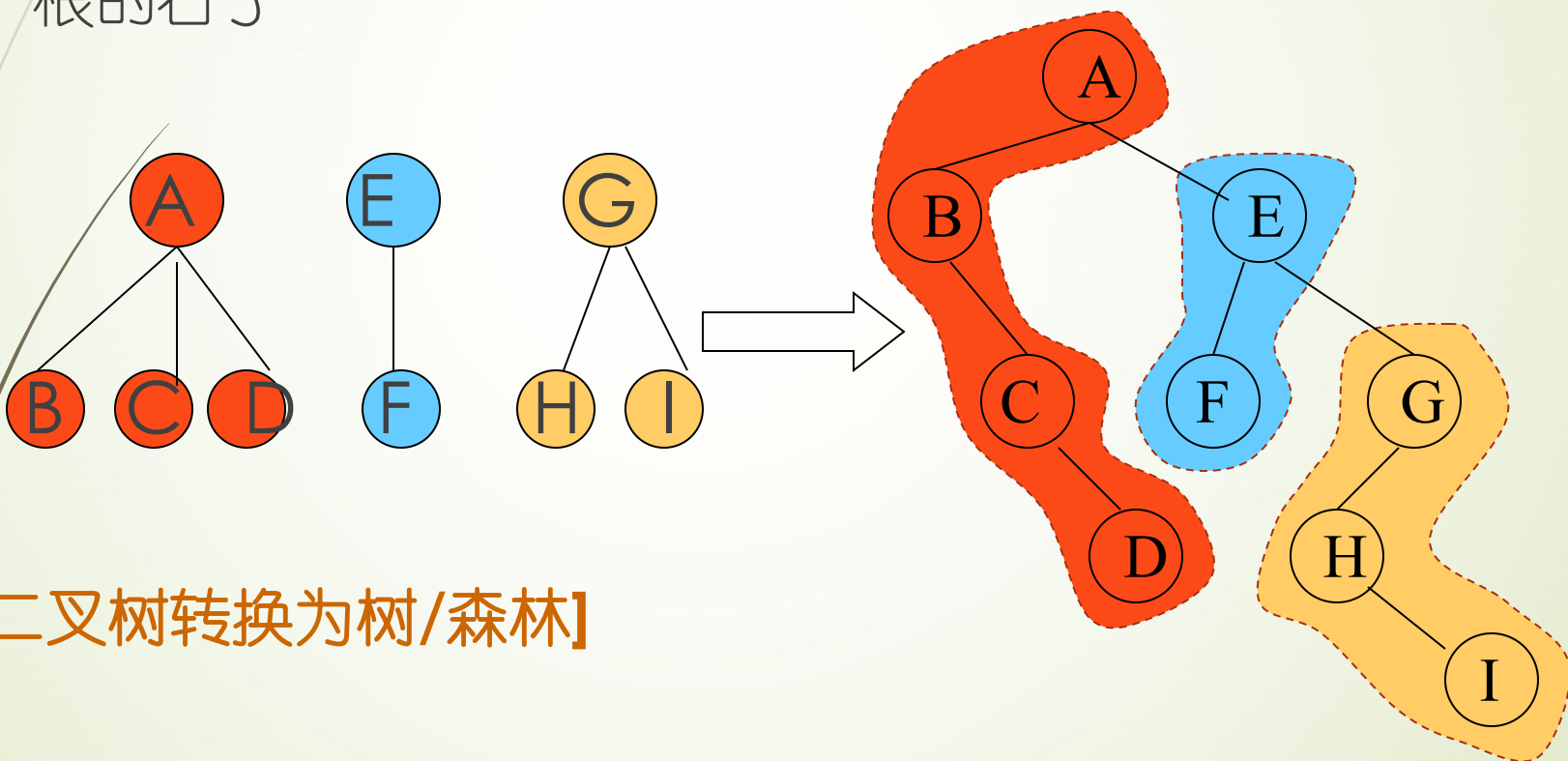
左支是孩子

右支是兄弟

树森林

[森林转换为二叉树]

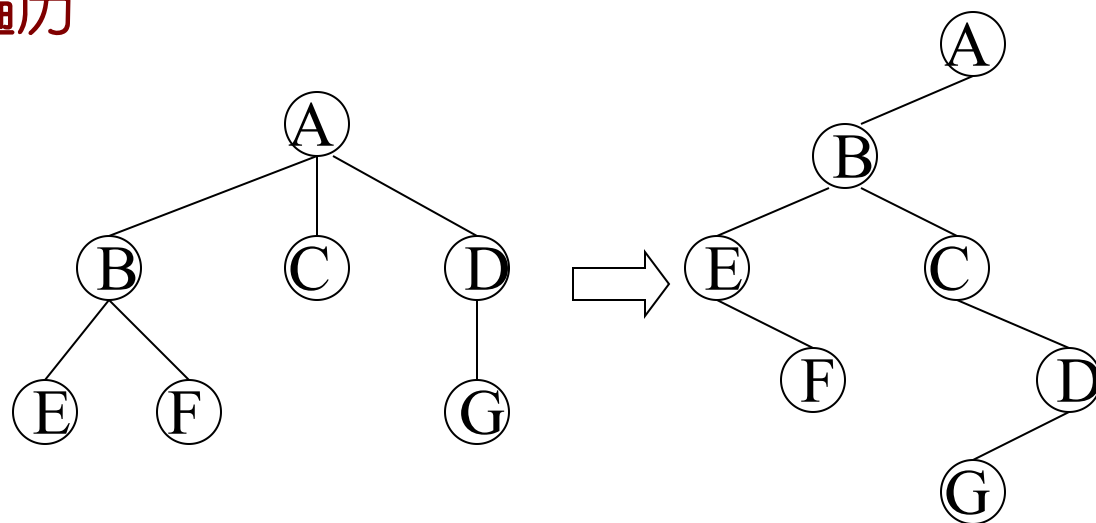
- 1) 先将森林里的每一棵树转换成一棵二叉树
- 2) 从最后一棵树开始，把后一棵树的作为前一棵树的根的右子



[二叉树转换为树/森林]

树和森林的遍历

树的遍历



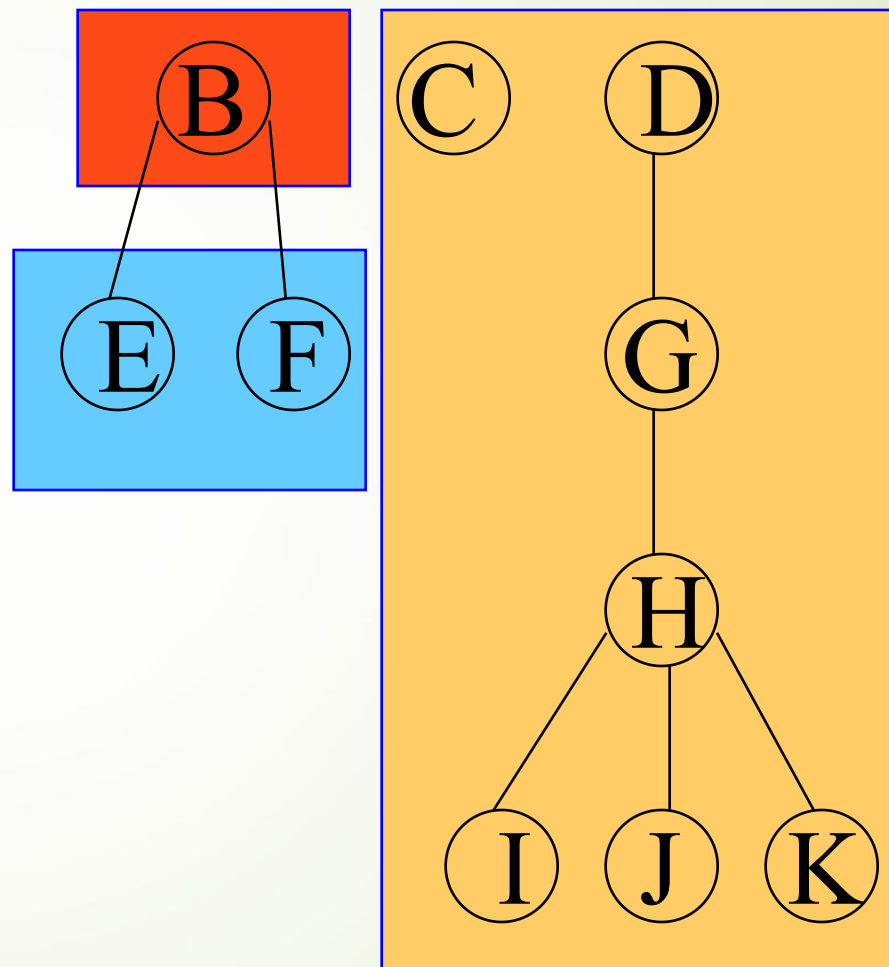
- 先(根)序遍历** 先访问树的根结点，然后依次先根遍历根的每棵子树
 AB EFC D G (二叉树先序)
- 后(根)序遍历** 先依次后根遍历根的每棵子树，然后访问树的根结点
 EFBC GDA (二叉树中序)
- 层次遍历** 若树不空，则自上而下自左至右访问树中每个结点
 ABCDEFG

6.6.2.2 森林的遍历

树 森林

可以分解成三部分：

1. 森林中第一棵树的根结点；
2. 森林中第一棵树的子树森林；
3. 森林中其它树构成的森林



6.6.2.2 森林的遍历

树 森林

➡ 先序遍历

访问第一棵树的根结点；
先序遍历第一棵树的根
的子树森林；

先序遍历除第一棵树外
剩余的树构成的森林

(逐棵先序遍历每棵子树/
对应二叉树的先序遍历)

➡ 中序遍历

中序遍历第一棵树的根
的子树森林；

访问第一棵树的根结点；
中序遍历除第一棵树外
剩余的树构成的森林

(逐棵后序遍历每棵子树/
对应二叉树的中序遍历)

6.6.3 树的存储结构

[多重链表表示法]

结点结构

data	child ₁	child ₂	...	child _m
------	--------------------	--------------------	-----	--------------------

数据域 指针域 (m: 树的度)

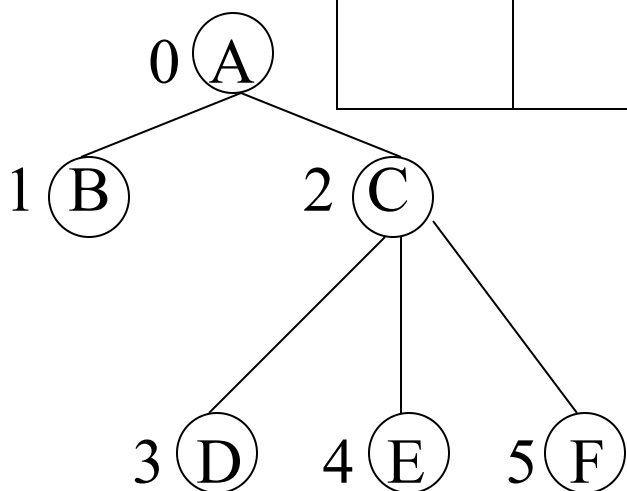
```
#define    TREE_DEGREE    100

typedef struct MultiTNode {
    TElemType  data;
    struct MultiTNode * ptr[TREE_DEGREE]
}MultiTNode, * MultiTree;
```

[双亲表示法] 用顺序存储（一维数组）存储树的信息

结点序号 data parent

0	A	-1
1	B	0
2	C	0
3	D	2
4	E	2
5	F	2



```
#define MAX_TREE_SIZE 100

typedef struct PTNode{
    TElemType    data;
    int           parent;
}PTNode;

typedef struct {
    PTNode    nodes[MAX_TREE_SIZE];
    int       r, n; //根的位置和结点数
}PTree;
```

[孩子链表表示法] 顺序（一维数组）+单链表

树 森林

结点下标	data	firstchild
0	A	—→ 1
1	B	^
2	C	—→ 3
3	D	^
4	E	^
5	F	^

1 (B)

```
#define MAX_TREE_SIZE 100
typedef struct CTNode{//孩子结点
    int    child;
    struct CTNode * next;
}CTNode, * ChildPtr;
typedef struct
{
    //向量结点
    TElemType  data;
    ChildPtr   firstchild;
}CTBox;
typedef struct {
    CTBox  nodes[MAX_TREE_SIZE];
    int    r,  n; //根的位置和结点数
}CTree;
```

[孩子兄弟表示法(二叉链表表示法)]

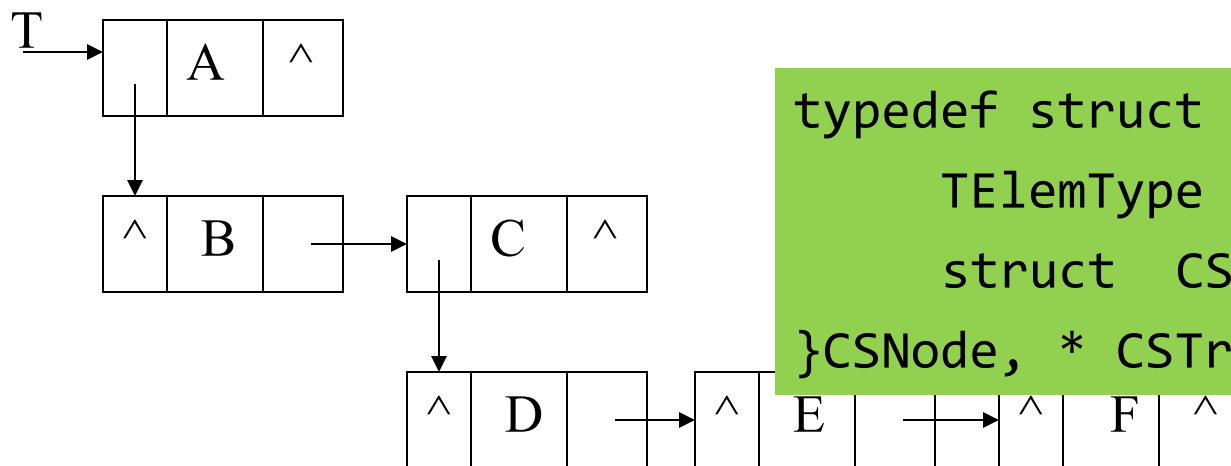
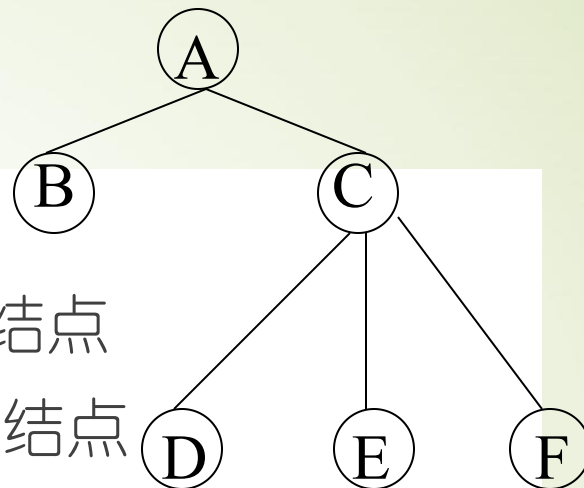
树 森林

结点结构

fch	data	nsib
-----	------	------

fch: 指向该结点的第一个孩子结点

nsib: 指向该结点的下一个兄弟结点



```
typedef struct CSNode {  
    TElemType data;  
    struct CSNode *fch,*nsib;  
}CSNode, * CSTree ;
```

访问某个结点的所有孩子结点：从fch链找到第一个孩子，沿第一个孩子的nsib链找第二个孩子，, 直至nsib链为空。

树的存储结构建立算法示例

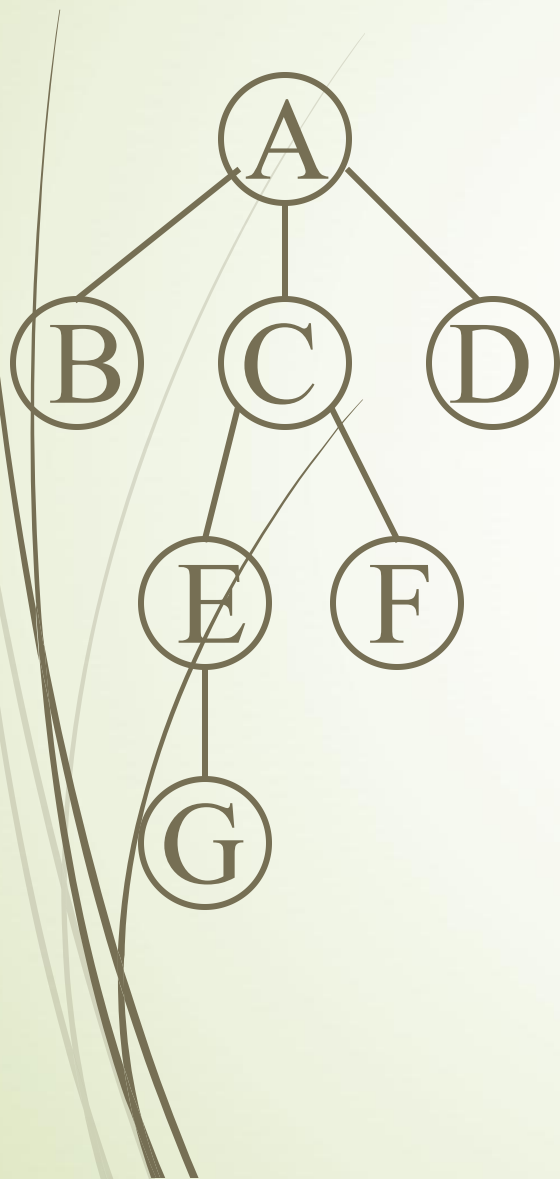
和二叉树类似，不同的存储结构定义有不同的算法。

假设以二元组(F,C)的形式自上而下、自左而右依次输入树的各边，建立树的**孩子-兄弟链表**。

```
typedef struct CSNode{  
    Elem    data;  
    struct CSNode *firstchild, *nextsibling;  
} CSNode, *CSTree;
```

例如：对下列所示树的输入序列应为：

树森林



(‘#’, ‘A’)

(‘A’, ‘B’)

(‘A’, ‘C’)

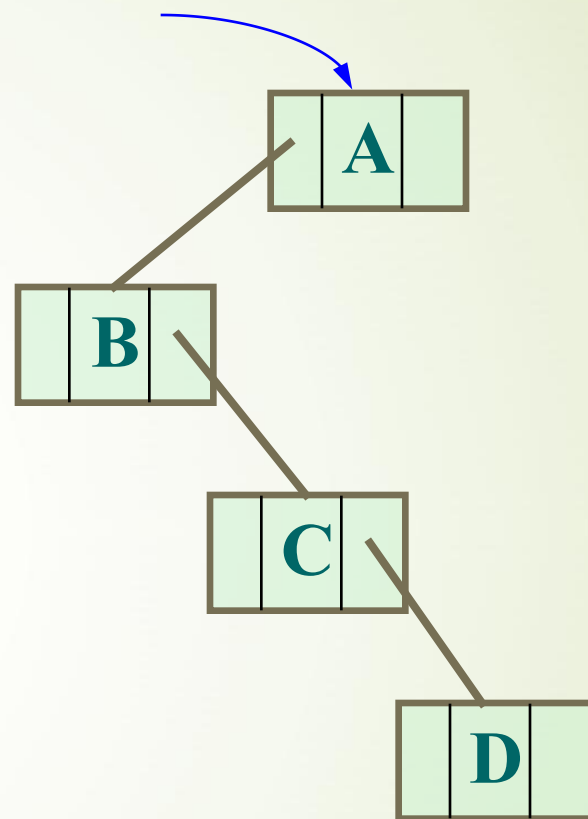
(‘A’, ‘D’)

(‘C’, ‘E’)

(‘C’, ‘F’)

(‘E’, ‘G’)

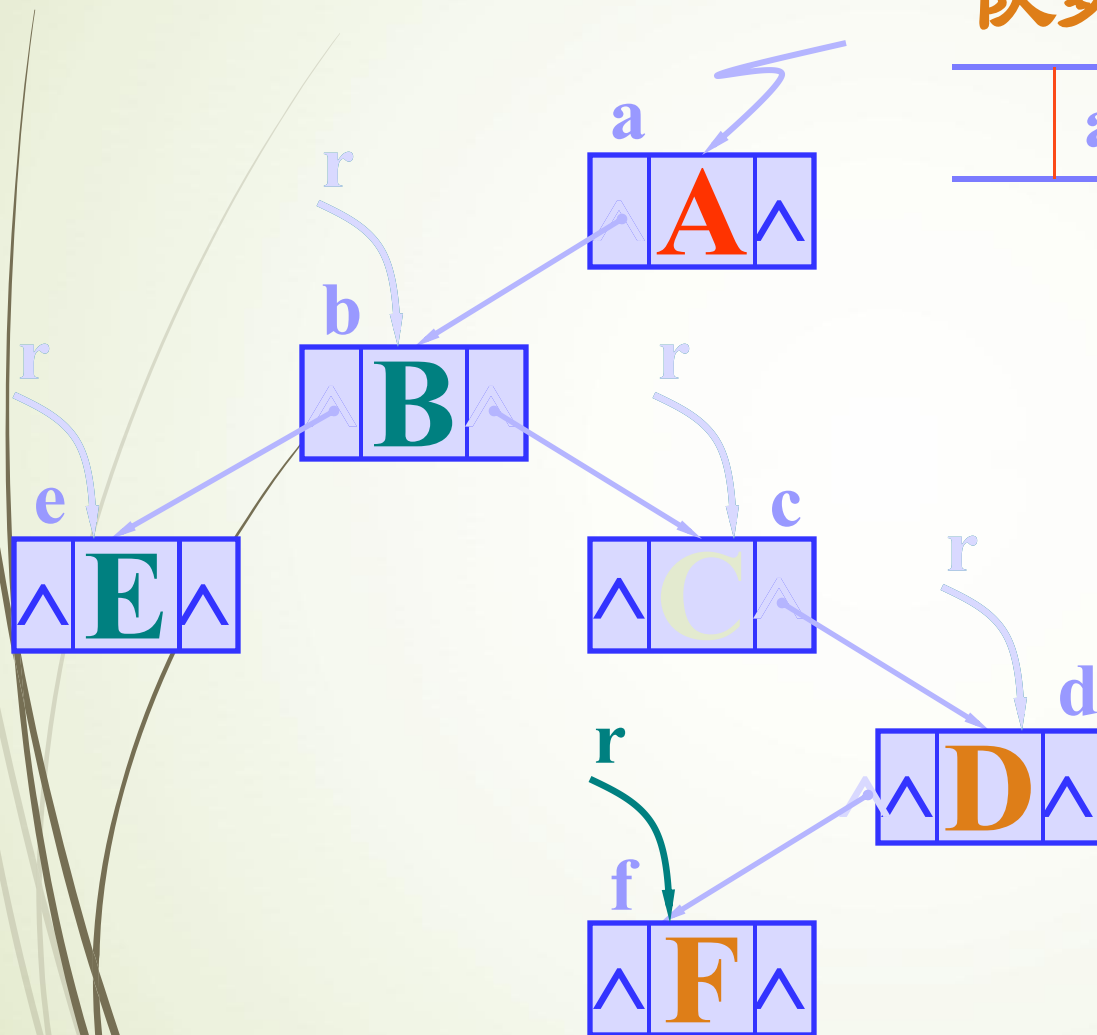
(‘#’, ‘#’)



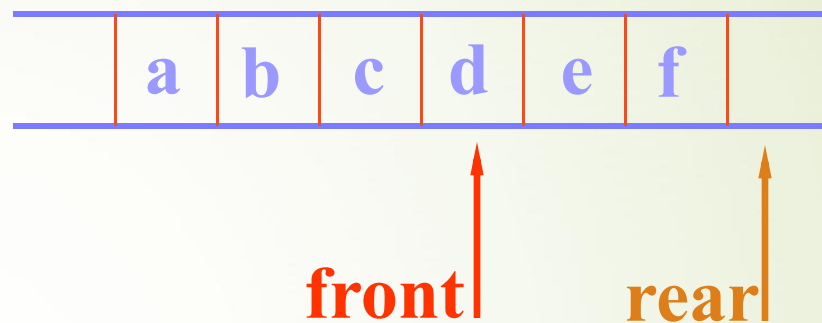
可见，算法中需要一个队列保存已建好的结点的指针

(A,B), (A,C),(A,D),(B,E),(D,F),(D,G),...

树森林



队列



```
void CreatTree( CStree &T ) {  
    T = NULL;  
    for( scanf(&fa, &ch); ch!='#'; scanf(&fa, &ch); )  
    {  
        p = GetTreeNode(ch);    // 创建结点  
        EnQueue(Q, p); // 指针入队列  
        if (fa == '#') T = p;    // 所建为根结点  
        else { ..... }        // 非根结点的情况  
    } // for  
} // Creat
```

```
    GetHead(Q,s); // 取队列头元素(指针值)  
    while (s->data != fa ) { // 查询双亲结点  
        DeQueue(Q,s);  
        GetHead(Q,s);  
    }  
    if (!(s->firstchild))  
        s->firstchild = p; // 链接第一个孩子结点  
    else  
        r->nextsibling = p; // 链接其它孩子结点  
    r=p;
```



本章学习要点

1. 熟练掌握二叉树的结构特性，了解相应的证明方法。
2. 熟悉二叉树的各种**存储结构**的特点及适用范围。
3. **遍历二叉树**是二叉树各种操作的基础。实现二叉树遍历的具体算法与所采用的存储结构有关。掌握**三种遍历策略的递归算法**，**灵活运用遍历算法实现二叉树的其它操作**。层次遍历是按另一种搜索策略进行的遍历。

4. 了解**最优树**的概念，掌握**建立最优二叉树和哈夫曼编码**的方法。

5. 熟悉**树的各种存储结构**及其特点，掌握**树和森林与二叉树的转换**方法。建立存储结构是进行其它操作的前提，因此应掌握 1 至 2 种**建立二叉树和树的存储结构**的方法。

6. 了解实现树的各种基本操作的方法。